REPUBLIQUE DU CAMEROUN
PAIX - TRAVAIL – PATRIE
****
UNIVERSITE DE DSCHANG
****
ÉCOLE DOCTORALE

REPUBLIC OF CAMEROON
PEACE - WORK - FATHERLAND
****
UNIVERSITY OF DSCHANG
****
POST GRADUATE SCHOOL

DSCHANG SCHOOL OF SCIENCES AND TECHNOLOGY

UNITE DE RECHERCHE EN INFORMATIQUE FONDAMENTALE, INGENIERIE ET APPLICATIONS (URIFIA)

SUJET :

INGÉNIERIE DES APPLICATIONS WEB PILOTÉES PAR DES MODÈLES BPMN

Mémoire soutenu publiquement en vue de l'obtention du

Diplôme de Master en Informatique

Option : Informatique Fondamentale
Spécialité : Intelligence Artificielle

Par :

NGONG KOUT Yuri Roosevelt

Matricule : ***CM-UDS-24SCI1029***

Licence en Informatique Fondamentale

Sous la direction de :

ZEKENG NDADJI Milliam Maxime
Chargé de Cours, Université de Dschang

Année académique 2025/2026

DÉCLARATION D'AUTHENTICITÉ

Je soussigné, NGONG KOUT Yuri Roosevelt, étudiant au Département de Mathématiques et Informatique de la Faculté des Sciences de l'Université de Dschang, certifie que ce mémoire de Master, intitulé « **Ingénierie des applications web pilotées par des modèles BPMN** » et réalisé au sein de l'Unité de Recherche en Informatique Fondamentale, Ingénierie et Applications (URIFIA) sous la direction du Dr ZEKENG NDADJI Milliam Maxime, est un travail original. Il n'a jamais été présenté en vue de l'obtention d'un autre diplôme.

L'étudiant
NGONG KOUT Yuri Roosevelt

Le directeur de mémoire

Le chef de département

DÉDICACE

À mon père **KOUT MBEPA GEORGES** *et à ma mère*
NGUIONG NGONG ALINE PRISCILLE

REMERCIEMENTS

Tout d'abord, je remercie infiniment *Dieu Tout-Puissant* pour ses bénédictions, sa grâce et son infinie bonté. Je tiens ensuite à adresser mes sincères remerciements :

— au *Dr ZEKENG NDADJI Milliam Maxime*, mon encadreur académique, pour la confiance qu'il m'a témoignée en acceptant la direction scientifique de mes travaux. Je lui suis reconnaissant de m'avoir fait bénéficier, tout au long de cette recherche, de sa rigueur intellectuelle, de sa disponibilité, de sa bienveillance, de son inspiration constante et de ses précieux conseils. Son expertise et son soutien indéfectible ont été essentiels à l'élaboration de ce mémoire ;

— à tous les *enseignants du Département de Mathématiques et Informatique de la Faculté des Sciences de l'Université de Dschang*, pour la qualité de leurs enseignements et leurs précieux conseils ;

— aux distingués membres du jury qui ont accepté d'examiner ce travail. Leurs observations contribueront à enrichir ma formation ;

— à mes frères et sœurs, *Fossi Benson, Mbepa Nathanaël, Mollo Josue, Kout Sterve, Goua Dorcas, Mangha Déograce et Kengne Godwin*, pour leur soutien inconditionnel et leur affection ;

— aux familles *MBEPA et NGONG*, pour leur soutien et leurs conseils ;

— à mes précieux amis, *Pecheu Aubin, Chedjou Triomphant, Guembou Marie, Lekane Vergez, Lomofouet Jaures, Fomo Vidal, Magni Dorine, Kalzibe Dominique, Tajeumi Warren, Souleyman Bechir, Tchiguia Francelle et Kenfack Victor* ;

— à mes aînés académiques, *Nguedia Daniela, Zemtso Arléon et Tueam Dimitri*, pour leur soutien inestimable et leurs conseils éclairés, qui ont enrichi mon parcours et nourri ma quête de connaissance ;

— que tous ceux dont les noms ne figurent pas sur cette page ne se sentent pas lésés : leur contribution m'est également précieuse.

TABLE DES MATIÈRES

Dédicace ii
Remerciements iii
Table des matières iv
Liste des symboles viii
Liste des acronymes ix
Liste des tableaux x
Liste des figures xi
Résumé xiii
Abstract xv

Introduction générale **1**
Contexte du travail 1
Problématique et objectifs 2
Organisation du manuscrit 4

Chapitre I — **Généralités sur le Business Process Model and Notation et applications web métier** **5**
I.1 - Introduction 5
I.2 - Processus métier et gestion des processus métier (BPM) 5
I.2.1 - Processus métier 6
I.2.2 - Gestion des processus métier (BPM) 7
I.2.3 - Cycle de vie du BPM 7
I.2.4 - Systèmes de gestion des processus métier 9
I.3 - Modélisation des processus métier 9
I.4 - Ingénierie dirigée par les modèles 14
I.4.1 - Principe général de l'ingénierie dirigée par les modèles 14
I.4.2 - Notions de base en ingénierie dirigée par les modèles 15
I.4.3 - Transformation de modèles 15
I.4.4 - Model-Driven Architecture 16
I.5 - Applications web métier 17
I.5.1 - Généralités sur le développement web 17
I.5.1.1 - Historique du développement web 17
I.5.1.2 - Définition d'application web et concepts du web . . . 18

I.5.1.3 - Frameworks d'application web 19
I.5.1.4 - Sécurité dans les applications web 19
I.5.2 - Applications web métier 20
I.5.2.1 - Définitions et principe 20
I.5.2.2 - Moteur de processus 20
I.6 - Conclusion 21

Chapitre II — **État de l'art sur la génération d'applications web pilotées par des modèles** **22**
II.1 - Introduction 22
II.2 - Démarche d'analyse des travaux existants 22
II.2.1 - Questions de revue 23
II.2.2 - Sources consultées et critères de sélection 23
II.2.3 - Critères d'analyse 24
II.3 - Revue de la littérature 24
II.3.1 - Systèmes d'information orientés processus et plateformes d'exécution 24
II.3.1.1 - Moteurs BPMS et plateformes d'exécution 24
II.3.1.2 - Architecture web pilotée par BPMN 25
II.3.1.3 - WebWorkFlow 26
II.3.1.4 - Plateforme AKIP 27
II.3.2 - Dérivation d'artefacts applicatifs à partir de BPMN 27
II.3.2.1 - Transformation vers des modèles d'interaction 28
II.3.2.2 - Génération de prototypes et d'API 29
II.3.2.3 - Vers une génération intégrée des artefacts applicatifs 29
II.3.3 - Ingénierie dirigée par les modèles et génération de systèmes orientés processus 31
II.3.3.1 - Composition de langages et génération progressive . 31
II.3.3.2 - Spécification grammaticale des workflows web avec LSAWfP4WEB 32
II.3.3.3 - Gabarits et compléments aux modèles BPMN 32
II.3.4 - Maintenabilité du code généré et mise en perspective avec le low-code 33
II.3.4.1 - Gestion conjointe du code généré et du code manuel 34
II.3.4.2 - Mise en perspective avec les approches low-code . . 34
II.4 - Discussion et positionnement de la recherche 34
II.5 - Conclusion 37

Chapitre III — **Approche d'ingénierie proposée pour la génération d'applications web pilotées par BPMN** **38**
III.1 - Introduction . . . 38
III.2 - Vue d'ensemble de l'approche d'ingénierie . . . 39
III.2.1 - Problème traité et principe général . . . 39
III.2.2 - Positionnement dans le cycle MDA . . . 39
III.2.3 - Contribution méthodologique de l'approche . . . 40
III.3 - Architecture globale du framework proposé . . . 40
III.3.1 - Modèles d'entrée de ProOWeb . . . 41
III.3.2 - Noyau de génération du framework . . . 42
III.3.3 - Application web générée et architecture logique du socle applicatif . . . 42
III.3.4 - Exploitation et évolution de l'application . . . 45
III.4 - Modèle d'entrée enrichi : spécification spec-v1 . . . 46
III.4.1 - Grammaire formelle de spec-v1 . . . 46
III.4.2 - Propriétés de la grammaire spec-v1 . . . 52
III.4.3 - Métadonnées des activités . . . 55
III.4.4 - Affectation des tâches et sécurité . . . 55
III.4.5 - Entrées, sorties et données partagées . . . 56
III.4.6 - Règles de visibilité . . . 57
III.5 - Règles de transformation et génération de l'application web . . . 57
III.5.1 - Validation avant génération . . . 58
III.5.2 - Transformation M2M . . . 58
III.5.3 - Transformation M2T . . . 60
III.5.4 - Règles de projection vers les artefacts . . . 62
III.5.5 - Traçabilité de bout en bout . . . 63
III.6 - Préservation du code personnalisé et synchronisation . . . 63
III.6.1 - Problème de la régénération . . . 64
III.6.2 - Manifeste et contrat de propriété . . . 64
III.6.3 - Préservation côté logique applicative . . . 64
III.6.4 - Préservation côté interface utilisateur . . . 65
III.6.5 - Cycle de synchronisation itératif . . . 65
III.7 - Conclusion . . . 66

Chapitre IV — **Mise en œuvre et expérimentation** **67**
IV.1 - Introduction . . . 67
IV.2 - Environnement de mise en œuvre . . . 68

IV.2.1 - Objectif de réalisation 68
IV.2.2 - Choix technologiques 68
IV.2.3 - Chaîne de génération 69
IV.3 - Cas d'étude organisationnel 70
IV.3.1 - Présentation de l'organisation retenue 70
IV.3.2 - Acteurs et responsabilités 70
IV.3.3 - Processus retenus 71
IV.3.4 - Scénario nominal et scénarios alternatifs 72
IV.4 - Déroulement complet du processus de commande client 72
IV.4.1 - Initialisation du projet 73
IV.4.2 - Modélisation du processus 73
IV.4.3 - Enrichissement du BPMN avec *spec-v1* 74
IV.4.4 - Analyse et validation 74
IV.4.5 - Construction du modèle pivot 75
IV.4.6 - Génération des artefacts applicatifs 77
IV.4.7 - Observation de l'application web générée 78
IV.5 - Discussion et positionnement 82
IV.6 - Conclusion 83

Conclusion générale **84**
Bilan du travail 84
Quelques perspectives 85

Bibliographie **87**

LISTE DES SYMBOLES

$G_{\text{spec-v1}}$	Grammaire formelle de la spécification `spec-v1`.
N	Ensemble des non-terminaux de $G_{\text{spec-v1}}$.
T	Ensemble des terminaux de $G_{\text{spec-v1}}$.
P	Ensemble des productions de $G_{\text{spec-v1}}$.
S	Symbole de départ de $G_{\text{spec-v1}}$, avec $S = \langle Specification \rangle$.
B	Modèle BPMN importé.
A_B	Ensemble des activités du modèle BPMN B.
A_s	Ensemble des activités enrichies par la spécification `spec-v1`.
E_s	Ensemble des entités partagées déclarées dans `spec-v1`.
C	Ensemble des contraintes de validité appliquées avant la construction du modèle pivot.
L_{syn}	Langage syntaxique engendré par $G_{\text{spec-v1}}$.
L_{val}	Langage des couples BPMN/`spec-v1` satisfaisant les contraintes d'attributs.
enc	Fonction de représentation finie d'un modèle ou d'une spécification.

LISTE DES ACRONYMES

AFNOR	Association française de normalisation ;
AKIP	AKIP Process Automation Platform ;
API	Application Programming Interface ;
ATL	ATLAS Transformation Language ;
BPM	Business Process Management ;
BPMI	Business Process Management Institute ;
BPMN	Business Process Model and Notation ;
BPMS	Business Process Management System ;
CIM	Computation Independent Model ;
DMN	Decision Model and Notation ;
EBNF	Extended Backus–Naur Form ;
EPC	Event-driven Process Chain ;
HTML	HyperText Markup Language ;
HTTP	HyperText Transfer Protocol ;
IBM	International Business Machines ;
IDM	Ingénierie dirigée par les modèles ;
IFML	Interaction Flow Modeling Language ;
jBPM	Java Business Process Management ;
JSON	JavaScript Object Notation ;
M2M	Model-to-Model ;
M2T	Model-to-Text ;
MDA	Model-Driven Architecture ;
MOF	Meta-Object Facility ;
OMG	Object Management Group ;
PIM	Platform Independent Model ;
PSM	Platform Specific Model ;
QVT	Query/View/Transformation ;
RBAC	Role-Based Access Control ;
REST	Representational State Transfer ;
RFC	Request for Comments ;
SPA	Single Page Application ;
SWEL	Domain-Specific Language for Modeling Data-Intensive Workflows ;
UML	Unified Modeling Language ;
WebDSL	Web Domain-Specific Language ;
XOR	Exclusive OR.

LISTE DES TABLEAUX

I - Composantes essentielles d'un processus métier 6

II - Grille concrète de lecture des travaux étudiés 24
III - Récapitulatif des travaux étudiés 35
IV - Limites transversales des travaux étudiés et implications pour la recherche 36

V - Positionnement de ProOWeb dans le cycle MDA 40
VI - Stratégies d'affectation 56
VII - Entrées, sorties et données partagées d'une activité BPMN enrichie . 57
VIII - Structures principales du modèle pivot ProOWeb 60
IX - Règles de projection vers l'application générée 62
X - Traçabilité d'une activité BPMN vers les artefacts générés 63
XI - Mécanismes de préservation et de synchronisation dans ProOWeb . . . 66

XII - Principaux choix de mise en œuvre de ProOWeb 69
XIII - Acteurs du cas d'étude e-commerce 71
XIV - Processus métier retenus dans le cas d'étude 72

LISTE DES FIGURES

1 - Exemple de processus métier 7
2 - Cycle de vie du BPM, adapté de [12] 8
3 - Historique du BPMN, adapté de [15] 10
4 - Principaux éléments de BPMN 2.0 11
5 - Principaux types de tâches dans BPMN 2.0 11
6 - Événements de début, intermédiaires et de fin dans BPMN 2.0 11
7 - Principales passerelles de BPMN 2.0 12
8 - Flux de séquence dans BPMN 2.0 12
9 - Pool comportant deux lanes dans BPMN 2.0 12
10 - Représentation des données dans BPMN 2.0 13
11 - Groupes et annotations textuelles dans BPMN 2.0 13
12 - Exemple de diagramme de processus BPMN 14
13 - Exemple de diagramme de collaboration BPMN 14
14 - Exemple de diagramme de chorégraphie BPMN 14
15 - Pyramide de modélisation de l'OMG, d'après [5]. 15
16 - Schéma de base d'une transformation de modèle [16]. 16
17 - Illustration des trois niveaux d'abstraction de la MDA [16]. 17
18 - Cœur du modèle RBAC, adapté de [25] 20
19 - Modélisation d'un processus BPMN exécutable dans Camunda Modeler, d'après la documentation Camunda [26]. 25
20 - Fonctionnalités et technologies d'une application web pilotée par processus [33]. 26
21 - Interface générée dans WebWorkFlow pour une tâche de workflow, extraite de [34]. 26
22 - Architecture et principe de génération d'une application dans AKIP, adaptés de [35]. 27
23 - Transformation d'un modèle IFML vers un modèle Symfony, adaptée de [36]. 28
24 - Cycle de prototypage rapide d'applications web proposé par [39]. . . . 30
25 - Extraits de code généré pour le contrôleur et la vue dans l'approche de [39]. 30

26 - Génération en deux étapes d'un système d'information orienté processus, adaptée de [9]. 31
27 - Architecture du système proposé par Zemtsop Ndadji pour la gestion des workflows dans une application web, adaptée de [41]. 32

28 - Architecture générale de ProOWeb 41
29 - Architecture logique du socle applicatif web métier généré 43

30 - Chaîne de génération de ProOWeb 70
31 - Initialisation du projet `E-commerce_Application` 73
32 - Modélisation du processus de commande client dans ProOWeb 74
33 - Spécification ajoutée à l'activité « Passer la commande » 74
34 - Saisie d'une commande par l'utilisateur client 79
35 - Instance de commande démarrée par le client 79
36 - Reprise de l'instance par le service commercial 80
37 - Reprise de l'instance par le gestionnaire de stock 81
38 - Chronologie finale de l'instance de commande client 81

RÉSUMÉ

Ce mémoire s'intéresse à la génération d'applications web métier à partir de modèles de processus. Il part du constat que, dans de nombreuses organisations, les processus décrivent déjà une part essentielle du travail à informatiser : les activités à enchaîner, les acteurs concernés, les informations manipulées et les décisions à prendre. Pourtant, au moment de développer une application web, ces modèles restent souvent cantonnés à un rôle documentaire et leur contenu doit être réinterprété manuellement par les développeurs. Cette séparation fragilise la cohérence entre le besoin métier et le logiciel produit, complique la maintenance et rend l'évolution des applications plus coûteuse lorsque les procédures changent. L'objectif de ce travail est donc de concevoir un framework capable d'exploiter plus directement un modèle BPMN enrichi pour produire un socle applicatif web métier structuré, traçable et adaptable. Pour cela, nous avons conçu ProOWeb, un framework qui associe le modèle BPMN, chargé de représenter le déroulement du processus, à une spécification complémentaire. Cette spécification apporte les informations que le modèle ne porte pas suffisamment, notamment les rôles, les affectations, les données, les entrées, les sorties, les règles de visibilité et les indications nécessaires à la génération des formulaires. Le framework valide ces éléments, les organise dans un modèle pivot, puis les projette vers les artefacts du socle applicatif : vues de traitement, logique applicative, structures de données, règles d'accès, suivi des instances et traces de génération. Le travail met également l'accent sur la régénération contrôlée, afin que le projet cible puisse évoluer sans rompre le lien entre le processus, la spécification et le code. Ainsi, ProOWeb ne vise pas seulement à automatiser l'écriture de fichiers, mais à rapprocher durablement la modélisation métier et la réalisation logicielle.

Mots clés : Processus métier, application web métier, BPMN, ProOWeb, génération d'application, ingénierie dirigée par les modèles, spécification enrichie, régénération contrôlée.

BPMN MODEL-DRIVEN WEB APPLICATION ENGINEERING

ABSTRACT

This thesis focuses on generating business web applications from process models. It begins with the observation that, in many organizations, processes already describe a significant portion of the work to be computerized : the activities to be sequenced, the stakeholders involved, the information handled, and the decisions to be made. However, when developing a web application, these models often remain confined to a purely documentary role, and their content must be manually reinterpreted by developers. This separation weakens the coherence between business needs and the software produced, complicates maintenance, and makes application evolution more expensive when procedures change. The objective of this work is therefore to design a framework capable of making more direct use of an enriched BPMN model to produce a structured, traceable, and adaptable business web application foundation. To this end, we have designed ProOWeb, a framework that combines the BPMN model, responsible for representing the process flow, with a complementary specification. This specification provides the information that the model does not adequately contain, including roles, assignments, data, inputs, outputs, visibility rules, and the necessary information for form generation. The framework validates these elements, organizes them into a pivot model, and then projects them onto the application foundation : processing views, application logic, data structures, access rules, instance tracking, and generation traces. The work also emphasizes controlled regeneration, ensuring that the target project can evolve without breaking the link between the process, the specification, and the code. Thus, ProOWeb aims not only to automate file writing but also to foster a lasting connection between business modeling and software development.

Keywords : Business process, business web application, BPMN, ProOWeb, application generation, model-driven engineering, rich specification, controlled regeneration.

INTRODUCTION GÉNÉRALE

SOMMAIRE

Contexte du travail 1
Problématique et objectifs 2
Organisation du manuscrit 4

Contexte du travail

Une part importante des services proposés aujourd'hui sur le web repose sur des activités organisées sous forme de processus métier. Lorsqu'un usager effectue une inscription en ligne, dépose une demande administrative, suit une réclamation, réalise une commande ou valide une opération interne dans une organisation, il participe à une suite d'actions coordonnées, portées à la fois par des humains, des règles métier et des systèmes logiciels. Dans ce contexte, l'application web ne se limite plus à présenter des pages ou à stocker des informations : elle devient le support d'exécution d'un travail métier, avec des rôles, des formulaires, des données, des validations et des enchaînements d'activités.

Le Business Process Management (BPM) fournit un cadre pour identifier, modéliser, analyser et améliorer ces processus. Les travaux de référence dans ce domaine montrent que la maîtrise d'un processus passe autant par sa représentation explicite que par sa capacité à être exécuté et adapté dans le système d'information [1, 2]. Parmi les notations utilisées, BPMN occupe une place importante, car elle permet de décrire les acteurs, les événements, les tâches et les flux de contrôle d'un processus selon une notation largement reconnue [3]. Cependant, un modèle BPMN, même correct du point de vue du processus, ne suffit généralement pas à spécifier directement un socle applicatif web métier. Il indique ce qui doit s'enchaîner, mais ne décrit pas toujours avec précision les écrans attendus, les données ma-

nipulées, les champs visibles ou obligatoires, les règles de création des formulaires, ni la manière dont les utilisateurs interagissent concrètement avec le processus.

Parallèlement, l'ingénierie dirigée par les modèles propose de placer les modèles au centre du développement logiciel, afin de transformer des descriptions abstraites en artefacts plus proches de l'exécution [4, 5]. Cette orientation est particulièrement intéressante pour les applications web orientées processus, car elle permet d'envisager une continuité entre la compréhension métier, la spécification du processus et la production de l'application. Toutefois, les approches existantes restent souvent limitées à la génération de fragments d'interface, de prototypes ou de structures partielles. Plusieurs travaux ont exploré le passage de BPMN vers des modèles d'interaction ou vers des applications guidées par les processus [6–9], mais le problème demeure lorsqu'il s'agit de produire un socle applicatif web métier structuré et cohérent avec les exigences exprimées dans le modèle.

C'est dans cette perspective que s'inscrit ce mémoire, intitulé « Ingénierie des applications web pilotées par des modèles BPMN ». Le travail présenté porte sur le framework ProOWeb, qui vise à générer un socle applicatif web métier à partir de modèles BPMN enrichis. L'idée centrale n'est pas seulement de représenter un processus ni de produire une maquette, mais de faire du modèle enrichi une source exploitable pour construire un socle capable de prendre en charge les acteurs, les tâches, les données, les formulaires, les transitions et le suivi des instances. Le framework ProOWeb cherche ainsi à rapprocher la modélisation métier et la réalisation logicielle, tout en conservant une organisation suffisamment claire pour permettre l'évolution du socle généré.

Problématique et objectifs

La difficulté principale vient du décalage entre la richesse attendue d'une application web métier et les informations habituellement contenues dans un modèle BPMN standard. Un processus peut préciser qu'une tâche doit être exécutée par un acteur donné, mais l'application correspondante doit aussi savoir quelles données afficher, quelles informations demander, quelles validations appliquer, quelles pages produire, comment relier les actions de l'utilisateur à l'avancement du processus et comment préserver les parties personnalisées lorsque le modèle évolue. Sans mécanisme structuré, le passage du modèle au code devient manuel, répétitif et fragile ; il expose le projet à des incohérences entre le processus conçu et l'application réellement livrée.

La problématique de ce travail peut donc être formulée de la manière suivante : comment concevoir un framework permettant, à partir de modèles BPMN enrichis, de générer un socle applicatif web métier fonctionnel, traçable et extensible, tout en conservant la cohérence entre les processus, les données, les rôles, les interfaces et l'évolution future de l'application ?

L'objectif principal de ce mémoire est de concevoir et de mettre à l'épreuve ProOWeb, un framework de génération d'applications web métier pilotées par des modèles BPMN enrichis. La réponse proposée à la problématique repose sur une chaîne explicite : le BPMN décrit le déroulement du travail, *spec-v1* apporte les informations applicatives manquantes, une validation croisée contrôle leur cohérence, puis des contrats intermédiaires stabilisent les transformations modèle-à-modèle et modèle-à-texte. Ces transformations produisent un socle comprenant le backend, le frontend, les structures de données, les règles de sécurité et les ressources du runtime. Un manifeste relie enfin les sources aux fichiers générés et prépare leur régénération contrôlée.

De manière spécifique, il s'agit de :

— étudier les notions de processus métier, de BPMN, d'applications web orientées processus et d'ingénierie dirigée par les modèles, afin de poser les bases conceptuelles du travail ;

— analyser les travaux existants sur la génération d'applications web à partir de modèles de processus, en mettant en évidence leurs apports et leurs limites ;

— définir et formaliser un modèle d'entrée enrichi associant BPMN et *spec-v1*, ainsi que les contrôles nécessaires à leur mise en cohérence ;

— proposer l'architecture de ProOWeb, ses contrats intermédiaires, les algorithmes de transformation et ses mécanismes de traçabilité et de régénération ;

— mettre en œuvre cette chaîne et l'évaluer, dans les limites d'un cas d'étude, sur trois processus d'une même organisation.

La contribution ne se réduit donc pas à un outil de production de fichiers. Elle porte sur une démarche d'ingénierie qui précise les informations attendues, contrôle les modèles d'entrée, organise leur transformation et conserve la trace des décisions projetées dans le code. ProOWeb vise ainsi un socle applicatif fonctionnel, traçable et extensible, que le développeur doit encore vérifier et adapter aux contraintes propres à son organisation.

Organisation du manuscrit

La suite de ce mémoire est organisée en quatre chapitres, suivis d'une conclusion générale.

Chapitre I — Généralités sur le Business Process Model and Notation et les applications web métier. Ce chapitre présente les concepts de base nécessaires à la compréhension du sujet. Il revient sur les processus métier, le BPM, la notation BPMN, l'ingénierie dirigée par les modèles et les applications web métier. Il permet ainsi de fixer le vocabulaire et le cadre conceptuel utilisés dans la suite du mémoire.

Chapitre II — Revue de la littérature sur la génération d'applications web pilotées par des modèles. Ce chapitre étudie les travaux existants liés à l'automatisation des processus, aux systèmes BPM, à la génération de code, aux approches low-code/no-code et aux transformations de modèles. Il met en évidence les limites rencontrées lors du passage d'un modèle de processus à un socle applicatif web métier fonctionnel et évolutif.

Chapitre III — Framework d'ingénierie proposé pour la génération d'applications web pilotées par BPMN. Ce chapitre constitue le cœur de la contribution. Il présente ProOWeb, son principe général, son architecture, son modèle d'entrée enrichi, ses règles de transformation et les mécanismes prévus pour produire un socle applicatif web métier, que le développeur peut ensuite vérifier, adapter et compléter.

Chapitre IV — Mise en œuvre et expérimentation. Ce chapitre est consacré à la réalisation concrète du framework et à son évaluation. Il présente le socle généré pour une organisation de commerce électronique, les choix de mise en œuvre, les trois processus retenus, les interfaces obtenues, les résultats observés ainsi que les limites du travail réalisé.

Conclusion générale — elle rappelle la problématique traitée, synthétise les contributions du mémoire, discute les résultats obtenus et ouvre quelques perspectives pour l'amélioration de ProOWeb.

I

CHAPITRE

GÉNÉRALITÉS SUR LE BUSINESS PROCESS MODEL AND NOTATION ET APPLICATIONS WEB MÉTIER

SOMMAIRE

I.1 - Introduction 5
I.2 - Processus métier et gestion des processus métier (BPM) 5
I.3 - Modélisation des processus métier 9
I.4 - Ingénierie dirigée par les modèles 14
I.5 - Applications web métier 17
I.6 - Conclusion 21

I.1. Introduction

Dans ce chapitre, nous présentons les notions de base nécessaires à la compréhension de notre travail. Nous abordons dans un premier temps les processus métier, le BPM et BPMN. Dans un second temps, nous rappelons les concepts du développement web utiles à l'étude des applications métier. Cette présentation permet de préciser le lien entre modèle métier, moteur de processus et application web.

I.2. Processus métier et gestion des processus métier (BPM)

I.2.1. Processus métier

Un processus peut être compris comme un ensemble d'activités corrélées qui transforment des éléments d'entrée en éléments de sortie. Cette définition, proche de celle de l'ISO 9000, met l'accent sur l'enchaînement, la transformation et le résultat attendu. Hammer et Champy définissent également le processus comme un ensemble finalisé d'activités produisant une valeur pour un client [10].

Un processus métier est un processus lié à un objectif concret de l'organisation. Il décrit comment une structure produit un service, traite une demande, prend une décision ou coordonne plusieurs intervenants.

Le caractère métier du processus est important. Le processus exprime d'abord une réalité organisationnelle avant de devenir une réalité informatique. Il précise ce qui doit être fait, par qui, avec quelles informations, selon quelles règles et avec quel résultat.

Un processus métier comporte généralement trois dimensions. La dimension fonctionnelle répond à la question : que faut-il faire ? La dimension organisationnelle répond à la question : qui le fait ? La dimension informationnelle répond à la question : quelles données sont nécessaires ou produites ? Ces trois dimensions forment une base utile pour passer ensuite vers la conception d'un système d'information.

TABLE I – *Composantes essentielles d'un processus métier*

Composante	**Rôle dans le processus**
Entrées	Éléments nécessaires au démarrage ou à l'exécution : demande, formulaire, document, événement ou information reçue.
Activités	Opérations réalisées par des acteurs humains ou des systèmes.
Acteurs	Personnes, services, rôles ou applications responsables des activités.
Règles	Conditions qui orientent l'exécution, les décisions, les validations et les autorisations.
Sorties	Résultats produits : dossier validé, demande rejetée, document généré, notification envoyée.

La figure 1 illustre l'enchaînement de ces composantes dans un processus simple.

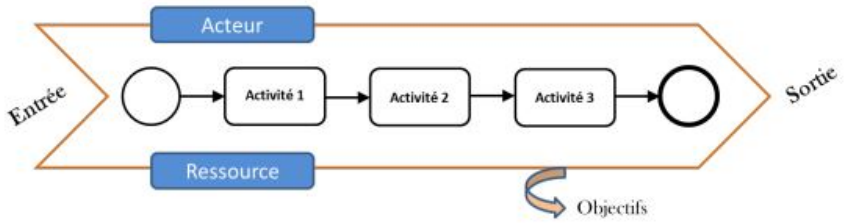


FIGURE 1 – *Exemple de processus métier*

I.2.2. Gestion des processus métier (BPM)

Définitions de la gestion des processus métier

Plusieurs définitions ont été données pour le terme de gestion des processus métier.

D'après van der Aalst et al., le *Business Process Management* (BPM) comprend « les méthodes, les techniques et les outils permettant de concevoir, d'exécuter, de gérer et d'analyser les processus opérationnels, en s'appuyant sur des acteurs pouvant être des êtres humains, des organisations, des applications et d'autres sources d'information » [11].

Le *Business Process Management* (BPM) peut également être défini comme l'art de modéliser, de gérer et d'optimiser les processus métier afin d'améliorer la performance des entreprises. Il nécessite l'utilisation d'outils permettant de modéliser, d'intégrer, de superviser et d'analyser le fonctionnement des processus tout au long de leur cycle de vie, dans une logique d'amélioration continue [1, 2].

I.2.3. Cycle de vie du BPM

Le BPM s'inscrit dans un cycle d'amélioration continue, illustré par la figure 2. Selon Dumas [12], ce cycle relie l'étude du fonctionnement existant, la conception d'un processus cible, sa mise en œuvre et l'observation de ses résultats.

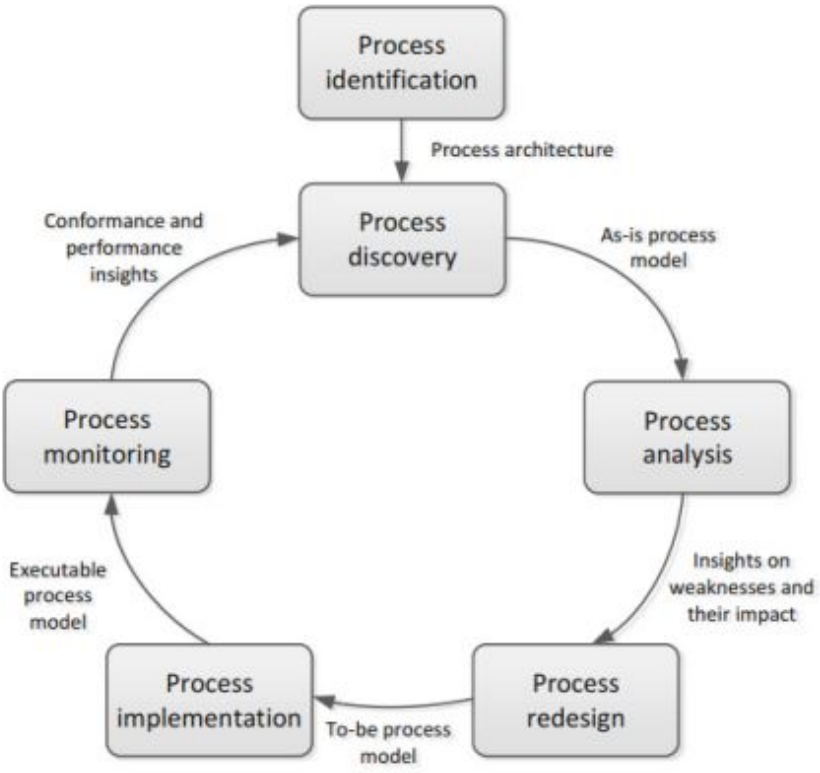


FIGURE 2 – *Cycle de vie du BPM, adapté de [12]*

Les six phases peuvent être résumées ainsi :

1. l'**identification** délimite les processus à étudier et précise les objectifs poursuivis ;
2. la **découverte** représente le fonctionnement réel du processus et les responsabilités des acteurs ;
3. l'**analyse** met en évidence les difficultés, les écarts et les résultats mesurables ;
4. la **refonte** élabore un processus cible répondant aux problèmes constatés ;
5. la **mise en œuvre** applique les changements organisationnels et introduit le soutien nécessaire des systèmes d'information ;
6. le **suivi** confronte l'exécution aux objectifs et alimente un nouveau cycle d'amélioration.

Pour ce mémoire, la mise en œuvre est particulièrement importante : elle relie le modèle de processus à l'automatisation et aux applications chargées de prendre en charge les tâches, les données et les interactions avec les acteurs.

I.2.4. Systèmes de gestion des processus métier

Les systèmes de gestion des processus métier, ou BPMS *(Business Process Management Systems)*, soutiennent la démarche BPM par des fonctions de modélisation, d'exécution, de gestion des tâches, de supervision et d'intégration avec le système d'information [13]. Camunda, Bonita, IBM Business Automation Workflow et Flowable illustrent cette catégorie. Un BPMS orchestre et suit les processus ; il ne se confond pas avec l'ensemble de l'application web métier, qui doit également prendre en charge les interfaces, les données et les règles propres à l'organisation.

I.3. Modélisation des processus métier

La modélisation des processus métier consiste à représenter le fonctionnement d'un processus afin de préciser les activités qui le composent ainsi que leur ordre d'exécution. Dans la littérature, deux grandes approches sont généralement distinguées pour décrire le comportement d'un processus [14] :

— **L'approche impérative** : elle décrit de manière explicite la façon dont le processus doit être exécuté. Elle précise donc l'enchaînement des activités, les conditions de passage d'une étape à une autre et les différents chemins possibles. Les langages tels que BPMN, les diagrammes d'activités UML ou encore les EPC s'inscrivent dans cette logique.

— **L'approche déclarative** : elle s'intéresse davantage aux contraintes que le processus doit respecter qu'à la description détaillée de toutes les séquences d'exécution. Autrement dit, elle indique ce qui est autorisé, obligatoire ou interdit, sans imposer un chemin unique d'exécution. Parmi les langages associés à cette approche, on peut citer Declare, ConDec, PLM-flow et PENELOPE.

Business Process Model and Notation (BPMN)

Définition et objectif

BPMN, pour *Business Process Model and Notation*, est un standard de modélisation des processus métier maintenu par l'Object Management Group. Sa version 2.0.2 définit à la fois une notation graphique et des éléments de méta-modèle permettant de représenter des processus, des événements, des activités, des passerelles, des participants et des flux [3]. Son intérêt principal est de proposer un langage visuel compréhensible par les acteurs métier, tout en restant exploitable par des outils informatiques.

BPMN répond à un besoin fréquent dans les organisations : disposer d'une notation commune pour décrire les processus. Avant l'existence d'une notation partagée, chaque équipe pouvait représenter les processus à sa manière, avec des symboles locaux et des règles implicites. Cette diversité rendait les échanges difficiles. BPMN cherche à réduire cette ambiguïté en fournissant une grammaire visuelle plus stable.

Historique du BPMN

BPMN est né du besoin de disposer d'une notation commune pour représenter les processus métier. Les premiers travaux ont été conduits par la Business Process Management Initiative, avant que la norme soit reprise et maintenue par l'OMG. Son évolution a progressivement rapproché la notation graphique d'un méta-modèle exploitable par les outils. La version 2.0.2 constitue la référence utilisée dans ce mémoire [3, 15]. La figure 3 résume les principales étapes de cette évolution.



FIGURE 3 – *Historique du BPMN, adapté de [15]*

Éléments de base de BPMN

BPMN regroupe ses éléments graphiques en cinq familles : les objets de flux, les objets de connexion, les partitions, les données et les artefacts. La figure 4 en donne une vue d'ensemble. Ces familles suffisent ici à introduire les concepts mobilisés dans les modèles du mémoire, sans détailler l'ensemble des symboles définis par la norme.

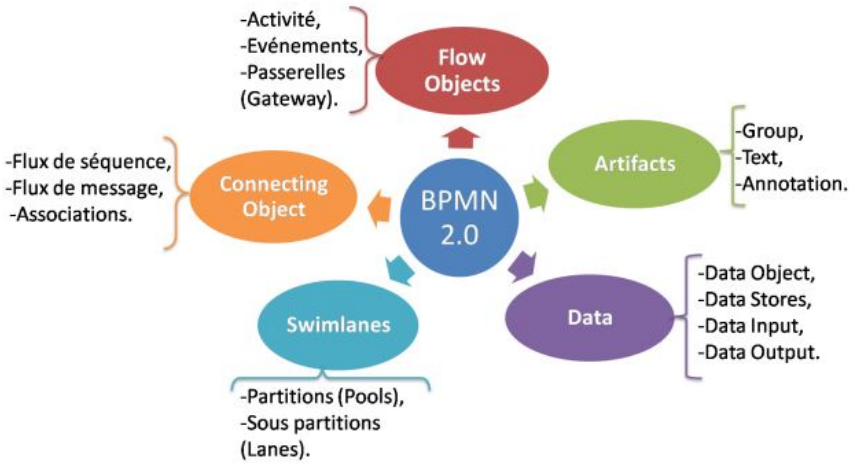


FIGURE 4 – *Principaux éléments de BPMN 2.0*

A. Objets de flux

Les objets de flux décrivent le comportement du processus. Une activité représente un travail à accomplir ; elle peut correspondre à une tâche atomique ou à un sous-processus. Parmi les tâches utiles pour la suite, la tâche utilisateur associe une action humaine à une application, la tâche manuelle est réalisée sans assistance du moteur de processus, tandis que la tâche de service déclenche un traitement automatisé. La figure 5 rappelle les principaux types proposés par BPMN.

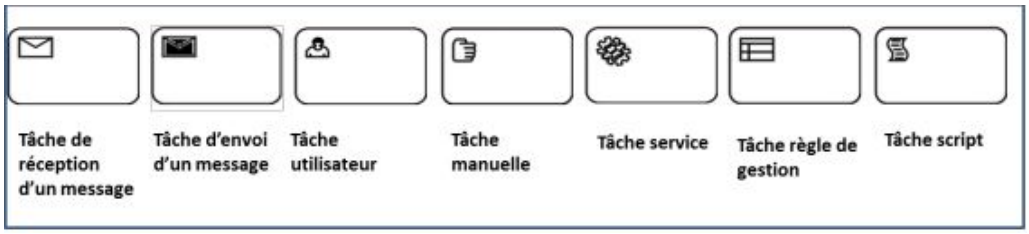


FIGURE 5 – *Principaux types de tâches dans BPMN 2.0*

Les événements marquent ce qui déclenche, interrompt ou termine une partie du processus. Un événement de début crée une instance, un événement intermédiaire intervient pendant son exécution et un événement de fin clôt le chemin concerné. Cette distinction est illustrée par la figure 6.



FIGURE 6 – *Événements de début, intermédiaires et de fin dans BPMN 2.0*

Les passerelles contrôlent la divergence et la convergence des chemins. Une passerelle exclusive sélectionne une seule branche, une passerelle inclusive peut en activer plusieurs selon les conditions, et une passerelle parallèle déclenche ou synchronise plusieurs branches. La figure 7 présente leurs symboles usuels.



FIGURE 7 – *Principales passerelles de BPMN 2.0*

B. Objets de connexion

Les objets de connexion relient les éléments du diagramme. Le flux de séquence indique l'ordre d'exécution au sein d'un processus ; le flux de message représente un échange entre participants distincts ; l'association rattache une donnée ou une annotation à un élément. Le flux de séquence est central dans ce mémoire, car il porte le déroulement que ProOWeb conserve lors du traitement du modèle BPMN. La figure 8 en rappelle la notation.



FIGURE 8 – *Flux de séquence dans BPMN 2.0*

C. Partitions

Les partitions rendent visibles les responsabilités organisationnelles. Un *pool* représente un participant, alors qu'une *lane* organise les activités de ce participant selon un rôle ou une fonction. Les flux de séquence relient les activités d'un même pool, tandis que les messages assurent les échanges entre pools distincts [3]. Cette distinction prépare la prise en compte des rôles et des interactions dans l'application web métier. La figure 9 montre un pool organisé en deux lanes.



FIGURE 9 – *Pool comportant deux lanes dans BPMN 2.0*

D. Données

Les objets de données indiquent les informations consommées ou produites par une activité. Les entrées alimentent le processus ou une activité, tandis que les sorties en représentent les résultats. BPMN permet de montrer leur association au flux, mais ne suffit pas toujours à exprimer les structures, les mappings, le stockage et les règles de visibilité attendus par une application web métier. La figure 10 présente les symboles correspondants.



FIGURE 10 – *Représentation des données dans BPMN 2.0*

E. Artefacts

Les artefacts complètent la lecture sans modifier directement le déroulement du processus. Les groupes rapprochent visuellement des éléments liés et les annotations textuelles apportent une précision destinée au lecteur. Ils restent utiles pour documenter un diagramme, mais ne portent pas à eux seuls les informations applicatives nécessaires à la génération. La figure 11 illustre ces deux formes.



FIGURE 11 – *Groupes et annotations textuelles dans BPMN 2.0*

Types de diagrammes BPMN

BPMN distingue les diagrammes de processus, de collaboration et de chorégraphie [3]. Le diagramme de processus décrit l'enchaînement interne des activités d'un participant ; il est privilégié dans ce mémoire, car il porte le déroulement utilisé comme entrée de la génération. Le diagramme de collaboration ajoute les échanges de messages entre plusieurs participants, tandis que la chorégraphie se concentre sur leurs interactions sans détailler leur orchestration interne. La figure 12 présente la vue privilégiée dans ce travail. La figure 13 illustre la collaboration, tandis que la figure 14 donne un exemple de la seconde.



FIGURE 12 – *Exemple de diagramme de processus BPMN*

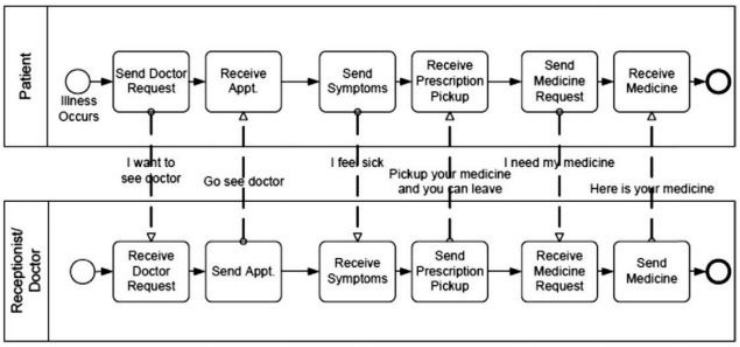


FIGURE 13 – *Exemple de diagramme de collaboration BPMN*

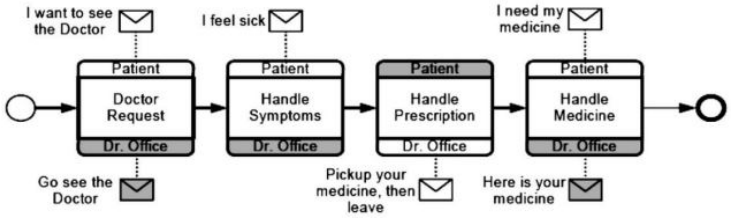


FIGURE 14 – *Exemple de diagramme de chorégraphie BPMN*

I.4. Ingénierie dirigée par les modèles

I.4.1. Principe général de l'ingénierie dirigée par les modèles

L'ingénierie dirigée par les modèles (IDM) désigne une démarche de développement dans laquelle le modèle devient un artefact de travail central. Le modèle n'est donc pas seulement un dessin destiné à expliquer une idée : il sert à décrire un système à un niveau d'abstraction choisi, à vérifier certains choix, à organiser les connaissances du domaine et, lorsque l'outillage le permet, à produire d'autres modèles ou des artefacts techniques. Cette approche répond principalement à deux difficultés du génie logiciel : la croissance de la complexité des systèmes et l'évolution rapide des technologies. En plaçant les modèles au centre du développement, l'IDM cherche à mieux séparer les préoccupations, à réduire la dépendance immédiate aux détails techniques et à rendre certaines tâches répétitives plus maîtrisables [5, 16]. L'objectif n'est pas de supprimer l'intervention humaine, mais de déplacer une partie de l'effort vers des modèles explicites, contrôlables et transformables.

I.4.2. Notions de base en ingénierie dirigée par les modèles

Un système est l'entité étudiée. Il peut être composé d'éléments reliés entre eux, posséder une organisation interne et interagir avec son environnement. Dans l'IDM, le système n'est généralement pas manipulé directement dans toute sa complexité ; il est représenté par un ou plusieurs modèles.

Un modèle est une abstraction d'un système, construite dans un objectif donné. Il retient les informations utiles pour répondre à certaines questions, tout en laissant de côté les détails qui ne sont pas nécessaires au point de vue choisi. Cette idée est importante : un modèle n'a pas vocation à tout contenir. Sa valeur dépend de sa pertinence par rapport au besoin d'analyse, de conception ou de vérification. Jézéquel, Combemale et Vojtisek définissent ainsi le modèle comme un ensemble de faits caractérisant un aspect d'un système selon un objectif précis [5].

Un méta-modèle définit le langage dans lequel les modèles sont exprimés. Il précise les concepts autorisés, leurs relations et les contraintes qui structurent les modèles valides. Le méta-modèle joue donc un rôle de cadre : il rend les modèles manipulables par des outils, car il décrit leur structure attendue. Au-dessus, le méta-méta-modèle fournit le langage permettant de définir les méta-modèles. L'Object Management Group (OMG) organise cette logique selon quatre niveaux : M0 pour le système réel, M1 pour les modèles, M2 pour les méta-modèles et M3 pour le méta-méta-modèle, généralement associé à MOF, Meta-Object Facility [5, 17].

La figure 15 synthétise cette organisation en quatre niveaux.

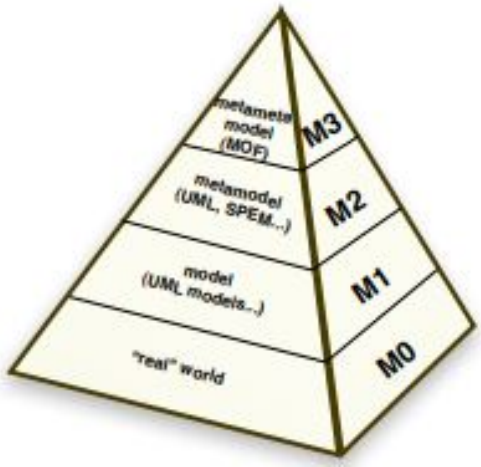


FIGURE 15 – *Pyramide de modélisation de l'OMG, d'après [5].*

I.4.3. Transformation de modèles

La transformation de modèle consiste à produire un modèle cible à partir d'un modèle source, selon des règles définies. Ces règles décrivent les correspondances

entre les éléments du modèle d'origine et ceux du modèle obtenu. Une transformation peut être endogène lorsque les modèles source et cible sont conformes au même méta-modèle ; elle est exogène lorsqu'ils relèvent de méta-modèles différents. Elle peut aussi être horizontale lorsqu'elle reste au même niveau d'abstraction, ou verticale lorsqu'elle fait évoluer le niveau d'abstraction, par raffinement ou par abstraction [5].

Les transformations occupent une place centrale dans l'IDM parce qu'elles donnent un caractère productif aux modèles. Elles peuvent servir à raffiner une spécification, restructurer une représentation, migrer d'une technologie à une autre, générer du texte ou produire du code. Les deux grandes familles généralement retenues sont les transformations modèle-vers-modèle (M2M) et modèle-vers-texte (M2T). Les standards et langages de transformation, comme QVT, ATL ou les approches par templates, cherchent à formaliser ces passages afin qu'ils soient reproductibles et moins dépendants d'une manipulation manuelle [5, 18].

La figure 16 représente le passage d'un modèle source à un modèle cible au moyen de règles de transformation.

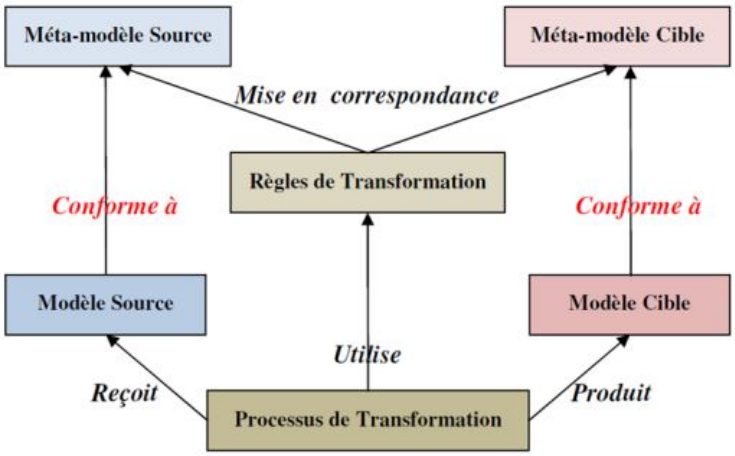


FIGURE 16 – *Schéma de base d'une transformation de modèle [16].*

I.4.4. Model-Driven Architecture

La Model-Driven Architecture (MDA) est une initiative de l'OMG qui applique les principes de l'IDM à l'architecture logicielle. Elle vise à décrire une application en distinguant les modèles indépendants de la plateforme et les modèles liés à une plateforme particulière. La figure 17 présente les trois niveaux d'abstraction correspondants.

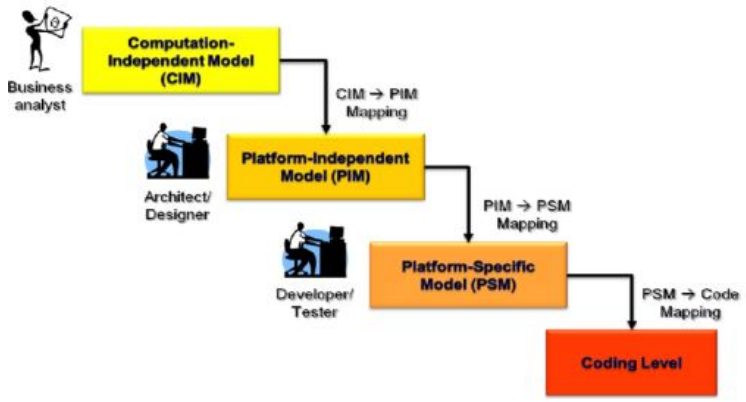


FIGURE 17 – *Illustration des trois niveaux d'abstraction de la MDA [16].*

— **CIM** ***(Computation Independent Model)*** : il décrit les besoins métier et les objectifs sans se préoccuper des aspects techniques. C'est le niveau où l'on formalise ce que le système doit faire, indépendamment de la façon dont il sera implémenté.
— **PIM** ***(Platform Independent Model)*** : ce modèle représente l'architecture et les fonctionnalités du système sans faire référence à une technologie ou une plateforme donnée. Il constitue un pont entre les exigences métier et la solution technique.
— **PSM** ***(Platform Specific Model)*** : il traduit le PIM en une implémentation concrète, en intégrant les contraintes et les choix technologiques spécifiques comme le framework, le langage ou l'environnement d'exécution.

L'intérêt de la MDA est de préserver les décisions de conception à un niveau moins fragile que le code. Lorsque les technologies changent, les modèles indépendants peuvent être conservés plus longtemps, tandis que les transformations et les modèles spécifiques sont adaptés aux nouvelles plateformes. La MDA ne se limite donc pas à une génération automatique de code ; elle propose une organisation du développement où la séparation entre abstraction, plateforme et implémentation devient explicite.

I.5. Applications web métier

I.5.1. Généralités sur le développement web

I.5.1.1. Historique du développement web

Initialement conçu pour publier et relier des documents, le Web est devenu une plateforme d'exécution d'applications accessibles à distance [19]. Aux pages de

consultation se sont ajoutés des formulaires, des espaces authentifiés, des tableaux de bord et des services interactifs. Le navigateur constitue désormais l'interface d'un système qui traite des données et applique des règles métier. Les échanges restent fondés sur le modèle requête-réponse du Web [20], mais leur organisation doit désormais répondre à des exigences de sécurité, de maintenabilité et d'intégration propres aux applications métier.

I.5.1.2. Définition d'application web et concepts du web

Définition

Une application web peut être définie comme un logiciel accessible à partir d'un navigateur ou d'un client reposant sur les technologies du Web. Contrairement à un site essentiellement informatif, elle prend en charge des interactions avec l'utilisateur, traite des données, applique des règles métier et produit des résultats exploitables. Ainsi, une plateforme de demande d'attestation ne se limite pas à afficher des pages : elle permet de soumettre une demande, de suivre son état, de contrôler les droits d'accès et de générer un document. Elle relève donc pleinement d'une application web métier.

Concepts du web

La conception d'une application web repose sur des choix architecturaux qui influencent directement sa maintenabilité, son évolutivité et la séparation des responsabilités. Le modèle client-serveur constitue la base de ce fonctionnement : le client, généralement un navigateur, transmet les actions de l'utilisateur, tandis que le serveur traite les requêtes, accède aux données et retourne les réponses appropriées [21]. Cette séparation est particulièrement importante dans les applications métier, où plusieurs utilisateurs interagissent avec les mêmes ressources selon des droits et des règles différents.

Pour améliorer l'organisation interne du système, les applications web modernes s'appuient souvent sur une architecture en couches. La couche de présentation gère les interfaces et les formulaires, la couche métier applique les règles du domaine, et la couche d'accès aux données assure la communication avec la base. Cette structuration limite le couplage entre les composants et facilite les évolutions du système. Le patron Modèle-Vue-Contrôleur s'inscrit dans cette logique en distinguant les données et règles du domaine, leur présentation à l'utilisateur et le traitement des actions déclenchées par celui-ci [22].

Dans les architectures récentes, les applications monopages *(Single Page Applications)* renforcent la séparation entre le client et le serveur en confiant une partie

importante de l'interface au navigateur. Elles offrent une expérience plus fluide, mais exigent une maîtrise accrue de la complexité du frontend, notamment lorsque l'application doit représenter des tâches, des états de dossier ou des actions liées à un processus métier [23]. Les API web jouent alors un rôle central en exposant les fonctionnalités du système sous forme de ressources accessibles, comme `/demandes`, `/documents` ou `/taches`. Le style REST, formalisé par Fielding, a fortement contribué à cette organisation des échanges autour des ressources, des représentations et d'interactions sans état [24].

Ainsi, le développement d'une application web ne consiste pas seulement à produire des interfaces. Il implique de définir une architecture capable d'articuler présentation, logique métier, persistance des données et échanges entre composants. Cette base conceptuelle est essentielle pour comprendre les applications web orientées processus, dont le comportement dépend étroitement de l'évolution du workflow.

I.5.1.3. Frameworks d'application web

Un framework d'application web fournit des conventions et des composants réutilisables pour organiser le développement. Côté backend, il structure les points d'entrée, les services, l'accès aux données, la validation et la sécurité. Côté frontend, il aide à construire les vues, les formulaires et les interactions présentées à l'utilisateur. Dans une application web métier orientée processus, ces deux parties coopèrent : le backend prend en charge les opérations et les règles applicatives, tandis que le frontend rend visibles les tâches, les états et les actions autorisées. Le framework facilite cette organisation, sans définir à lui seul le processus métier.

I.5.1.4. Sécurité dans les applications web

Les politiques de contrôle d'accès permettent de définir les règles qui encadrent l'accès aux informations au sein d'une entreprise. Dans les applications web, elles contribuent à sécuriser la manipulation des données sensibles, notamment les données des utilisateurs. Parmi ces politiques, le modèle RBAC, ou contrôle d'accès basé sur les rôles, est largement utilisé, car il permet d'organiser les droits d'accès en fonction des responsabilités attribuées aux utilisateurs.

Le principe du RBAC consiste à associer des permissions à des rôles, puis à affecter les utilisateurs aux rôles correspondant à leurs besoins. Cette approche simplifie la gestion de la sécurité, car les administrateurs peuvent créer des rôles, leur attribuer des autorisations et contrôler ainsi l'accès aux ressources. Elle favorise également des principes essentiels comme le moindre privilège, la séparation des tâches et la protection des informations sensibles.

La figure 18 résume les relations entre utilisateurs, rôles et permissions dans le modèle RBAC.

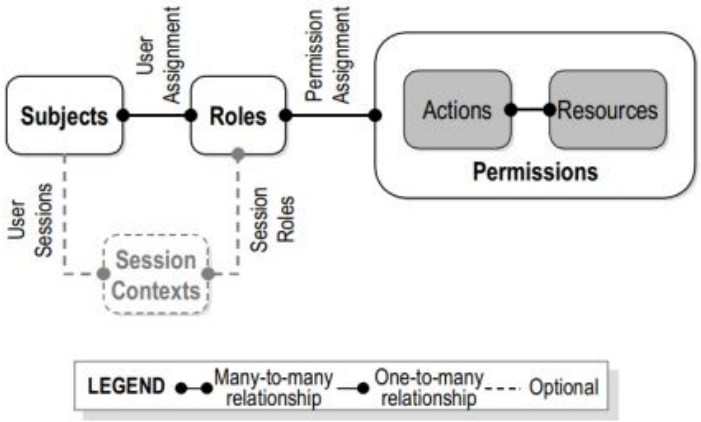


FIGURE 18 – *Cœur du modèle RBAC, adapté de [25]*

I.5.2. Applications web métier

I.5.2.1. Définitions et principe

Une application web orientée processus, ou *process-aware web application*, est une application dont le comportement est organisé autour d'un processus métier explicite. Les écrans, les actions disponibles, les rôles des utilisateurs et les changements d'état suivent l'avancement d'un workflow. Cette idée rejoint la définition des systèmes d'information orientés processus, dans lesquels les activités, les acteurs et les règles d'enchaînement occupent une place centrale [1, 2].

Dans une application classique, le développeur peut coder directement les conditions de passage d'un état à un autre. Dans une application orientée processus, ces passages sont plutôt décrits par un modèle, souvent en BPMN. Le modèle précise les tâches à réaliser, les décisions possibles, les événements attendus et les responsabilités des acteurs. L'application web fournit ensuite les interfaces, les services et les données nécessaires à l'exécution concrète de ce processus.

I.5.2.2. Moteur de processus

Un moteur de processus est un composant logiciel chargé d'exécuter un modèle de processus. Lorsqu'un processus BPMN exécutable est déployé, il peut créer des instances, enregistrer leurs variables, distribuer des tâches humaines, déclencher des services automatiques et suivre l'historique d'exécution [26]. Il joue donc un rôle d'orchestration : il indique ce qui doit être fait, par qui, à quel moment et selon quelles conditions.

Dans une application web orientée processus, le moteur ne remplace pas le frontend, le backend ni la base de données. Il coordonne le déroulement du travail. Le frontend permet aux utilisateurs d'interagir avec les tâches, le backend applique les règles propres à l'application et dialogue avec le moteur, tandis que la base de données conserve les informations métier. Cette répartition évite de confondre l'exécution du workflow avec la construction complète de l'application.

I.6. Conclusion

Dans ce chapitre, nous avons présenté les concepts nécessaires à la suite du mémoire : processus métier, BPM, BPMN, développement web et applications web métier. Nous avons montré qu'un modèle BPMN peut guider l'organisation du travail, mais qu'une application web doit aussi gérer l'interface, les données, la sécurité et l'exécution technique. Le chapitre suivant s'appuie sur ces bases pour analyser les travaux existants.

II

CHAPITRE

ÉTAT DE L'ART SUR LA GÉNÉRATION D'APPLICATIONS WEB PILOTÉES PAR DES MODÈLES

SOMMAIRE

II.1 - Introduction 22
II.2 - Démarche d'analyse des travaux existants 22
II.3 - Revue de la littérature 24
II.4 - Discussion et positionnement de la recherche 34
II.5 - Conclusion 37

II.1. Introduction

La génération d'applications web à partir de modèles de processus se situe au croisement du BPM, de BPMN et du développement web. BPMN permet de représenter le déroulement du travail, mais il ne suffit pas à décrire les interfaces, les données, les règles d'accès et l'architecture d'un socle applicatif web métier [3].

Ce chapitre analyse donc les travaux qui rapprochent les modèles de processus des artefacts logiciels, afin d'en dégager les apports, les limites et le positionnement de notre recherche.

II.2. Démarche d'analyse des travaux existants

La revue n'a pas cherché à rassembler tous les travaux sur BPMN. Elle suit une démarche structurée, inspirée des recommandations de Kitchenham et Char-

ters et de l'esprit de PRISMA, sans se présenter comme une revue systématique exhaustive [27, 28].

II.2.1. Questions de revue

La question principale est la suivante :

> **RQ0 : Quelles approches permettent de transformer un modèle BPMN ou un modèle de processus en artefacts logiciels web utilisables, et quelles limites subsistent pour produire un socle applicatif structuré, traçable et évolutif ?**

Cette question est déclinée en cinq questions secondaires :

— **RQ1 :** quel rôle BPMN joue-t-il dans la conception et l'exécution des systèmes d'information orientés processus ?
— **RQ2 :** quels artefacts les transformations existantes produisent-elles à partir de BPMN ou de modèles complémentaires ?
— **RQ3 :** quelles informations supplémentaires sont nécessaires pour générer les interfaces, les services, les données et les règles d'accès ?
— **RQ4 :** comment les approches étudiées traitent-elles la traçabilité, la cohabitation avec le code manuel et la régénération ?
— **RQ5 :** quelles limites récurrentes restent ouvertes pour un projet web dont le code source demeure accessible et adaptable ?

II.2.2. Sources consultées et critères de sélection

Les sources consultées appartiennent à trois catégories : les normes et ouvrages de référence sur BPMN, le BPM et l'ingénierie dirigée par les modèles ; les articles de revues et de conférences consacrés aux transformations de modèles, aux applications orientées processus, à IFML, à la génération de code et au low-code ; enfin, les documentations officielles de moteurs BPMN lorsque leur fonctionnement technique devait être précisé [26, 29–31].

Un travail est inclus s'il décrit une méthode, une architecture, une transformation ou un outil reliant les modèles de processus à l'exécution ou à des artefacts applicatifs, ou s'il analyse la maintenance et l'évolution du code généré. Sont exclus les documents purement introductifs, les travaux sans lien direct avec la transformation ou l'exécution de processus, les descriptions ne permettant pas d'identifier les artefacts produits et les doublons d'une même contribution lorsqu'une version plus complète est disponible.

II.2.3. Critères d'analyse

L'analyse s'effectue en trois étapes : comprendre l'objectif de chaque approche, identifier les artefacts réellement produits et examiner les limites annoncées ou observables. La même grille est appliquée aux différentes familles de travaux afin que le chapitre ne se réduise pas à une succession de résumés. Le tableau II présente les critères retenus.

TABLE II – *Grille concrète de lecture des travaux étudiés*

Point observé	Question posée pendant la lecture
Point de départ	Le travail part-il d'un modèle BPMN seul, d'un processus enrichi, d'un modèle de données ou d'un autre artefact ?
Résultat obtenu	Produit-il une interface, une API, une application exécutable, un prototype ou seulement une méthode de transformation ?
Place de la plate-forme	Le résultat dépend-il fortement d'un moteur, d'un framework ou d'un environnement low-code ?
Preuve apportée	Les auteurs valident-ils leur proposition par un cas d'étude, une expérimentation, une comparaison ou une revue ?
Limite retenue	Que reste-t-il à faire pour obtenir un socle applicatif maintenable et adaptable ?

La revue qui suit présente les travaux par familles, puis met en évidence les limites qui se dégagent de leur analyse.

II.3. Revue de la littérature

II.3.1. Systèmes d'information orientés processus et plateformes d'exécution

Les systèmes d'information orientés processus placent le déroulement du travail au centre de l'organisation du logiciel. Ils s'inscrivent dans le domaine plus large de la gestion des processus métier, qui couvre la découverte, la modélisation, l'exécution, le suivi et l'amélioration des processus [1, 32]. Dans cette perspective, le modèle ne sert plus uniquement à documenter une organisation : il peut devenir une définition exécutable utilisée par un moteur de processus.

II.3.1.1. Moteurs BPMS et plateformes d'exécution

Les systèmes de gestion des processus métier, ou *Business Process Management Systems* (BPMS), tels que Camunda, Flowable, jBPM ou Activiti interprètent des définitions de processus, créent des instances, distribuent des tâches

et conservent un historique d'exécution [26, 29–31]. Leur principal apport réside dans la séparation entre la logique d'orchestration et les services appelés au cours du processus. Cette séparation rend le déroulement visible et facilite son suivi. Elle impose néanmoins aux applications clientes de manipuler les notions propres au moteur, notamment les instances, les variables et les tâches. La plateforme exécute ainsi le processus sans nécessairement produire l'ensemble de l'application qui l'entoure.

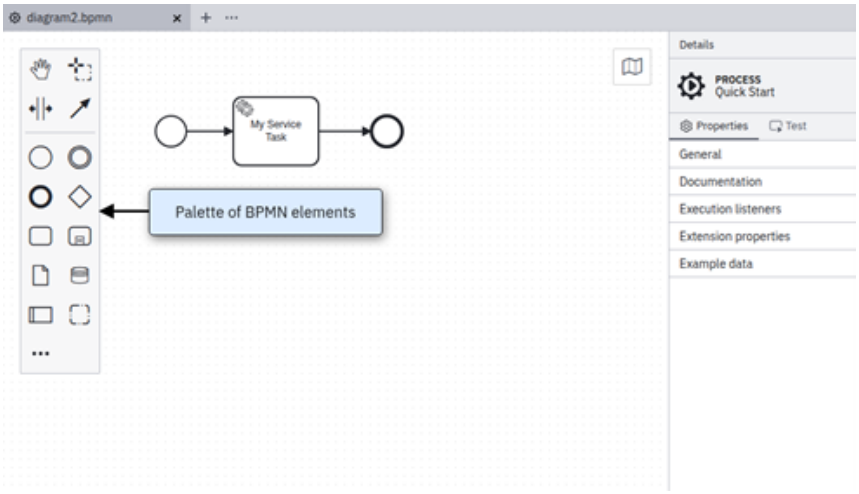


FIGURE 19 – *Modélisation d'un processus BPMN exécutable dans Camunda Modeler, d'après la documentation Camunda [26].*

La figure 19 montre le point d'entrée d'un moteur comme Camunda : un processus BPMN enrichi de propriétés exécutables. Une fois déployé, le moteur crée les instances, affecte les tâches et suit l'exécution, mais il ne génère pas à lui seul toute l'application web qui l'exploite.

II.3.1.2. Architecture web pilotée par BPMN

Schäffer et al. proposent une architecture de référence et une méthode de développement agile pour une plateforme web pilotée par BPMN [33]. Leur travail montre comment associer une application web, un moteur de processus et des services afin de conserver le processus comme fil conducteur de l'implémentation. L'approche apporte une organisation cohérente aux systèmes dont le comportement dépend fortement d'un workflow. Elle reste toutefois centrée sur l'exécution et l'intégration du processus : les autres dimensions de l'application doivent toujours être conçues et développées autour de cette architecture.

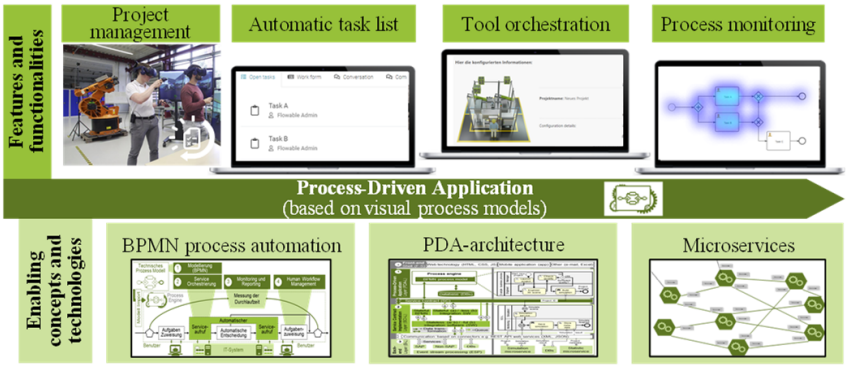


FIGURE 20 – *Fonctionnalités et technologies d'une application web pilotée par processus [33].*

La figure 20 montre qu'une application pilotée par processus combine moteur BPMN, tâches, orchestration, supervision et services. Le moteur de workflow y apparaît central, mais il doit être entouré de composants applicatifs et d'intégration.

II.3.1.3. WebWorkFlow

WebWorkFlow adopte une direction différente. Hemel et al. considèrent qu'un langage de workflow isolé n'est pas suffisant pour générer une application web complète. Ils étendent donc WebDSL avec des abstractions de workflow, puis traduisent ces abstractions vers les constructions du langage sous-jacent [34]. Cette intégration permet de générer des applications personnalisées tout en couvrant les principaux patrons de contrôle de workflow.

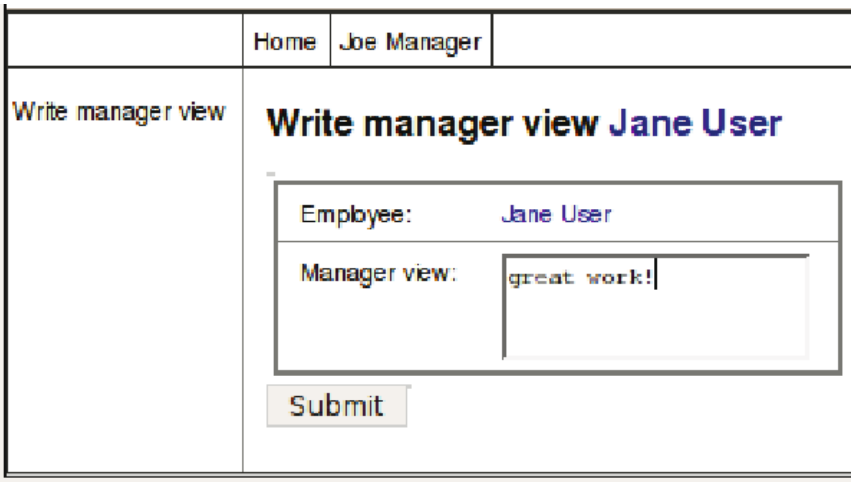


FIGURE 21 – *Interface générée dans WebWorkFlow pour une tâche de workflow, extraite de [34].*

La figure 21 illustre le résultat attendu : une tâche de workflow devient une interface exploitable. L'apport est concret, mais la portée de la génération reste liée au langage et aux abstractions de WebDSL.

II.3.1.4. Plateforme AKIP

La plateforme ouverte AKIP, nom donné par ses auteurs, cherche à produire des applications web orientées processus à partir de BPMN exécutable et de spécifications complémentaires [35]. Son architecture associe un générateur, un frontend, un backend, un moteur de processus, des connecteurs et plusieurs fonctions transversales. Les auteurs répondent notamment aux limites observées dans des plateformes existantes : dépendance envers un fournisseur, interfaces peu flexibles, difficultés d'intégration et technologies devenues anciennes.

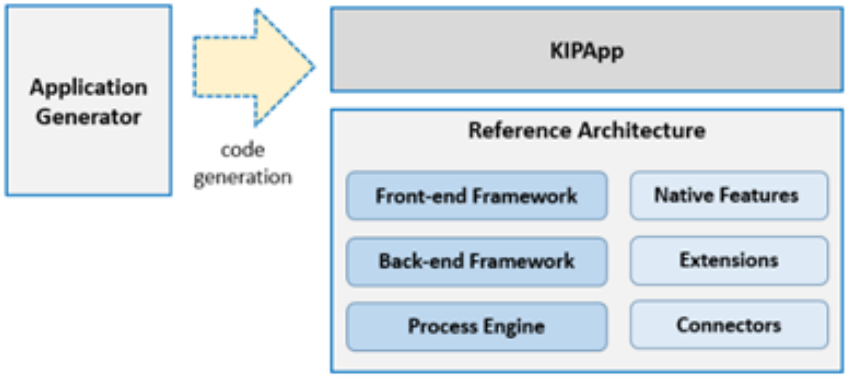


FIGURE 22 – *Architecture et principe de génération d'une application dans AKIP, adaptés de [35].*

La figure 22 montre une génération organisée autour d'une architecture de référence. AKIP produit une base applicative moderne, mais son évaluation reste limitée à une organisation et certaines constructions BPMN ne sont pas encore couvertes.

Ces travaux montrent que l'exécution d'un processus est aujourd'hui bien maîtrisée. Ils montrent également que le moteur ne résout qu'une partie du problème applicatif. Plus une proposition élargit sa couverture, plus elle doit associer au workflow des informations sur le domaine, les interfaces et l'intégration technique.

II.3.2. Dérivation d'artefacts applicatifs à partir de BPMN

Une deuxième famille de recherches utilise BPMN comme point de départ pour dériver des artefacts web précis. Ces travaux ne cherchent pas toujours à générer l'ensemble d'un système. Ils étudient plutôt la manière dont les éléments du processus peuvent fournir des informations utiles à la conception d'interfaces, d'API ou de prototypes.

II.3.2.1. Transformation vers des modèles d'interaction

Sajji et al. proposent une transformation de BPMN vers l'*Interaction Flow Modeling Language* (IFML) dans le cadre de la *Model Driven Architecture* (MDA) [7]. BPMN représente le niveau indépendant du calcul, tandis qu'IFML décrit les vues, les événements utilisateur et les flux d'interaction au niveau indépendant de la plateforme. Le travail établit des règles de correspondance entre les éléments du processus et ceux de l'interface. Il confirme que la structure du workflow peut guider la construction de la navigation, mais que BPMN doit être complété pour exprimer les détails de l'interaction.

Ces mêmes auteurs poursuivent cette logique en étudiant la transformation d'IFML vers Symfony [36]. L'intérêt de ce prolongement est de montrer que le modèle d'interaction peut servir de point de départ à une génération orientée framework web concret. La transformation s'appuie sur des règles ATL, c'est-à-dire un langage utilisé pour définir des transformations entre modèles, pour passer d'un modèle IFML indépendant de la plateforme vers un modèle Symfony plus proche de l'implémentation.

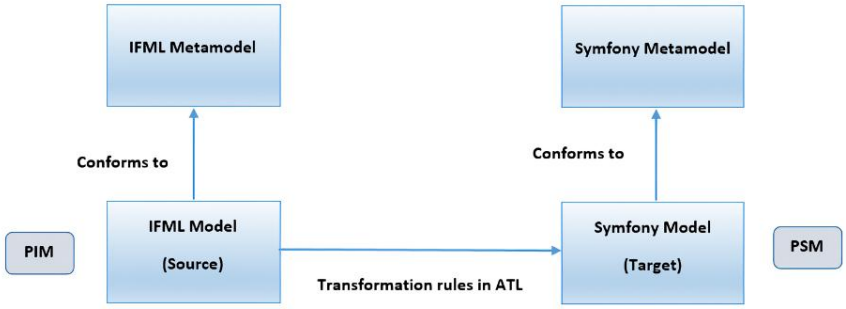


FIGURE 23 – *Transformation d'un modèle IFML vers un modèle Symfony, adaptée de [36].*

La figure 23 illustre la logique MDA : IFML sert de modèle indépendant, puis Symfony rapproche la représentation de l'implémentation. Cette continuité réduit l'écart avec le code, mais renforce aussi la dépendance aux conventions du framework cible.

Laaz et al. enrichissent les modèles BPMN avec des ontologies de domaine avant de produire des modèles IFML [37]. Les ontologies permettent de préciser les concepts métier que le processus manipule sans les surcharger directement dans le diagramme. Cette proposition répond à une faiblesse récurrente de BPMN : les activités et les flux sont explicites, mais la sémantique des données demeure souvent implicite. La qualité du résultat dépend toutefois de la disponibilité et de la qualité de l'ontologie, dont la construction représente elle-même un effort important.

II.3.2.2. Génération de prototypes et d'API

Díaz et al. associent les processus BPMN à des modèles structurels afin de générer des prototypes d'interfaces graphiques [6]. Leur travail montre l'intérêt d'un prototype obtenu tôt dans le cycle de développement : il permet de confronter le processus modélisé aux interactions attendues par les utilisateurs. Le terme de prototype marque cependant la limite assumée de la proposition. L'interface obtenue facilite la validation, mais elle ne constitue pas encore un socle applicatif exploitable couvrant le backend, le frontend, le runtime et les données.

Pietruck et Steffens s'intéressent à la génération d'API conformes au style architectural *Representational State Transfer* (REST) pour les moteurs de workflow, à partir de modèles de processus et de données [38]. Les API produites exposent des opérations plus proches du domaine métier que les interfaces génériques d'un moteur. Cette abstraction réduit le couplage entre les applications clientes et les concepts techniques de son environnement d'exécution. Elle couvre cependant la couche d'accès au processus et aux données, sans générer les interfaces utilisateur ni l'ensemble des fonctions transversales.

II.3.2.3. Vers une génération intégrée des artefacts applicatifs

Hoyos et Restrepo-Calle proposent de générer automatiquement un prototype d'application web orientée processus à partir d'une spécification rédigée dans un langage naturel restreint [39]. Leur chaîne analyse la spécification, extrait les informations utiles, puis produit des classes du domaine, des formulaires, des vues et des contrôleurs.

La figure 24 présente le cycle itératif suivi, depuis l'expression des besoins jusqu'à la validation du prototype.

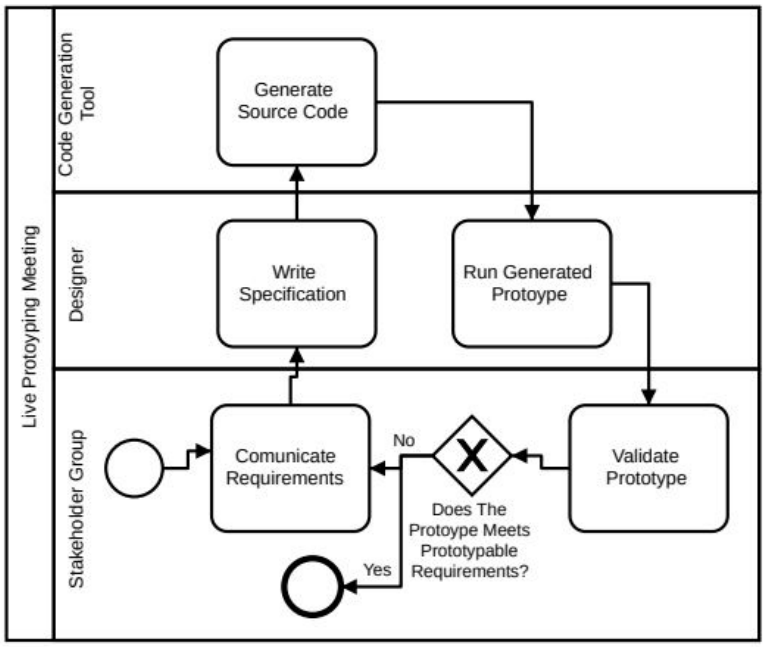


FIGURE 24 – *Cycle de prototypage rapide d'applications web proposé par [39].*

```
@app.route('/projectList/<pid>/showSelectedProject',
methods=['GET','POST'])
def projectList_showSelectedProject(pid):
    viewparams={}
    data = get_instance_data(pid)
    formdata = request.form.copy()
    for file in request.files:
        formdata[file] = request.files[file].filename
    request.form = formdata
    project = session.query(Project).filter(
       Project.id==data.get("selected_Project",0)
    ).first()
    viewparams['project']=project
    task_list = session.query(Task).filter(
        Task.project_id==data.get('selected_Project',0)
    ).all()
    viewparams['task_list']=task_list
```

```
<div class="panel-heading">Show Selected Project</div>
<div class="panel-body">
{% for field in project.__table__.columns._data.keys()%}
<div class="form-group">
       <b>{{field}}</b><br /> {{project[field]}}
</div>
{% endfor %}
<div class="col-lg-12">
<div class="table-responsive">
<table class="table">
<tr>
{% for prop in task_list[0].__mapper__.columns.keys() %}
<th>{{prop}}</th>
{% endfor %}
</tr>
{% for ent in task_list %}
<tr>
{% for prop in ent.__mapper__.columns.keys() %}
<td>{{ent[prop]}}</td> {% endfor %}
</tr>
{% endfor %}
</table></div></div></div>
```

FIGURE 25 – *Extraits de code généré pour le contrôleur et la vue dans l'approche de [39].*

La figure 25 complète le cycle précédent en montrant deux artefacts produits : une route côté contrôleur et un gabarit de vue. Elle confirme que l'approche génère un prototype exploitable, même si la qualité finale dépend encore des règles de génération.

Uyanik et Sayar développent un environnement web qui réunit conception interactive du processus, gestion des flux et génération automatique de code [40]. Dans leur contexte d'évaluation, les auteurs rapportent une amélioration de 98,68 % du

processus de développement, de 95,84 % pour les conditions de test et de 36,01 % concernant la taille du code. Ces résultats soulignent le potentiel d'une chaîne intégrée pour des applications principalement organisées autour de formulaires et de processus. Ils restent toutefois liés au protocole et au domaine évalués ; ils ne permettent pas d'établir que les mêmes gains seraient obtenus pour des architectures plus variées.

Pris ensemble, ces travaux confirment que BPMN contient des informations exploitables pour produire plusieurs parties d'une application web. Ils montrent aussi qu'une transformation ciblée conserve généralement une couverture ciblée : l'interface, l'API ou le prototype progresse, tandis que d'autres préoccupations restent hors du périmètre.

II.3.3. Ingénierie dirigée par les modèles et génération de systèmes orientés processus

L'ingénierie dirigée par les modèles considère les modèles comme des artefacts productifs pouvant être validés, transformés et reliés au code. Cette orientation est particulièrement adaptée aux systèmes orientés processus, car elle permet d'associer plusieurs descriptions spécialisées au lieu d'exiger qu'un seul modèle représente toute l'application.

II.3.3.1. Composition de langages et génération progressive

Drave et al. proposent une démarche générative pour les systèmes d'information orientés processus [9]. Leur approche combine un langage textuel pour BPMN avec des langages spécifiques décrivant notamment les structures de données et les rôles. Les modèles sont contrôlés, transformés en représentations intermédiaires, puis utilisés pour générer un système extensible. Cette composition de langages vise à couvrir à la fois les activités manuelles et automatisées, là où de nombreuses propositions se concentrent sur une seule de ces dimensions.

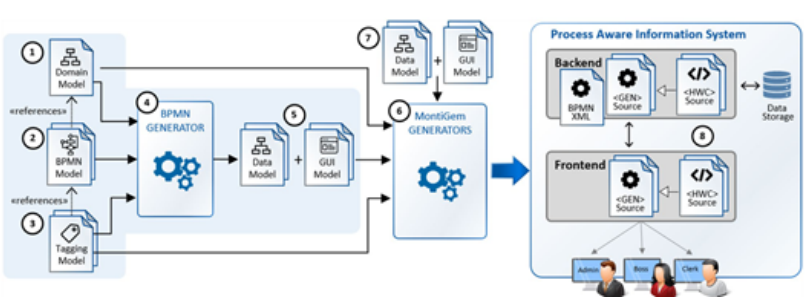


FIGURE 26 – *Génération en deux étapes d'un système d'information orienté processus, adaptée de [9].*

La figure 26 montre une génération en deux étapes qui facilite le contrôle des transformations et la réutilisation des informations. Sa limite tient à la couverture incomplète de BPMN et au coût de maintenance des langages spécifiques mobilisés.

II.3.3.2. Spécification grammaticale des workflows web avec LSAWfP4WEB

Zemtsop Ndadji propose une démarche fondée sur l'ingénierie dirigée par les modèles pour intégrer la gestion des workflows dans une application web. Son travail part du constat que les processus métier décrivent une partie importante des exigences d'une organisation, mais qu'ils restent souvent secondaires dans les cycles classiques de développement web. La contribution porte alors sur une architecture d'application web capable de concilier deux logiques : le développement classique de fonctionnalités et le développement orienté processus. Pour cela, l'auteur introduit une application de base organisée autour d'un module de sécurité RBAC, d'un système de gestion de workflows et d'une bibliothèque de processus. Il propose également LSAWfP4WEB, une extension du langage LSAWfP destinée à spécifier des workflows exécutables dans une application web, ainsi qu'un DSEL JSON pour faciliter l'export, la sauvegarde et le partage des spécifications [41].

La figure 27 présente l'organisation générale de ce système et la place accordée à la gestion des workflows.

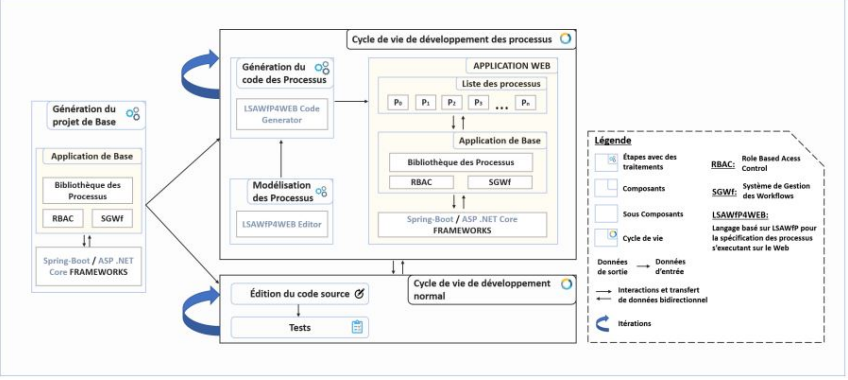


FIGURE 27 – *Architecture du système proposé par Zemtsop Ndadji pour la gestion des workflows dans une application web, adaptée de [41].*

II.3.3.3. Gabarits et compléments aux modèles BPMN

La génération fondée sur des templates, c'est-à-dire des gabarits de fichiers complétés automatiquement, reste l'une des techniques les plus utilisées pour passer d'un modèle vers du code. L'étude de cartographie menée par Syriani et al. montre que cette technique est répandue et qu'elle facilite l'adaptation aux tech-

nologies cibles [42]. Dans le cadre d'une génération d'applications web métier, les gabarits ne sont toutefois utiles que s'ils traduisent correctement les informations du processus en artefacts concrets : routes, contrôleurs, formulaires, services, règles métier et accès aux données. Leur maintenance devient alors un enjeu, car ils doivent évoluer avec les cadres logiciels, les conventions de développement et la structure des modèles d'entrée.

Plusieurs travaux montrent que le modèle de processus doit souvent être complété pour produire une application exploitable. Fedeli et al. introduisent des concepts propres aux environnements enrichis par l'Internet des objets [43], tandis que Salado-Cid et al. insistent sur la représentation explicite des données, des opérations et des dépendances dans les workflows intensifs en données [44]. Ces contributions ne visent pas directement les applications web métier classiques, mais elles soulignent une limite importante de BPMN : le flux de contrôle ne suffit pas toujours à décrire les données manipulées, les interactions utilisateur et les contraintes métier nécessaires à la génération.

Les travaux analysés font apparaître une limite récurrente : BPMN décrit efficacement le flux de contrôle, les activités, les événements, les passerelles et la répartition des responsabilités, mais il ne porte pas, à lui seul, toutes les informations nécessaires à la génération d'une application web métier. Dès que l'on cherche à produire des formulaires, à organiser la persistance, à contrôler les droits d'accès, à définir la visibilité des données ou à relier les sorties d'une activité aux entrées d'une autre, le modèle de processus doit être complété. Selon les approches, ce complément prend la forme de modèles de données, de modèles d'interaction, de métadonnées, de règles propres au domaine, de mappings ou de spécifications supplémentaires. Cette observation montre qu'une approche de génération ne peut pas demander à BPMN de tout décrire ; elle doit plutôt conserver BPMN comme modèle du processus et lui associer une description complémentaire des informations applicatives.

II.3.4. Maintenabilité du code généré et mise en perspective avec le low-code

La plupart des travaux de génération accordent une attention importante au premier passage du modèle vers le code. Or, une application continue d'évoluer après sa génération initiale. Les développeurs ajoutent des règles propres au domaine, corrigent des comportements et adaptent les interfaces. En parallèle, les processus et les technologies cibles changent. La coexistence du code généré et du code écrit manuellement devient alors une question centrale.

II.3.4.1. Gestion conjointe du code généré et du code manuel

Greifenberg et al. comparent plusieurs mécanismes d'intégration du code manuscrit et du code généré, notamment la séparation des fichiers, l'héritage, la délégation et les zones protégées [45]. Leur analyse montre qu'aucun mécanisme ne répond parfaitement à toutes les situations. Certains protègent clairement les modifications manuelles, mais imposent une architecture plus complexe. D'autres offrent davantage de souplesse, au prix d'un risque de conflit lors de la régénération. Le choix doit donc être prévu dès la conception du générateur.

Rahad et al. étudient la place du développeur dans la boucle de l'ingénierie dirigée par les modèles [46]. Ils montrent que l'automatisation ne supprime pas l'intervention humaine : celle-ci se déplace vers la vérification des modèles, l'extension du code et la résolution des écarts entre le résultat généré et le besoin réel. SeamlessMDD poursuit une préoccupation similaire en cherchant à intégrer plus naturellement le code généré aux pratiques habituelles de développement [47]. Ces contributions rappellent que la qualité d'une chaîne générative se mesure aussi après la première génération.

Les travaux consacrés à l'intégration du code généré et du code manuscrit montrent que plusieurs stratégies peuvent être utilisées : séparation des fichiers, héritage, délégation, zones protégées, points d'extension ou mécanismes d'override. Aucune de ces stratégies n'est universelle. Leur pertinence dépend du type d'artefacts générés, du niveau de personnalisation attendu et de la fréquence des régénérations. Cette difficulté confirme que la maintenabilité doit être prise en compte dès la conception d'une chaîne de génération.

II.3.4.2. Mise en perspective avec les approches low-code

Les plateformes low-code cherchent également à réduire l'effort de développement grâce à des modèles visuels, composants et connecteurs [48, 49]. Elles partagent avec l'IDM une recherche d'abstraction et d'automatisation. Toutefois, elles restent souvent dépendantes des plateformes, de leurs métamodèles, de leurs règles de gouvernance et de leurs connecteurs. Dans le cadre de ce mémoire, l'enjeu est différent : il s'agit de conserver des sources explicites, des transformations identifiables et des artefacts dont l'origine peut être inspectée. Cette différence situe le travail dans une logique d'ingénierie dirigée par les modèles plutôt que dans une démarche low-code généraliste.

II.4. Discussion et positionnement de la recherche

La littérature étudiée confirme qu'il est possible d'utiliser un modèle de processus pour exécuter un workflow, produire des interfaces, exposer des API ou générer une base applicative. Elle ne fait toutefois pas apparaître une réponse unique couvrant, avec le même niveau de maturité, toutes les dimensions d'une application web. Les propositions se distinguent surtout par le compromis qu'elles établissent entre couverture, automatisation, flexibilité et dépendance technologique.

Le tableau III montre que les travaux étudiés apportent des réponses utiles, mais rarement complètes. Certains exécutent le processus, d'autres génèrent une interface, une API ou un prototype. Les approches les plus larges produisent une base applicative, mais elles restent liées à une architecture, à un langage ou à une plateforme particulière.

TABLE III – *Récapitulatif des travaux étudiés*

Travail	**Portée et résultat produit**	**Évaluation présentée**	**Limite principale**
WebWorkFlow [34]	Intégration d'abstractions de workflow à WebDSL pour générer des applications web personnalisées	Couverture des principaux patrons de contrôle de workflow	Dépendance au langage WebDSL et à ses abstractions
Sajji et al. [7, 8, 36]	Transformation de BPMN vers IFML, puis d'IFML vers des technologies web comme WebRatio ou Symfony	Cas d'étude d'une application de service après-vente et règles de transformation vers Symfony	Couverture principalement limitée aux interactions et dépendance aux conventions de la plateforme cible
Hoyos et Restrepo-Calle [39]	Génération de classes, formulaires, vues et contrôleurs depuis un langage naturel restreint	Étude de cas démontrant la génération rapide d'un prototype	Langage d'entrée contraint et résultat orienté prototypage
Pietruck et Steffens [38]	Génération d'API REST orientées métier pour des moteurs de workflow	Démonstration de la transformation de modèles de processus et de données	Interfaces utilisateur et fonctions transversales hors du périmètre
Delgado et al. [50]	Gestion dirigée par les modèles des familles de processus BPMN	Représentation de variantes proches d'un même processus	Configuration et contrôle de cohérence plus complexes
AKIP [35]	Génération d'une application web fonctionnelle intégrant interface, serveur et moteur de processus	Étude industrielle portant sur trois processus	Personnalisations métier nécessaires et couverture BPMN partielle

La comparaison des travaux étudiés ne permet pas seulement d'identifier des contributions isolées ; elle met aussi en évidence des limites récurrentes qui traversent plusieurs propositions. Ces limites concernent notamment l'insuffisance des informations portées par BPMN, la couverture partielle des artefacts générés, la dépendance à certaines plateformes, ainsi que les difficultés de traçabilité et de régénération. Le tableau IV synthétise ces limites et en dégage les implications pour le framework à concevoir dans ce mémoire.

TABLE IV – *Limites transversales des travaux étudiés et implications pour la recherche*

Limite observée dans la littérature	**Conséquence pour la génération applicative**	**Implication pour l'approche à concevoir**
BPMN ne contient pas suffisamment d'informations applicatives.	Les formulaires, les données, les rôles, les mappings et les règles de visibilité ne peuvent pas être déduits de manière fiable du diagramme seul.	Prévoir une spécification complémentaire qui enrichit BPMN sans lui faire porter toute la description technique de l'application.
Les travaux génèrent souvent des fragments : interface, API ou prototype.	Certaines couches de l'application restent hors du périmètre de génération et doivent être développées séparément.	Viser une génération couvrant plusieurs couches du socle applicatif : interface, logique applicative, runtime, données et sécurité.
Certaines approches dépendent fortement d'une plateforme, d'un langage ou d'un framework.	Le résultat obtenu devient difficile à transposer ou à faire évoluer hors de l'environnement prévu initialement.	Séparer autant que possible les décisions métier des détails technologiques, au moyen de représentations intermédiaires et de règles de projection explicites.
La traçabilité entre modèles et fichiers générés reste souvent limitée.	Lorsqu'un fichier est modifié ou régénéré, il devient difficile de retrouver l'élément du modèle qui l'a produit.	Conserver des liens explicites entre les modèles sources, les représentations intermédiaires et les artefacts produits.
La cohabitation entre code généré et code manuel est difficile.	Les adaptations du développeur peuvent être perdues ou entrer en conflit avec une nouvelle génération.	Organiser dès le départ la séparation entre éléments générés, extensions manuelles et mécanismes de personnalisation.
Les évaluations sont souvent limitées à un cas d'étude ou à un contexte spécifique.	La généralisation des résultats doit rester prudente et dépend du domaine, du modèle et de l'environnement technique.	Évaluer l'approche sur un cas suffisamment représentatif, tout en reconnaissant clairement les limites de généralisation.

II.5. Conclusion

Les travaux étudiés montrent que les modèles de processus peuvent guider l'exécution d'un workflow et soutenir la production d'interfaces, d'API, de prototypes ou de bases applicatives. Ils confirment aussi que BPMN constitue un point d'entrée pertinent pour représenter le fonctionnement métier. Cependant, ils révèlent plusieurs limites persistantes : le modèle BPMN doit être complété par des informations applicatives, la couverture générée reste souvent partielle, certaines solutions demeurent liées à une plateforme ou à un langage, la traçabilité modèle–code reste fragile et l'évolution du code généré demeure délicate.

Ces exigences structurent la conception du framework ProOWeb, présenté au chapitre suivant comme la solution proposée dans ce mémoire pour générer un socle applicatif web métier à partir de modèles BPMN enrichis.

III

CHAPITRE

APPROCHE D'INGÉNIERIE PROPOSÉE POUR LA GÉNÉRATION D'APPLICATIONS WEB PILOTÉES PAR BPMN

SOMMAIRE

III.1 - Introduction 38
III.2 - Vue d'ensemble de l'approche d'ingénierie 39
III.3 - Architecture globale du framework proposé 40
III.4 - Modèle d'entrée enrichi : spécification spec-v1 46
III.5 - Règles de transformation et génération de l'application web 57
III.6 - Préservation du code personnalisé et synchronisation 63
III.7 - Conclusion 66

III.1. Introduction

Ce chapitre expose le cœur de la contribution : l'approche d'ingénierie proposée et sa concrétisation dans le framework ProOWeb. Il présente la vue d'ensemble de la démarche, l'architecture du framework et celle de l'application web générée. Il décrit ensuite le modèle d'entrée BPMN–*spec-v1*, la grammaire et les propriétés de la spécification, puis les transformations M2M et M2T. La chaîne de traçabilité et les mécanismes qui préparent une régénération contrôlée complètent cette présentation. L'objectif est de préciser le passage des modèles aux artefacts sans assimiler le résultat à une application directement exploitable en production.

III.2. Vue d'ensemble de l'approche d'ingénierie

III.2.1. Problème traité et principe général

L'approche d'ingénierie proposée définit la démarche qui relie le processus métier au projet cible ; ProOWeb est le framework qui met cette démarche en œuvre. L'application générée en constitue le résultat : un socle applicatif web métier que l'équipe de développement doit vérifier, adapter et compléter.

BPMN est conservé comme modèle du déroulement, car il représente les activités, les événements, les passerelles et les flux. Il ne porte cependant pas toutes les informations nécessaires aux formulaires, aux affectations, à la visibilité, au stockage ou aux contrôles d'accès. La spécification *spec-v1* complète donc le BPMN sans se substituer à lui. Les deux sources sont validées ensemble, puis normalisées dans un modèle pivot afin que les gabarits n'aient pas à réinterpréter directement des structures hétérogènes.

La chaîne sépare enfin la transformation M2M, qui construit les contrats du modèle pivot, de la transformation M2T, qui écrit les fichiers du workspace. Cette séparation permet de contrôler les décisions avant leur projection vers le code. Le manifeste et les empreintes prolongent cette organisation après la première production : ils préparent la comparaison entre les modèles mis à jour, les fichiers gérés et les adaptations introduites dans le projet cible.

III.2.2. Positionnement dans le cycle MDA

La Model-Driven Architecture fournit un cadre pertinent pour comprendre la progression proposée. ProOWeb ne reprend pas ce cadre comme une classification abstraite ; il l'utilise pour clarifier les responsabilités de chaque niveau de la chaîne de génération.

Le tableau V relie les représentations utilisées par le framework aux niveaux d'abstraction retenus dans ce mémoire.

TABLE V – *Positionnement de ProOWeb dans le cycle MDA*

Niveau	Contenu dans ProOWeb	Rôle dans la génération
PIM	BPMN et spécification *spec-v1*	Décrire le workflow, les rôles, les entrées, les sorties, la visibilité et les données sans dépendre directement du code cible.
PSM	Contrats intermédiaires, catalogues et registres	Stabiliser les informations dans une forme exploitable par les générateurs d'interface, de logique applicative et de gestion des données.
Code	Projet applicatif comprenant interface, logique métier, données, sécurité et configuration	Produire un socle applicatif structuré, versionnable et personnalisable.

Cette correspondance sépare les décisions indépendantes des technologies, leur normalisation dans les contrats intermédiaires et leur projection vers le projet cible. Elle situe ainsi les transformations M2M et M2T dans la progression retenue par ProOWeb.

III.2.3. Contribution méthodologique de l'approche

La contribution de ce travail se situe à l'intersection de trois préoccupations. La première est la modélisation du processus. Le BPMN reste la référence du workflow et permet de maintenir une continuité avec les pratiques de modélisation métier [41]. La deuxième est la génération d'une application web métier. Le framework ProOWeb ne se limite pas à produire des écrans ou des services isolés ; il organise un projet cible avec ses modules fonctionnels et techniques. La troisième est la maintenance du code généré. La régénération est traitée comme une opération contrôlée, accompagnée par un manifeste, des sauvegardes, des rapports et des mécanismes d'extension.

ProOWeb vise donc un résultat exploitable par une équipe de développement : un socle applicatif web métier généré à partir du processus, traçable et ouvert à l'adaptation. Le framework automatise les parties décrites par ses modèles et ses gabarits ; il ne prétend ni déduire les règles absentes des sources ni supprimer le travail d'intégration du développeur.

III.3. Architecture globale du framework proposé

L'architecture du framework ProOWeb est organisée autour de quatre blocs : les modèles d'entrée, le noyau de génération, l'application web générée, puis son exploitation et son évolution. La figure 28 situe ces blocs et distingue ce qui est fourni par le concepteur, transformé par le framework, écrit dans le projet cible et repris après la génération.

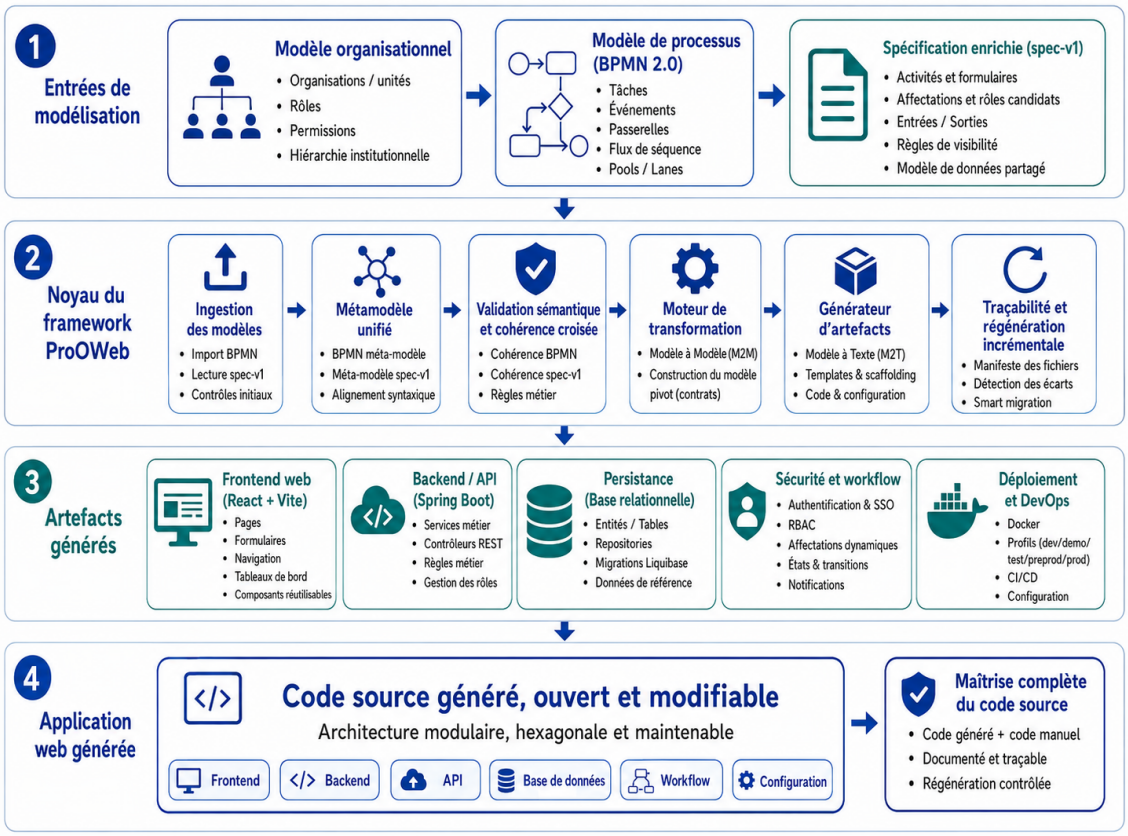


FIGURE 28 – *Architecture générale de ProOWeb*

III.3.1. Modèles d'entrée de ProOWeb

Le premier bloc de l'architecture constitue le point de départ de la génération. Il regroupe le modèle BPMN, la spécification *spec-v1*, les informations organisationnelles et la configuration du projet cible. Le modèle BPMN décrit le déroulement du processus à travers les activités, événements, passerelles et flux, tandis que la spécification *spec-v1* apporte les éléments nécessaires à l'application, notamment les rôles, les données, les règles d'affectation, de stockage et de visibilité. Les informations organisationnelles relient les tâches aux acteurs concernés, alors que la configuration précise les options générales du projet à générer. Ce bloc joue ainsi le rôle de source de vérité : ProOWeb y lit, valide et transforme les informations métier afin de préserver la traçabilité entre le modèle initial et le code produit.

III.3.2. Noyau de génération du framework

Le noyau transforme les modèles d'entrée en représentations exploitables par les générateurs au moyen de plusieurs traitements : ingestion du BPMN, enrichissement par la spécification *spec-v1*, validation croisée, construction du modèle pivot et génération des fichiers applicatifs. L'ingestion extrait les activités, événements, passerelles et flux du processus, puis l'enrichissement associe à chaque activité les rôles, données, règles de visibilité et comportements définis dans la spécification. La validation vérifie ensuite la cohérence structurelle entre le BPMN et *spec-v1*, afin d'éviter de propager des références incompatibles dans les fichiers. Après cette étape, ProOWeb construit un modèle pivot composé de contrats intermédiaires, tels que le *runtime contract*, le *data contract*, les catalogues d'interface et le registre des processus. Ces contrats structurent les transformations M2M et M2T nécessaires à la production du code source.

III.3.3. Application web générée et architecture logique du socle applicatif

L'application web générée constitue le résultat principal de la chaîne mise en œuvre par ProOWeb. Elle ne correspond pas à un simple assemblage de fichiers ou de fragments isolés, mais à un projet web métier structuré autour du processus et des acteurs concernés. Le socle produit rassemble les vues de traitement, les listes de tâches, les formulaires, les interfaces de suivi et les espaces de supervision. Il comprend également les services nécessaires au démarrage d'un processus, à l'exécution des activités, à la gestion des transitions, à la persistance des données et au contrôle des droits d'accès. Les données peuvent être rattachées à une instance ou conservées comme informations partagées selon les règles définies dans *spec-v1*. L'interface, la logique applicative, les données, la sécurité, le runtime, la configuration et le suivi forment ainsi un même projet cible, sans que celui-ci soit présenté comme directement prêt pour un déploiement en production.

Cette vue générale est précisée par l'architecture logique du socle applicatif. Elle décrit les responsabilités internes du projet généré, les échanges entre ses composants et les règles de dépendance qui les organisent, indépendamment des langages, des frameworks, des bibliothèques ou des mécanismes retenus lors de l'implémentation.

La vue adoptée est inspirée de l'architecture hexagonale. Le cœur métier et les services applicatifs sont placés au centre. Les interactions avec les acteurs, les données et les systèmes de l'organisation passent par des ports, puis par des adaptateurs chargés de traduire les échanges. Cette séparation limite la dépendance du cœur en-

vers les interfaces, la persistance et les services externes. La figure 29 présente les composants logiques retenus et leurs principales relations.

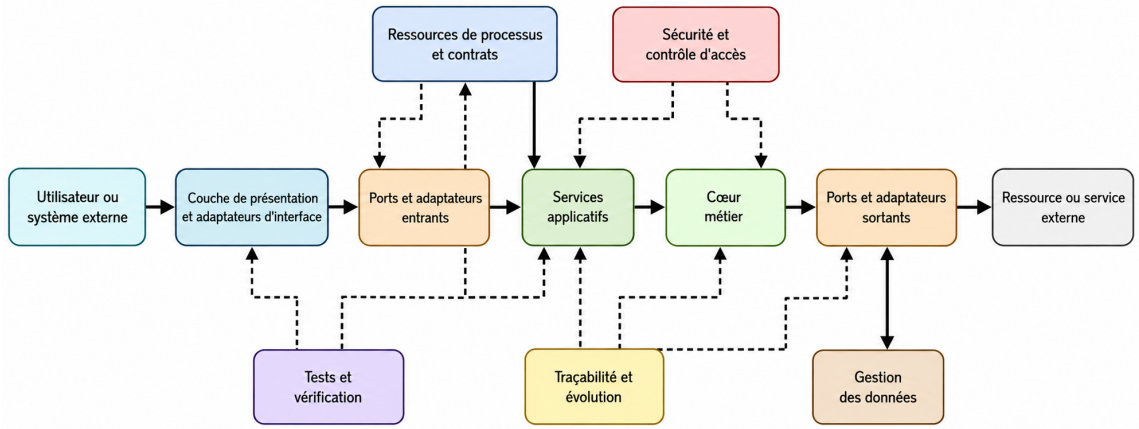


FIGURE 29 – *Architecture logique du socle applicatif web métier généré*

Couche de présentation et adaptateurs d'interface. Cette partie accueille les éléments générés avec lesquels les utilisateurs interagissent : pages, vues, formulaires, tableaux de consultation, composants de saisie et écrans de suivi. Elle comprend également les interfaces permettant de démarrer un processus, de traiter une activité et de superviser l'avancement d'une instance. Leur contenu est projeté à partir des activités, des rôles et des données décrits par BPMN et *spec-v1*.

Ports et adaptateurs entrants. Les ports entrants définissent les opérations accessibles depuis une interface utilisateur ou un autre système. Les contrôleurs, les points d'entrée applicatifs, les interfaces d'accès aux cas d'utilisation et les mécanismes de validation des requêtes sont placés dans cette partie. Ils traduisent notamment les demandes de démarrage, de traitement d'une activité et de consultation d'une instance, puis les transmettent aux services applicatifs sans introduire de règle propre au domaine.

Services applicatifs. Cette partie reçoit les services générés ou configurés pour coordonner les cas d'utilisation. Elle orchestre le démarrage d'une instance, la progression du processus, l'affectation des activités, la consultation de l'état courant et les échanges avec le *runtime contract* et le *data contract*. Les services applicatifs organisent ces opérations, mais délèguent au cœur métier les invariants et décisions qui relèvent du domaine.

Cœur métier. Le cœur rassemble les concepts indépendants des mécanismes d'interface et de stockage : processus, instances, activités, rôles, affectations, règles de visibilité, données métier, entités partagées et contraintes applicatives. Il ne dépend ni de la présentation, ni de la persistance, ni des services externes. ProOWeb ne produit pas nécessairement l'intégralité de ce cœur pour chaque processus : les structures génériques appartiennent au socle stable, tandis que les éléments propres au processus sont projetés depuis BPMN, *spec-v1* et le modèle pivot.

Ports et adaptateurs sortants. Les ports sortants expriment les besoins du cœur applicatif envers son environnement. Ils regroupent les interfaces de persistance, les dépôts de données, les mécanismes de notification, l'accès aux utilisateurs et aux rôles, ainsi que les connecteurs nécessaires aux échanges avec les systèmes de l'organisation. Les adaptateurs associés réalisent ces échanges sans imposer leurs détails techniques au cœur métier.

Gestion des données. Cette partie reçoit les éléments générés qui décrivent les modèles de données, les structures persistantes, les entités partagées, les mappings et les contraintes de stockage. Elle comprend également les éléments nécessaires à l'initialisation et à l'évolution des structures de données. Les accès sont présentés au cœur par des ports sortants, ce qui maintient la gestion physique des données en dehors des règles métier.

Ressources de processus et contrats. Ce composant conserve le modèle BPMN, la spécification *spec-v1*, le *runtime contract*, le *data contract*, les catalogues de processus, les informations de version et les correspondances nécessaires à l'exécution. BPMN reste responsable du déroulement du processus, tandis que *spec-v1* apporte les rôles, les données, les règles d'affectation et les informations de visibilité nécessaires au socle applicatif. Les contrats issus du modèle pivot rendent ces informations exploitables par les services applicatifs.

Sécurité et contrôle d'accès. La sécurité constitue une responsabilité transversale plutôt qu'un composant isolé. Les éléments projetés concernent les rôles autorisés à démarrer une instance, les rôles candidats, l'affectation des activités, la supervision, la visibilité des activités et des données, ainsi que le contrôle des opérations exposées. Ces règles s'appliquent aux interfaces, aux ports et aux services applicatifs tout en restant traçables vers *spec-v1*.

Tests et vérification. Cette partie rassemble les catégories de tests générés ou préparés pour vérifier les parcours utilisateurs, le démarrage du processus, l'exécution des activités, la visibilité, les échanges de données, la conformité des contrats et l'intégration entre les composants. Ces vérifications contribuent à contrôler le socle produit dans le périmètre étudié ; elles ne suffisent pas à démontrer, à elles seules, son aptitude à un déploiement industriel.

Traçabilité et évolution. Ce composant regroupe les informations qui relient BPMN, *spec-v1*, le modèle pivot, les contrats et les artefacts générés. Il permet d'identifier l'origine d'un élément, de distinguer les parties gérées des personnalisations, de conserver les empreintes et de préparer les rapports d'évolution ainsi que la régénération contrôlée. Cette responsabilité traverse les différentes couches, car une modification des modèles peut affecter plusieurs composants du projet cible.

Le cheminement principal suit alors la chaîne suivante : une action provenant d'un utilisateur ou d'un système externe traverse un adaptateur entrant, puis un service applicatif qui sollicite le cœur métier. Lorsque l'opération nécessite une interaction avec l'environnement, le cœur s'appuie sur un port sortant, dont l'adaptateur dialogue avec la ressource externe concernée. Les modèles et les contrats alimentent les services applicatifs, tandis que la sécurité, les tests et la traçabilité interviennent de manière transversale. Cette organisation permet à ProOWeb de produire un socle applicatif web métier fonctionnel, structuré, traçable et ouvert à l'évolution. Le projet cible reste toutefois une base que le développeur doit vérifier, adapter et compléter selon les contraintes de l'organisation et de son environnement de déploiement.

III.3.4. Exploitation et évolution de l'application

Après la génération, l'application est utilisée par les acteurs métier pour exécuter les tâches, par les superviseurs pour suivre les instances, par les administrateurs pour gérer les paramètres et par les développeurs pour adapter le code lorsque des besoins spécifiques apparaissent. Cette dimension évolutive est essentielle, car un processus, une règle d'affectation, un champ ou une interface peuvent être modifiés au cours du temps. ProOWeb prend donc en compte la régénération à travers le suivi des fichiers produits, la détection des modifications manuelles, la création de sauvegardes et la production de rapports de migration. L'architecture forme ainsi une boucle itérative reliant les modèles, le code généré et les adaptations réalisées.

III.4. Modèle d'entrée enrichi : spécification spec-v1

La spécification *spec-v1* constitue le complément applicatif du modèle BPMN. Elle rend explicites les informations nécessaires à la génération d'un socle applicatif web métier : rôles autorisés, supervision, stratégies d'affectation, données d'entrée, sorties, règles de visibilité, stockage et comportement des activités automatiques.

III.4.1. Grammaire formelle de spec-v1

Le modèle BPMN conserve la structure de contrôle du processus : activités, événements, passerelles et flux de séquence [3]. Ces éléments ne suffisent toutefois pas à déterminer les rôles autorisés, les règles d'affectation, les sources de données, leur visibilité ou leur mode de stockage. Dans ProOWeb, *spec-v1* forme donc un langage complémentaire qui porte ces informations applicatives. Sa grammaire fixe les constructions admises avant la validation sémantique et la transformation vers le modèle pivot. Cette séparation entre langage source, validation et transformation rejoint le principe de modèles explicitement structurés et manipulables propre à l'IDM [5].

Définition de la grammaire

Une grammaire formelle est décrite par un quadruplet composé de non-terminaux, de terminaux, de productions et d'un symbole de départ. Pour *spec-v1*, nous retenons la définition suivante :

$$G_{\text{spec-v1}} = (N, T, P, S), \qquad S = \langle Specification \rangle. \tag{III.1}$$

L'ensemble N contient les catégories syntaxiques à développer, T les symboles qui apparaissent dans une spécification, P les productions qui organisent ces symboles et S l'axiome à partir duquel une spécification complète est dérivée. La grammaire adopte une forme EBNF abstraite : elle décrit les concepts de *spec-v1* sans les rattacher à un format de sérialisation ni à l'organisation physique d'un projet.

Les principaux non-terminaux sont répartis en trois groupes :

— les blocs de processus, avec $\langle Specification \rangle$, $\langle Start \rangle$, $\langle Monitors \rangle$ et $\langle Activities \rangle$;

— l'enrichissement d'une activité, avec $\langle Activity \rangle$, $\langle Assignment \rangle$, $\langle AutomaticExecution \rangle$, $\langle Input \rangle$, $\langle Source \rangle$, $\langle Output \rangle$, $\langle Mapping \rangle$ et $\langle Visibility \rangle$;

— le modèle de données partagé, avec ⟨*SharedData*⟩, ⟨*Entity*⟩, ⟨*Field*⟩ et ⟨*Relation*⟩.

⟨*Specification*⟩ représente le document complet. ⟨*Start*⟩ décrit son démarrage, ⟨*Monitors*⟩ sa supervision et ⟨*Activities*⟩ la collection des activités enrichies. Pour chaque activité, ⟨*Activity*⟩ rassemble les informations applicatives ; ⟨*Assignment*⟩ formalise son affectation et ⟨*AutomaticExecution*⟩ les modalités propres à une exécution automatique. Les non-terminaux ⟨*Input*⟩ et ⟨*Source*⟩ décrivent les provenances des valeurs, tandis que ⟨*Output*⟩ et ⟨*Mapping*⟩ précisent leur destination. ⟨*Visibility*⟩ porte les droits de consultation. Enfin, ⟨*SharedData*⟩ regroupe les ⟨*Entity*⟩ ; chacune est décrite par ses ⟨*Field*⟩ et ses éventuelles ⟨*Relation*⟩.

L'ensemble T comprend d'abord les mots-clés de *spec-v1*, les symboles de ponctuation, les identifiants, les chaînes, les booléens et les entiers positifs. Il contient aussi les domaines finis effectivement contrôlés par la spécification :

— types d'activités : `MANUAL` et `AUTOMATIC` ;
— modes d'affectation : `AUTOMATIC` et `MANUAL` ;
— stratégies d'affectation : `ROLE_QUEUE`, `SUPERVISOR_ONLY`, `SUPERVISOR_THEN_ANCESTORS`, `UNIT_MEMBERS`, `SINGLE_MATCH_ONLY`, `MANUAL_ONLY` et `ROUND_ROBIN` ;
— sources d'entrée : `PREVIOUS_ACTIVITY`, `PROCESS_CONTEXT`, `SHARED_DATA`, `BACKEND_SERVICE` et `EXTERNAL_SERVICE` ;
— modes de stockage : `INSTANCE`, `SHARED` et `BOTH` ;
— types de champs : `STRING`, `TEXT`, `INTEGER`, `LONG`, `DECIMAL`, `BOOLEAN`, `DATE`, `DATETIME`, `JSON` et `UUID` ;
— types de relations : `MANY_TO_ONE`, `ONE_TO_MANY`, `ONE_TO_ONE` et `MANY_TO_MANY`.

Pour les activités automatiques, le domaine terminal contient également les modes de déclenchement `MANUAL_TRIGGER`, `IMMEDIATE` et `DEFERRED`. Les clés telles que `schemaVersion`, `start`, `monitors`, `activities`, `assignment`, `input`, `output`, `visibility` et `sharedData` servent de repères structurels. Cette organisation par clés nommées rend chaque bloc identifiable sans l'associer à un langage de programmation ou à une organisation de fichiers. Les catégories lexicales ⟨*Identifier*⟩, ⟨*DataPath*⟩, ⟨*String*⟩, ⟨*Boolean*⟩ et ⟨*PositiveInteger*⟩ désignent respectivement une référence nommée, un chemin de donnée, une chaîne, un booléen et un entier strictement positif. Les non-terminaux ⟨*ActivityType*⟩, ⟨*AssignmentMode*⟩, ⟨*AssignmentStrategy*⟩, ⟨*SourceType*⟩, ⟨*StorageType*⟩, ⟨*FieldType*⟩, ⟨*RelationType*⟩ et ⟨*TriggerMode*⟩ prennent exactement leurs valeurs dans les domaines finis énumérés ci-dessus.

Productions principales

Les productions ci-dessous décrivent la syntaxe abstraite normalisée de *spec-v1*.

```
<Specification> ::= <SchemaVersion> <Start> <Monitors>
                    <Activities> <SharedData>

<SchemaVersion> ::= "schemaVersion" "=" "1"

<Start> ::= "start" {
              <StartableRoles>
              [ <StartActivities> ]
              [ <AutoStart> ]
            }
<StartableRoles> ::= "startableByRoles" "=" <RoleList>
<StartActivities> ::= "startActivities" "=" <ActivityIdList>
<AutoStart> ::= "allowAutoStartWhenNoManualEntry" "=" <Boolean>

<Monitors> ::= "monitors" { <MonitorRoles> }
<MonitorRoles> ::= "monitorRoles" "=" <RoleList>

<Activities> ::= "activities" { [ <ActivityList> ] }
<ActivityList> ::= <ActivityEntry>
                 | <ActivityEntry> "," <ActivityList>
<ActivityEntry> ::= <ActivityId> "=" <Activity>

<Activity> ::= {
                 <ActivityType>
                 <CandidateRoles>
                 <Assignment>
                 [ <AutomaticExecution> ]
                 <Input>
                 <Output>
                 <Visibility>
               }
<ActivityType> ::= "activityType" "=" <ActivityTypeValue>
<CandidateRoles> ::= "candidateRoles" "=" <RoleList>

<Assignment> ::= "assignment" {
                   <AssignmentMode>
                   <AssignmentStrategy>
                   <PreviousAssigneePolicy>
                   <ManualAssignerRoles>
                   <MaximumAssignees>
                 }
<AssignmentMode> ::= "mode" "=" <AssignmentModeValue>
<AssignmentStrategy> ::= "strategy" "=" <AssignmentStrategyValue>
<PreviousAssigneePolicy> ::=
    "allowPreviouslyAssignedAssignee" "=" <Boolean>
<ManualAssignerRoles> ::= "manualAssignerRoles" "=" <RoleList>
<MaximumAssignees> ::= "maxAssignees" "=" <PositiveInteger>

<AutomaticExecution> ::= "automaticExecution" {
                           <HandlerReference>
                           <TaskTypeKey>
```

```
                          <TriggerMode>
                          [ <DeferredDelay> ]
                          <Configuration>
                        }
<HandlerReference> ::= "handlerRef" "=" <Identifier>
<TaskTypeKey> ::= "taskTypeKey" "=" <Identifier>
<TriggerMode> ::= "triggerMode" "=" <TriggerModeValue>
<DeferredDelay> ::= "deferredDelayMinutes" "=" <PositiveInteger>
<Configuration> ::= "configuration" "=" <StructuredValue>

<Input> ::= "input" { "sources" "=" [ <SourceList> ] }
<SourceList> ::= <Source> | <Source> "," <SourceList>
<Source> ::= {
               <SourceType>
               <SourceReference>
               <InputMappings>
             }
<SourceType> ::= "sourceType" "=" <SourceTypeValue>
<SourceReference> ::= "sourceRef" "=" <DataPath>
<InputMappings> ::= "mappings" "=" [ <MappingList> ]

<Output> ::= "output" {
               <StorageType>
               <OutputMappings>
             }
<StorageType> ::= "storage" "=" <StorageTypeValue>
<OutputMappings> ::= "mappings" "=" [ <MappingList> ]
<MappingList> ::= <Mapping> | <Mapping> "," <MappingList>
<Mapping> ::= { "from" "=" <DataPath> "to" "=" <DataPath> }

<Visibility> ::= "visibility" {
                   <ActivityViewerRoles>
                   <DataViewerRoles>
                 }
<ActivityViewerRoles> ::= "activityViewerRoles" "=" <RoleList>
<DataViewerRoles> ::= "dataViewerRoles" "=" <RoleList>

<SharedData> ::= "sharedData" { "entities" "=" [ <EntityList> ] }
<EntityList> ::= <Entity> | <Entity> "," <EntityList>
<Entity> ::= {
               <EntityKey>
               [ <DisplayName> ]
               [ <TableName> ]
               <Fields>
               [ <Relations> ]
             }
<EntityKey> ::= "entityKey" "=" <Identifier>
<DisplayName> ::= "displayName" "=" <String>
<TableName> ::= "tableName" "=" <Identifier>
<Fields> ::= "fields" "=" <FieldList>
<FieldList> ::= <Field> | <Field> "," <FieldList>
<Field> ::= {
              "name" "=" <Identifier>
              "type" "=" <FieldType>
              "required" "=" <Boolean>
```

```
                "indexed" "=" <Boolean>
                "unique" "=" <Boolean>
              }

<Relations> ::= "relations" "=" <RelationList>
<RelationList> ::= <Relation> | <Relation> "," <RelationList>
<Relation> ::= {
                  "name" "=" <Identifier>
                  "type" "=" <RelationType>
                  "targetEntityKey" "=" <Identifier>
                  [ "mappedBy" "=" <Identifier> ]
                  [ "joinColumn" "=" <Identifier> ]
                  [ "joinTable" "=" <Identifier> ]
                  [ "inverseJoinColumn" "=" <Identifier> ]
                  "required" "=" <Boolean>
                }

<RoleList> ::= [ <RoleSequence> ]
<RoleSequence> ::= <Role> | <Role> "," <RoleSequence>
<ActivityIdList> ::= [ <ActivityIdSequence> ]
<ActivityIdSequence> ::= <ActivityId>
                       | <ActivityId> "," <ActivityIdSequence>

<ActivityTypeValue> ::= "MANUAL" | "AUTOMATIC"
<AssignmentModeValue> ::= "AUTOMATIC" | "MANUAL"
<AssignmentStrategyValue> ::= "ROLE_QUEUE"
                            | "SUPERVISOR_ONLY"
                            | "SUPERVISOR_THEN_ANCESTORS"
                            | "UNIT_MEMBERS"
                            | "SINGLE_MATCH_ONLY"
                            | "MANUAL_ONLY"
                            | "ROUND_ROBIN"
<SourceTypeValue> ::= "PREVIOUS_ACTIVITY" | "PROCESS_CONTEXT"
                    | "SHARED_DATA" | "BACKEND_SERVICE"
                    | "EXTERNAL_SERVICE"
<StorageTypeValue> ::= "INSTANCE" | "SHARED" | "BOTH"
<TriggerModeValue> ::= "MANUAL_TRIGGER" | "IMMEDIATE" | "DEFERRED"
<FieldType> ::= "STRING" | "TEXT" | "INTEGER" | "LONG"
              | "DECIMAL" | "BOOLEAN" | "DATE" | "DATETIME"
              | "JSON" | "UUID"
<RelationType> ::= "MANY_TO_ONE" | "ONE_TO_MANY"
                 | "ONE_TO_ONE" | "MANY_TO_MANY"
<Boolean> ::= "true" | "false"
```

Listing III.1 – *Productions EBNF principales de la grammaire spec-v1*

Les productions restent volontairement indépendantes des technologies retenues pour le prototype. Elles expriment ce que la chaîne de génération doit connaître, et non la manière dont un contrôleur, une vue ou un mécanisme de persistance sera programmé. Cette distinction permet à *spec-v1* de jouer le rôle d'un langage dédié à la description applicative, tandis que le BPMN demeure la source du comportement processuel. La séparation entre structure grammaticale et

traitements qui l'exploitent est également conforme aux principes de l'ingénierie des grammaires [51].

Contraintes syntaxiques et sémantiques

Les productions P déterminent la structure d'une spécification. La conformité syntaxique impose la présence des constructions obligatoires, l'unicité des propriétés nommées dans leur groupe et l'appartenance des valeurs contrôlées aux domaines terminaux précédents. Les identifiants doivent respecter leur forme lexicale, les listes doivent être finies et chaque mapping doit contenir une source `from` et une cible `to` non vides.

Une dérivation syntaxiquement correcte ne suffit pas pour autoriser la génération. Soient B le modèle BPMN importé, A_B l'ensemble de ses activités, A_s les clés de `activities` et E_s les clés des entités partagées. La validation applique notamment les contraintes suivantes :

1. la version de schéma vaut 1 et toute activité explicitement enrichie vérifie $A_s \subseteq A_B$; les identifiants placés dans `startActivities` appartiennent également à A_B ;
2. une activité de type `MANUAL` possède au moins un rôle candidat ; lorsque son affectation est manuelle, la liste `manualAssignerRoles` n'est pas vide ;
3. les listes `activityViewerRoles` et `dataViewerRoles` ne sont pas vides, et les rôles qu'elles contiennent doivent être compatibles avec le référentiel retenu par le projet ;
4. une activité `AUTOMATIC` désigne un gestionnaire et un type de tâche connus ; le mode `DEFERRED` exige en outre un délai positif ;
5. toute source `SHARED_DATA` et toute cible de sortie stockée en mode `SHARED` ou `BOTH` désignent une entité de E_s ;
6. les clés d'entités, les noms de champs et les noms de relations sont uniques dans leur portée. Toute relation cible une entité déclarée ; les paramètres de jointure doivent être cohérents avec son type.

Une contrainte bloquante non satisfaite interrompt la promotion de la spécification vers les contrats intermédiaires. Cette validation établit une cohérence structurelle entre BPMN et *spec-v1* ; elle ne constitue pas une preuve de correction complète des règles métier.

Ainsi définie, la grammaire $G_{\text{spec-v1}}$ rend explicite le langage complémentaire consommé par ProOWeb. Le BPMN continue de décrire le déroulement du processus, alors que *spec-v1* précise les informations applicatives nécessaires à la

construction du modèle pivot puis à la génération du socle web métier. Le passage du modèle de processus au projet cible repose donc sur deux sources coordonnées, dont les responsabilités restent distinctes.

III.4.2. Propriétés de la grammaire spec-v1

Soient B un modèle BPMN fini, s une spécification *spec-v1* et T_{M2M} la transformation qui construit le modèle pivot P. On note $\mathrm{Act}(B)$ l'ensemble des activités de B, $\mathrm{Act}(s)$ celui des activités enrichies de s et $\mathrm{id}(x)$ l'identifiant stable d'un élément x. Les propositions suivantes portent sur la structure de la chaîne ; elles ne démontrent pas la correction métier du processus.

Pour préciser la classification obtenue, nous distinguons le langage syntaxique $L_{\mathrm{syn}} = L(G_{\text{spec-v1}})$ du langage validé

$$L_{\mathrm{val}} = \{\mathrm{enc}(B) \# \mathrm{enc}(s) \mid s \in L_{\mathrm{syn}} \wedge C(B,s)\},$$

où enc désigne une représentation finie d'un modèle ou d'une spécification, et $C(B,s)$ l'ensemble des contraintes syntaxiques et sémantiques présentées précédemment. Cette distinction sépare la forme de *spec-v1* des correspondances que ProOWeb doit vérifier avec le BPMN.

Proposition 1 (Nature hors-contexte de la syntaxe) *La grammaire $G_{\text{spec-v1}} = (N,T,P,S)$ est une grammaire hors-contexte, ou grammaire de type 2 dans la hiérarchie de Chomsky.*

Preuve. Toute production de P possède à gauche un unique non-terminal $A \in N$ et s'écrit

$$A ::= \alpha, \qquad \alpha \in (N \cup T)^*.$$

Les notations EBNF d'optionalité et de groupement peuvent être remplacées par des non-terminaux auxiliaires sans modifier cette forme. Aucune règle ne fait dépendre sa partie gauche d'un contexte terminal ou non-terminal. La définition d'une grammaire hors-contexte est donc satisfaite. □

Proposition 2 (Caractère attribué) *Munie des informations héritées du modèle BPMN et des valeurs synthétisées lors de la validation, $G_{\text{spec-v1}}$ est une grammaire attribuée.*

Preuve. À chaque non-terminal $X \in N$, on associe un attribut hérité $X.\mathrm{env}$ et des attributs synthétisés $X.\mathrm{valide}$ et $X.\mathrm{trace}$. L'environnement contient notamment les

activités de B, les rôles retenus, les entités partagées et les types de tâches automatiques connus. Les productions portent alors des équations sémantiques. Par exemple, pour

$$\langle ActivityEntry \rangle ::= \langle ActivityId \rangle \texttt{"="} \langle Activity \rangle,$$

on définit

$$\langle ActivityEntry \rangle.\,\text{valide} = \big(\langle ActivityId \rangle.\,\text{lexeme} \in \text{id}(\text{Act}(B))\big) \wedge \langle Activity \rangle.\,\text{valide},$$

et l'attribut trace reçoit l'identifiant de l'activité BPMN correspondante. De la même manière, une production de mapping synthétise la validité de sa cible, tandis que les productions d'affectation et de visibilité vérifient leurs rôles dans l'environnement hérité. Ces valeurs sont calculées à partir des constituants de la production et transmises au modèle pivot. Les productions syntaxiques sont donc complétées par des attributs et des règles sémantiques, ce qui établit le caractère attribué de la grammaire [51, 52]. □

Proposition 3 (Non-régularité du langage validé) *Si le domaine lexical des identifiants d'activités n'est pas borné, le langage L_{val} des couples BPMN/spec-v1 acceptés par les contraintes d'attributs n'est pas régulier.*

Preuve. Considérons le sous-ensemble dans lequel le BPMN contient une unique activité et où *spec-v1* enrichit cette même activité, tous les autres constituants étant fixés. Restreignons les identifiants aux mots a^n, avec $n \geq 1$. Cette restriction est décrite par un langage régulier R. La contrainte de correspondance des identifiants impose alors, après suppression des terminaux fixes par un homomorphisme h,

$$h(L_{\text{val}} \cap R) = K = \{a^n \sharp a^n \mid n \geq 1\}.$$

Supposons K régulier et soit p sa longueur de pompage. Pour le mot $w = a^p \sharp a^p$, toute décomposition $w = xyz$ telle que $|xy| \leq p$ et $|y| > 0$ place y uniquement dans le premier bloc de a. Le pompage avec l'exposant 0 diminue donc seulement ce premier bloc ; le mot xz n'appartient plus à K, ce qui contredit le lemme de pompage. Ainsi, K n'est pas régulier. Or les langages réguliers sont fermés par intersection avec un langage régulier et par homomorphisme. Si L_{val} était régulier, K le serait également, d'où une contradiction. La non-régularité résulte donc des égalités de références imposées par les attributs, et non de la seule forme EBNF des productions. □

Proposition 4 (Récursivité bornée) *La récursivité de $G_{spec\text{-}v1}$ est limitée à la construction des listes finies de rôles, d'activités, de sources, de mappings, d'entités et de champs. Pour toute spécification finie s, l'analyse de ces productions récursives se termine.*

Preuve. Chaque liste d'éléments X est définie par une production droite récursive de la forme

$$\langle XList \rangle ::= \langle X \rangle \quad | \quad \langle X \rangle \texttt{","} \langle XList \rangle.$$

Soient $L_1, \ldots, L_q$ les listes contenues dans s et

$$\rho_0 = \sum_{i=1}^{q} |L_i|.$$

Le document étant fini, $\rho_0 \in \mathbb{N}$. À chaque application de la seconde production, l'analyse consomme exactement un élément de la liste courante ; le nombre d'éléments restant vérifie donc $\rho_{k+1} = \rho_k - 1$. Lorsque $\rho_k = 0$, seule la production vers ε est applicable. L'ordre usuel sur $\mathbb{N}$ étant bien fondé, aucune analyse guidée par S ne peut produire une récursion infinie. □

Proposition 5 (Non-ambiguïté structurelle conditionnelle) *Supposons que chaque propriété nommée apparaisse au plus une fois dans son groupe et que les valeurs discriminantes, notamment* `MANUAL` *et* `AUTOMATIC`*, appartiennent aux domaines définis par la grammaire. Alors toute spécification $s \in L(G_{spec\text{-}v1})$ possède un unique arbre syntaxique pour les blocs structurants de spec-v1.*

Preuve. On raisonne par induction sur la hauteur h de l'arbre de dérivation de s. Pour $h = 0$, une valeur atomique appartient à un domaine terminal déterminé. Supposons la dérivation unique pour toute construction de hauteur au plus h. Une construction de hauteur $h+1$ est introduite par un terminal nommé tel que `start`, `monitors`, `activities`, `assignment`, `input`, `output`, `visibility` ou `sharedData`. Ce terminal sélectionne une production unique. Pour une liste, la virgule sépare un premier élément du suffixe reconnu par la même production récursive. L'hypothèse d'induction assure alors l'unicité des sous-arbres. L'arbre de dérivation de la spécification est donc unique. Cette propriété exclut l'ambiguïté structurelle sous les hypothèses posées ; elle ne préjuge pas d'une éventuelle ambiguïté du sens métier. □

Au regard de ces propriétés, la grammaire de la spécification *spec-v1* est hors-contexte attribuée, non régulière et syntaxiquement non ambiguë sous les conditions d'unicité énoncées

III.4.3. Métadonnées des activités

Les métadonnées précisent comment une activité déjà identifiée dans le BPMN doit être prise en charge par le projet cible. Une activité manuelle indique les rôles candidats, l'affectation, les données visibles et les sorties attendues ; une activité automatique précise le gestionnaire, le déclenchement et sa configuration. Ces informations alimentent notamment `ActivityModel`, `FormModel`, le *runtime contract* et le *data contract*. Le listing III.2 en montre une forme compacte.

```
"activite_1": {
  "activityType": "MANUAL",
  "candidateRoles": ["ROLE_1"],
  "assignment": {
    "mode": "AUTOMATIC",
    "strategy": "ROLE_QUEUE"
  },
  "input": {
    "sources": [
      {"sourceType": "PROCESS_CONTEXT", "sourceRef": "process",
       "mappings": []}
    ]
  },
  "output": {
    "storage": "BOTH",
    "mappings": []
  },
  "visibility": {
    "activityViewerRoles": ["ROLE_1", "ROLE_2"],
    "dataViewerRoles": ["ROLE_1"]
  }
}
```

Listing III.2 – *Activité enrichie dans spec-v1*

Dans cet exemple, la clé `activite_1` conserve l'identifiant de l'activité BPMN. Les rôles candidats alimentent la politique d'affectation, les sources d'entrée préparent le formulaire et la stratégie de stockage oriente les sorties vers le contexte d'instance ou les données partagées. Les mêmes métadonnées sont ensuite projetées vers la page d'activité, les contrôles d'accès et les contrats consommés par le runtime.

III.4.4. Affectation des tâches et sécurité

L'affectation complète l'information organisationnelle portée par les pools et les couloirs BPMN. Dans *spec-v1*, les rôles candidats, le mode et la stratégie d'assignation précisent quels utilisateurs peuvent recevoir ou traiter une tâche. Le modèle pivot rassemble ces règles dans `RolePolicy`, puis les générateurs les projettent

vers les files de tâches, les filtres backend et les contrôles d'accès de l'interface. Le tableau VI résume les stratégies prises en charge.

TABLE VI – *Stratégies d'affectation*

Stratégie	Interprétation dans ProOWeb
`ROLE_QUEUE`	La tâche est visible dans une file associée aux rôles candidats.
`UNIT_MEMBERS`	La tâche est proposée aux membres d'une unité organisationnelle admissible.
`SUPERVISOR_ONLY`	La tâche est réservée aux rôles de supervision.
`SUPERVISOR_THEN_ANCESTORS`	La tâche peut être escaladée dans une chaîne hiérarchique lorsque la règle locale ne suffit pas.

Ces stratégies ne modifient pas le chemin défini par BPMN : elles déterminent qui peut agir lorsque le runtime atteint l'activité concernée. Leur validité organisationnelle dépend du référentiel de rôles fourni au framework.

III.4.5. Entrées, sorties et données partagées

Pour une activité BPMN enrichie, les entrées indiquent d'où viennent les données nécessaires à son exécution. ProOWeb distingue le contexte de l'instance, les résultats d'une activité antérieure, les données partagées et les informations obtenues auprès d'un service backend ou externe. Le champ `sourceRef` identifie la source précise et les mappings indiquent comment ses valeurs alimentent les données locales de l'activité.

Les sorties décrivent les données produites lorsque l'activité est achevée. Un mapping associe un chemin produit à un chemin cible. La stratégie `INSTANCE` conserve la valeur dans l'instance courante, `SHARED` la dirige vers une entité partagée et `BOTH` applique les deux formes de stockage. La sortie d'une activité peut ainsi devenir l'entrée d'une activité ultérieure.

Le bloc `sharedData` définit les entités et les champs accessibles à plusieurs activités du modèle. Il constitue l'espace d'échange durable du processus : une activité peut y publier une donnée par un mapping de sortie, puis une autre peut la lire en déclarant une source de type `SHARED_DATA`. Le tableau VII résume ces mécanismes de circulation.

TABLE VII – *Entrées, sorties et données partagées d'une activité BPMN enrichie*

Élément de spec-v1	Signification pour l'activité	Circulation de la donnée
`PROCESS_CONTEXT`	Données déjà présentes dans l'instance du processus.	Du contexte courant vers l'entrée de l'activité.
`PREVIOUS_ACTIVITY`	Données produites par une activité antérieure désignée par `sourceRef`.	De la sortie de l'activité source vers l'entrée de l'activité courante.
`SHARED_DATA`	Valeurs lues dans une entité partagée déclarée.	De l'espace partagé vers une ou plusieurs activités.
Service backend ou externe	Données résolues par un service applicatif ou un système tiers.	Du service invoqué vers l'entrée de l'activité.
`output.mappings`	Correspondances entre les chemins produits et leurs cibles.	De la sortie de l'activité vers l'instance ou une entité partagée.
`output.storage`	Choix entre `INSTANCE`, `SHARED` et `BOTH`.	Détermine où les valeurs produites sont conservées.
`sharedData.entities`	Schéma des données métier communes aux activités.	Une activité publie la valeur ; d'autres activités peuvent ensuite la consulter.

Après validation, ces déclarations alimentent `DataContract` et les modèles de formulaires. La transformation M2T peut alors produire les structures d'entrée, les mappings, les entités partagées, les dépôts et les scripts de migration sans déduire de données absentes de la spécification.

III.4.6. Règles de visibilité

La visibilité ne se réduit pas à l'affectation : un utilisateur peut consulter une instance sans traiter sa tâche courante, et un superviseur peut suivre le processus sans accéder à toutes les données produites. BPMN conserve le déroulement ; *spec-v1* distingue les rôles autorisés à voir l'activité de ceux qui peuvent lire ses données. Le modèle pivot intègre ces restrictions dans `RolePolicy`. Elles deviennent ensuite des filtres dans les services applicatifs et des règles d'affichage dans le frontend, sans faire de l'interface le seul niveau de protection.

III.5. Règles de transformation et génération de l'application web

Les règles de transformation déterminent la manière dont le double modèle d'entrée devient un socle applicatif web métier. Dans ProOWeb, la transformation M2M construit le modèle applicatif intermédiaire, tandis que la transforma-

tion M2T produit les fichiers du projet cible. Ce découpage permet de vérifier les contrats avant l'écriture du code.

III.5.1. Validation avant génération

La validation est une étape indispensable. Une erreur dans les modèles d'entrée peut se propager dans plusieurs fichiers générés : point d'accès applicatif, composant d'interface, transition, structure de données ou règle de sécurité. Le framework doit donc refuser ou signaler les situations ambiguës avant de produire le projet cible.

Les contrôles portent sur quatre dimensions. La dimension structurelle vérifie que les activités enrichies existent dans le BPMN et que les transitions forment un graphe exploitable. La dimension applicative vérifie que les activités manuelles disposent d'informations suffisantes pour être traitées par un utilisateur. La dimension données vérifie que les entrées, sorties et mappings référencent des structures connues. La dimension sécurité vérifie la cohérence des rôles utilisés pour l'exécution, la supervision et la visibilité.

Ces contrôles doivent produire des diagnostics compréhensibles. L'utilisateur du framework doit pouvoir identifier l'activité, la règle ou la donnée responsable de l'erreur.

III.5.2. Transformation M2M

La transformation M2M reçoit le modèle BPMN, *spec-v1*, le référentiel de rôles et la configuration du projet. ProOWeb extrait les activités, les événements, les passerelles, les flux, les pools et les couloirs du BPMN, puis lit les enrichissements applicatifs. Il contrôle l'identité du processus, l'existence des activités référencées, les rôles, les affectations, les entrées, les sorties, les mappings et la visibilité. Une erreur bloquante interrompt la transformation avant la production d'un modèle exploitable.

Après validation, chaque activité est normalisée avec son type, sa politique d'affectation, ses sources de données, ses sorties et son modèle de formulaire. Le graphe BPMN fournit ensuite les transitions du `RuntimeContract`, tandis que les entités partagées et les mappings alimentent le `DataContract`. Les identifiants du modèle, de sa version et des activités sont conservés pour assurer la traçabilité vers les futurs artefacts.

Algorithme de transformation M2M

L'algorithme 1 présente la transformation M2M. Celui-ci reçoit le modèle BPMN, la spécification *spec-v1*, le référentiel de rôles et la configuration du projet. Il produit le modèle pivot et un rapport de validation ; lorsqu'une erreur bloquante subsiste, aucun pivot exploitable n'est promu vers l'étape M2T.

Algorithm 1: TransformationM2M(B, S, R, C)

Entrées :
  B : modèle BPMN
  S : spécification `spec-v1`
  R : référentiel de rôles
  C : configuration du projet
Sorties :
  P : modèle pivot ProOWeb
  V : rapport de validation
Début
$V \leftarrow$ creerRapportValidation()
$E \leftarrow$ extraireBPMN(B)
  `// activités, événements, passerelles, flux, pools, lanes`
$D \leftarrow$ lireBlocsSpecV1(S)
  `// processId, version, start, monitors, activities, sharedData`
verifierIdentiteProcessus(D.processId, E.processus, C, V)
verifierExistenceActivites(D.activities, E.activities, V)
verifierRolesEtAffectations(D, R, V)
verifierEntreesSortiesMappings(D, V)
verifierVisibilite(D, R, V)
if V.contientErreurBloquante() **then**
  return (INDEFINI, V)
end
$P \leftarrow$ creerModelePivot(C)
P.processModel $\leftarrow$ creerProcessModel(E.processus, E.pools, E.lanes)
foreach *activité a de* E.activities **do**
  $e \leftarrow$ chercherEnrichissement(a.id, D.activities)
  $a_p \leftarrow$ creerActivityModel(a, e)
  a_p.rolePolicy $\leftarrow$ construireRolePolicy(e, R)
  a_p.formModel $\leftarrow$ construireFormModel(e.input, e.output)
  ajouter(P.processModel, a_p)
end
P.runtimeContract $\leftarrow$
  construireRuntimeContract(P.processModel, E.events, E.gateways, E.flows, D.start)
P.dataContract $\leftarrow$ construireDataContract(D.sharedData, D.activities.mappings)
P.artifactTraces $\leftarrow$
  construireArtifactTraces(B, S, P.processModel, P.runtimeContract, P.dataContract)
ajouterDiagnosticsNormalisation(V, P)
return (P, V)
Fin

Son fonctionnement s'organise autour de trois opérations. ProOWeb extrait d'abord la structure du processus et lit les blocs utiles de *spec-v1*. Il vérifie ensuite l'identité du processus, les activités enrichies, les rôles, les mappings et la visibilité ; une erreur bloquante interrompt la transformation. Les éléments validés

sont enfin normalisés dans le modèle de processus, les politiques d'affectation et les formulaires, puis dans les *runtime contract* et *data contract*. Les traces d'artefacts conservent l'origine des éléments destinés à la génération. Le pivot désigne ici cet ensemble cohérent de contrats, et non nécessairement un fichier unique. Dans le prototype, le contrôle des codes de rôle est effectif, mais leur confrontation exhaustive à un référentiel organisationnel externe reste à consolider.

Construction du modèle applicatif intermédiaire

Le modèle applicatif intermédiaire regroupe uniquement les informations nécessaires à la génération : déroulement du processus, rôles, données, formulaires et traces des artefacts produits. Il sert de contrat entre l'analyse du BPMN, l'enrichissement *spec-v1* et les gabarits de génération. Le tableau VIII en présente les principales structures.

TABLE VIII – *Structures principales du modèle pivot ProOWeb*

Structure conservée	Utilité dans ProOWeb
`ProcessModel`	Garde la référence au processus BPMN, à ses activités et à ses transitions.
`ActivityModel`	Transforme chaque activité BPMN enrichie en unité exploitable par l'application.
`RolePolicy`	Regroupe les rôles, règles d'affectation, droits d'accès et restrictions de visibilité.
`FormModel`	Décrit les champs affichés, les sorties produites et les validations des formulaires.
`RuntimeContract`	Fournit au moteur d'exécution les états, transitions et actions disponibles.
`DataContract`	Prépare les entités, champs, mappings et règles de stockage des données.
`ArtifactTrace`	Relie les éléments du modèle aux fichiers générés afin de contrôler les régénérations.

Ces structures séparent le processus, les règles d'accès, les formulaires, les données et les traces de génération. Les gabarits peuvent ainsi consommer une représentation validée sans réinterpréter directement le BPMN ou *spec-v1*.

III.5.3. Transformation M2T

La transformation M2T reçoit le modèle pivot, les gabarits, la configuration et le workspace cible. Elle projette d'abord le *runtime contract* et le *data contract*, puis produit les ressources backend, frontend, de sécurité, de configuration et de test. Les gabarits traduisent les décisions déjà stabilisées dans le pivot : ils ne doivent introduire ni rôle, ni transition, ni règle métier absente des contrats.

Les artefacts sont ordonnés par chemin avant leur écriture afin de conserver une structure reproductible. Pour chaque fichier géré, ProOWeb calcule une empreinte SHA-256 et enregistre son chemin, sa catégorie et ses propriétaires logiques dans le manifeste. Lorsqu'un fichier existant a été personnalisé, les services de déploiement et de migration détectent le conflit et créent, selon la politique appliquée, une sauvegarde avant tout remplacement contrôlé.

Algorithme de transformation M2T

L'algorithme 2 présente la transformation M2T. Il reçoit le modèle pivot validé, les gabarits, la configuration et le workspace cible, puis retourne les artefacts écrits et le manifeste qui permet de les suivre.

Algorithm 2: TransformationM2T(P, T, C, W)

Entrées :
  P : modèle pivot ProOWeb
  T : ensemble de gabarits de génération
  C : configuration du projet
  W : workspace cible
Sorties :
  A : artefacts applicatifs générés
  M : manifeste de génération
Début
$W \leftarrow$ initialiserOuChargerWorkspace(W, C)
$M_0 \leftarrow$ chargerManifesteSiPresent(W)
$A \leftarrow$ ensembleVide$()$
$M \leftarrow$ creerManifeste(C)
ajouter$(A,$ genererRessourcesRuntime$(P.$runtimeContract$, T, C))$
ajouter$(A,$ genererRessourcesDonnees$(P.$dataContract$, T, C))$
ajouter$(A,$ genererBackend$(P, T, C))$
  `// entités, services, contrôleurs, registres, règles applicatives`
ajouter$(A,$ genererFrontend$(P, T, C))$
  `// pages, formulaires, tableaux de bord, clients API`
ajouter$(A,$ genererAccesEtVisibilite$(P, T, C))$
ajouter$(A,$ genererConfigurationScriptsEtTests$(P, T, C))$
foreach *artefact a de* trierParChemin(A) **do**
  cible $\leftarrow$ resoudreChemin$(W, a.$chemin$)$
  etat $\leftarrow$ classifierFichier$($cible$, M_0)$
  if etat $=$ PERSONNALISE **then**
    signalerConflitSansEcraser$($cible$, a)$
  end
  else
    ecrireFichier$($cible$, a.$contenu$)$
    $h \leftarrow$ calculerEmpreinte$(a.$contenu$)$
    enregistrer$(M, \{$chemin : $a.$chemin$,$ categorie : $a.$categorie$,$ proprietaire : $a.$proprietaireLogique$,$ source : $a.$artifactTrace$,$ hash : $h\})$
  end
end
sauvegarderManifeste(W, M)
return (A, M)
Fin

L'algorithme initialise le workspace, projette le *runtime contract* et le *data contract*, puis génère les ressources backend, frontend, de sécurité, de configuration et de test. Les artefacts sont traités selon un ordre stable afin de conserver une structure reproductible. Pour chaque fichier effectivement écrit, ProOWeb calcule une empreinte et enregistre son chemin, sa catégorie, son propriétaire logique dans le manifeste. Le lien avec la source est déduit de ces propriétaires et des chemins versionnés ; le manifeste actuel ne sérialise pas un champ `ArtifactTrace` autonome. Une personnalisation détectée n'est jamais remplacée silencieusement : selon la politique de migration, ProOWeb la signale ou crée une sauvegarde avant remplacement. Cette protection mobilise les services de déploiement et de migration en complément du rendu des gabarits. La transformation M2T reste donc une projection : elle matérialise les décisions du pivot sans introduire de rôle, de transition ou de règle métier absente des contrats.

III.5.4. Règles de projection vers les artefacts

Les règles de projection définissent comment chaque élément du modèle pivot est traduit dans l'application cible. Elles ne sont pas de simples conventions de nommage. Elles indiquent la couche de destination et la responsabilité de l'artefact produit. Le tableau IX résume ces correspondances.

TABLE IX – *Règles de projection vers l'application générée*

Élément du modèle pivot	**Artefact généré**	**Couche cible**
Activité manuelle	Formulaire, page d'activité, structure d'entrée	Interface utilisateur et couche applicative.
Activité automatique	Opération système ou service applicatif	Couche applicative.
Rôle candidat	Règle d'accès et filtre de tâche	Sécurité, services applicatifs et interface.
Transition du workflow	Méthode de progression et changement d'état	Runtime du processus.
Entité partagée	Structure métier, stockage, migration	Domaine et persistance.
Mapping de sortie	Service de sauvegarde et projection cible	Application et persistance.

Cette projection maintient la cohérence entre le modèle et le projet produit. Le générateur ne crée pas seulement des fichiers ; il garde le lien entre chaque élément généré et le besoin métier qui l'a rendu nécessaire. Ce lien rend le code plus compréhensible lorsqu'il faut l'adapter ou le régénérer.

III.5.5. Traçabilité de bout en bout

La chaîne suivie par ProOWeb peut être résumée ainsi :

BPMN $\longrightarrow$ *spec-v1* $\longrightarrow$ modèle pivot $\longrightarrow$ contrats intermédiaires $\longrightarrow$ artefacts $\longrightarrow$ manifeste.

Elle est illustrée sur l'activité BPMN `activite_1`. L'objectif est de montrer comment une même clé reste reconnaissable depuis le processus jusqu'aux fichiers produits, sans répéter la structure complète du modèle pivot. Le tableau X suit cette clé à chaque niveau.

TABLE X – *Traçabilité d'une activité BPMN vers les artefacts générés*

Niveau suivi	Interprétation dans ProOWeb
BPMN	La tâche `activité_1` donne le point de passage métier et ses liens avec le reste du processus.
spec-v1	L'activité reçoit les précisions absentes du diagramme : rôle autorisé, données utiles, sorties attendues et visibilité.
Modèle pivot	ProOWeb rassemble ces informations dans une représentation stable, utilisable par les règles de génération.
Contrats intermédiaires	Le *runtime contract* et le *data contract* reprennent l'identifiant, les transitions, les rôles et les mappings validés.
Artefacts générés	Les fichiers produits matérialisent la même activité : écran de traitement, action de validation, contrôle d'accès et mise à jour des données.
Manifeste	Les fichiers rattachés à cette activité restent identifiables afin de maîtriser les futures régénérations.

Cette continuité permet d'expliquer l'origine d'un fichier, de vérifier quels artefacts matérialisent une activité et de circonscrire les éléments concernés lorsque le BPMN ou *spec-v1* évolue. Le manifeste prolonge la chaîne au niveau du workspace : il ne démontre pas la correction métier, mais fournit les informations nécessaires pour comparer, signaler et traiter les changements.

III.6. Préservation du code personnalisé et synchronisation

Le projet cible évolue après sa première production : le processus ou *spec-v1* peut changer, le développeur ajoute du code, l'interface est personnalisée et certaines règles métier sont précisées lors de l'intégration. Une nouvelle génération doit donc rapprocher deux états sans considérer le workspace comme un réper-

toire jetable. Le framework ProOWeb traite cette opération comme un problème de synchronisation entre modèles, fichiers gérés et extensions locales.

Cette préoccupation est bien identifiée dans les travaux sur la cohabitation entre code généré et code écrit à la main. Greifenberg et al. comparent plusieurs mécanismes d'intégration et montrent que le choix d'une stratégie influence directement la maintenabilité des systèmes générés [45].

III.6.1. Problème de la régénération

La régénération vise à répercuter une évolution du BPMN, des rôles, des données ou des projections de *spec-v1*. Elle devient risquée dès qu'un fichier produit a été modifié manuellement : un remplacement aveugle peut effacer une adaptation, désynchroniser le frontend et le backend ou rendre les tests caducs.

Le problème ne se limite pas à la perte de code. Une régénération non contrôlée peut produire des conflits de fichiers, des incohérences entre la partie applicative et l'interface, la disparition de règles spécifiques, un changement non documenté dans la sécurité ou une rupture des tests. ProOWeb distingue donc les fichiers gérés par le générateur et les fichiers non gérés. Les fichiers gérés sont suivis dans le manifeste `.prooweb-managed.json`. Les fichiers non gérés restent hors du contrôle du générateur.

III.6.2. Manifeste et contrat de propriété

Le manifeste `.prooweb-managed.json` décrit les fichiers produits par ProOWeb. Pour chaque fichier, il peut conserver le chemin, la catégorie, les propriétaires logiques et une empreinte. Cette information permet de savoir si un fichier correspond toujours à la dernière version générée ou s'il a été modifié manuellement.

Ce manifeste définit un contrat de propriété. Un fichier généré et non modifié peut être remplacé lors d'une nouvelle génération. Un fichier généré mais modifié doit être traité avec prudence. Un fichier non géré ne doit pas être écrasé. Cette règle donne au développeur une garantie pratique : ProOWeb ne considère pas le projet cible comme un répertoire jetable.

III.6.3. Préservation côté logique applicative

Côté logique applicative, la préservation est particulièrement importante, car le code généré porte les API, la sécurité, la persistance et le runtime. ProOWeb limite le risque en produisant une architecture modulaire. Les éléments issus du modèle sont générés dans des couches précises, tandis que les adaptations locales peuvent être placées dans des modules ou points d'extension séparés.

Cette stratégie rejoint le principe du *Generation Gap Pattern*, souvent utilisé pour séparer les classes générées des classes écrites manuellement [45]. Le générateur peut produire un squelette ou une classe de base, tandis que le développeur ajoute un comportement spécifique dans une classe séparée. Lors d'une régénération, la partie contrôlée par ProOWeb peut évoluer sans supprimer les extensions manuelles.

La préservation ne doit toutefois pas servir à contourner le modèle. Si une modification concerne le workflow, les rôles, les données structurantes ou les règles de visibilité, elle devrait être reportée dans le BPMN ou dans *spec-v1*. Les extensions manuelles doivent plutôt couvrir les détails d'intégration, les règles particulières ou les adaptations difficiles à généraliser.

III.6.4. Préservation côté interface utilisateur

Côté interface utilisateur, la personnalisation concerne souvent les libellés, les parcours utilisateur, les composants visuels, les contrôles d'ergonomie et certaines intégrations. ProOWeb génère les composants liés au processus, mais l'équipe doit pouvoir adapter l'expérience utilisateur sans perdre ses modifications lors d'un redéploiement.

La stratégie retenue repose sur la composition, la configuration et les overrides de templates. Les composants générés peuvent être encapsulés par des composants propres au projet. Les règles d'override déclarées peuvent remplacer, préfixer, suffixer ou modifier un bloc de contenu généré. Cette personnalisation est réappliquée lors des générations suivantes, ce qui réduit les modifications répétées directement dans les fichiers produits.

III.6.5. Cycle de synchronisation itératif

Le cycle de synchronisation décrit la manière dont ProOWeb accompagne les évolutions successives du modèle. Une première génération produit l'application de base. Les développeurs peuvent ensuite ajouter des extensions, adapter certains modules ou déclarer des overrides. Lorsque le BPMN ou *spec-v1* évolue, ProOWeb reconstruit un snapshot de génération, compare l'état attendu avec l'état du projet, applique les règles de migration et produit un rapport.

Ce cycle transforme la génération en activité continue. La synchronisation repose sur le manifeste, les empreintes, les sauvegardes, les points d'extension, les overrides et les rapports de migration. Ces éléments rendent les changements observables et permettent d'appliquer une politique explicite lorsqu'un conflit est détecté. Le tableau XI en résume les responsabilités.

TABLE XI – *Mécanismes de préservation et de synchronisation dans ProOWeb*

Mécanisme	Rôle	Effet attendu
Manifeste	Suivre les fichiers générés, leurs catégories et leurs empreintes.	Identifier les fichiers que ProOWeb peut gérer.
Migration contrôlée	Comparer l'état attendu et l'état réel du projet.	Éviter l'écrasement silencieux du code personnalisé.
Sauvegardes	Copier les fichiers conflictuels avant remplacement.	Permettre la récupération des adaptations manuelles.
Template overrides	Réappliquer des personnalisations déclarées.	Réduire les modifications manuelles répétées.
Rapports	Décrire les créations, mises à jour, conflits et sauvegardes.	Rendre l'évolution auditable et compréhensible.

Ces mécanismes préparent une régénération contrôlée ; ils ne suffisent pas à en établir la robustesse pour toutes les personnalisations possibles. Leur comportement doit être éprouvé sur des évolutions plus complexes et interprété à la lumière des limites présentées dans le chapitre d'évaluation.

III.7. Conclusion

Ce chapitre a présenté l'approche d'ingénierie proposée et sa concrétisation dans le framework ProOWeb. Le couple BPMN–*spec-v1* alimente le modèle pivot, que les transformations M2M et M2T projettent vers un socle applicatif web métier structuré et traçable.

Les mécanismes de suivi des artefacts et de préservation des adaptations préparent la régénération contrôlée du projet cible. Le chapitre suivant examine la mise en œuvre technique de cette proposition et son application à un cas d'étude.

IV

CHAPITRE

MISE EN ŒUVRE ET EXPÉRIMENTATION

SOMMAIRE

IV.1 - Introduction 67
IV.2 - Environnement de mise en œuvre 68
IV.3 - Cas d'étude organisationnel 70
IV.4 - Déroulement complet du processus de commande client 72
IV.5 - Discussion et positionnement 82
IV.6 - Conclusion 83

IV.1. Introduction

Le chapitre précédent a présenté le framework proposé pour générer un socle applicatif web métier à partir d'un modèle BPMN complété par une spécification applicative. Il a notamment défini l'architecture générale de ProOWeb, le rôle de *spec-v1*, les transformations modèle-à-modèle et modèle-à-texte, ainsi que les mécanismes de préservation du code personnalisé. Nous passons maintenant de cette description d'ensemble à sa mise en œuvre, en observant comment ProOWeb produit un projet cible destiné à une organisation de commerce électronique.

Le chapitre suit donc le chemin emprunté par le framework : initialisation du projet, modélisation ou importation des processus, validation des spécifications enrichies, déploiement des contrats runtime et data, génération des artefacts backend/frontend et vérification des parcours obtenus. Cette progression permet de relier le modèle conceptuel du chapitre III au résultat produit dans l'application générée.

IV.2. Environnement de mise en œuvre

IV.2.1. Objectif de réalisation

La mise en œuvre de ProOWeb a consisté à rendre opérationnel le framework décrit au chapitre précédent. Les blocs conceptuels présentés dans l'architecture de la section III.3 sont ici traduits en composants réalisés : un noyau de génération, des mécanismes de validation, des contrats intermédiaires, des gabarits et un projet applicatif cible. Le framework reçoit des informations métier structurées, vérifie leur cohérence, construit des contrats intermédiaires et produit une base applicative exploitable. Cette base contient les éléments nécessaires pour exécuter un processus, afficher les tâches, appliquer les rôles, manipuler les données et suivre les fichiers générés.

IV.2.2. Choix technologiques

Le générateur a été séparé de l'application produite. Cette séparation reprend la distinction établie au chapitre III entre le noyau de génération et l'application web générée. ProOWeb s'exécute comme une application Node.js. Il fournit une interface de configuration, des services de gestion du workspace et des modules spécialisés dans la modélisation de processus : catalogue, validation de *spec-v1*, simulation, déploiement, génération des contrats runtime et gestion des fichiers produits.

Le projet cible généré, quant à lui, repose sur Spring Boot pour le backend, React/Vite pour l'interface web et PostgreSQL comme base de données cible. Le tableau XII récapitule ces choix de mise en œuvre.

TABLE XII – *Principaux choix de mise en œuvre de ProOWeb*

Éléments	Choix retenu	Rôle dans ProOWeb
Noyau du framework	Node.js / JavaScript	Lire le workspace, importer les modèles, valider *spec-v1*, construire les contrats et déclencher le déploiement.
Interface de configuration	Pages web ProOWeb	Initialiser le projet, accéder au tableau de bord et piloter certaines opérations de génération.
Backend généré	Spring Boot modulaire	Porter les modules système, organisation, sécurité, runtime de processus et ressources métier générées.
Frontend généré	React/Vite	Fournir les vues de travail, notamment le *Process Runtime Workbench* utilisé dans les tests.
Base de données cible	PostgreSQL avec Liquibase	Préparer la persistance et les scripts de migration nécessaires au projet produit.
Suivi des fichiers	Manifeste de génération	Identifier les fichiers générés et soutenir les régénérations contrôlées.
Vérification	Scripts et tests Cypress	Vérifier les parcours frontend liés aux processus déployés.

IV.2.3. Chaîne de génération

La figure 30 présente la chaîne de génération mise en œuvre dans ProOWeb. Elle reprend les étapes conceptuelles décrites dans la section III.5 : lecture des modèles, validation, construction d'une représentation intermédiaire, projection vers les artefacts et suivi de la génération. Elle montre ainsi le passage des modèles d'entrée vers les contrats intermédiaires, puis vers les artefacts applicatifs et le manifeste de traçabilité utilisé lors des régénérations.

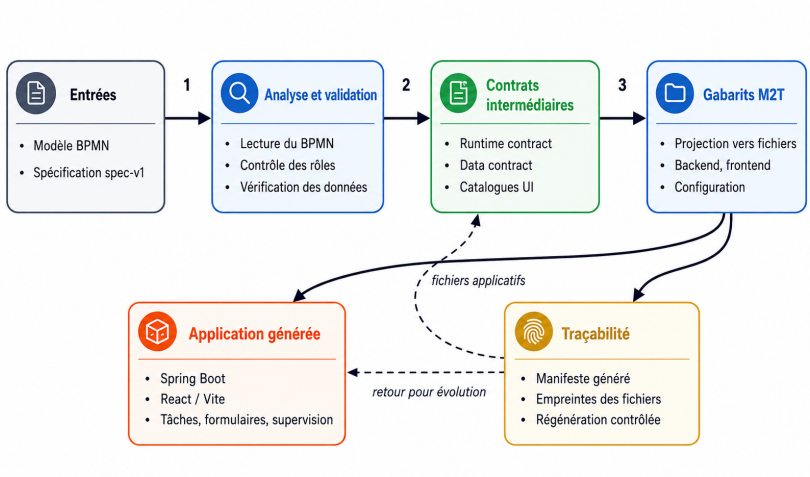


FIGURE 30 – *Chaîne de génération de ProOWeb*

IV.3. Cas d'étude organisationnel

IV.3.1. Présentation de l'organisation retenue

Le cas d'étude retenu concerne une organisation de commerce électronique. Cette organisation vend des produits à travers une boutique en ligne et s'appuie sur plusieurs pôles internes : prise en charge des commandes, gestion du stock, préparation logistique, service client et supervision commerciale. Son activité ne se résume donc pas à l'affichage d'un catalogue. Elle implique des décisions, des contrôles, des validations et des échanges entre acteurs avant qu'une commande soit clôturée ou qu'une réclamation soit traitée.

IV.3.2. Acteurs et responsabilités

Les acteurs retenus correspondent aux responsabilités courantes d'une organisation de commerce électronique. Le tableau XIII précise leur rôle dans les processus étudiés.

TABLE XIII – *Acteurs du cas d'étude e-commerce*

Acteur	Responsabilité principale	Interaction avec l'application générée
Client	Passer une commande ou déposer une réclamation de produit	Renseigne les informations nécessaires et suit l'état de ses demandes.
Service commercial	Suivre les commandes et arbitrer les cas nécessitant une décision	Consulte les commandes, confirme certaines décisions et suit les exceptions.
Gestionnaire de stock	Vérifier les disponibilités, préparer les commandes et enregistrer les mouvements	Reçoit les tâches liées au stock, à la préparation et à la réception des produits.
Responsable achats	Valider les réapprovisionnements et suivre les demandes fournisseur	Examine les besoins de stock et autorise les commandes d'achat.
Service client	Qualifier les réclamations de produit et proposer une solution	Traite les demandes après-vente et renseigne les décisions prises.
Administrateur	Configurer les rôles, produits, catégories et paramètres généraux	Maintient les données de base utilisées par les processus générés.

IV.3.3. Processus retenus

Trois processus ont été retenus pour représenter le fonctionnement de cette organisation. La commande client couvre le parcours qui va de la soumission de la commande à sa préparation. Le réapprovisionnement du stock intervient lorsqu'un produit devient insuffisant ou lorsqu'un responsable anticipe une rupture. La réclamation de produit prend en charge les demandes formulées après une vente. Ces processus partagent des données et des acteurs, mais ils répondent à des objectifs opérationnels différents. Le tableau XIV en donne une synthèse.

TABLE XIV – *Processus métier retenus dans le cas d'étude*

Processus	Objectif métier	Acteurs clés	Résultat observé
P1 : commande client	Recevoir, vérifier, préparer et clôturer une commande passée sur la boutique en ligne	Client, service commercial, gestionnaire de stock	Commande confirmée, préparée, annulée ou clôturée.
P2 : réapprovisionnement du stock	Détecter un besoin de stock, le faire valider et enregistrer la réception des produits	Gestionnaire de stock, responsable achats	Stock réapprovisionné ou demande refusée.
P3 : réclamation de produit	Traiter une réclamation liée à une commande et proposer une solution adaptée	Client, service client, service commercial, gestionnaire de stock	Réclamation clôturée avec décision tracée.

IV.3.4. Scénario nominal et scénarios alternatifs

Dans le scénario nominal, le client soumet sa commande. Le service commercial en vérifie les informations et la déclare recevable. Le gestionnaire de stock confirme ensuite la disponibilité des produits, prépare la commande et enregistre cette préparation. Le service commercial confirme enfin que la commande est prête, puis le client consulte la confirmation.

Deux scénarios alternatifs complètent ce parcours. Lorsque la commande n'est pas recevable, elle est annulée et le client en est informé. Lorsqu'elle est recevable mais que le stock est insuffisant, elle est placée en attente et une notification est adressée au client. Ces trois issues servent de base à l'exemple détaillé présenté ci-après.

IV.4. Déroulement complet du processus de commande client

Cette section suit le chemin effectivement pris en charge par ProOWeb : initialiser le projet, modéliser le processus, compléter le BPMN avec *spec-v1*, valider les deux entrées, construire le modèle pivot puis générer le socle applicatif. Le processus de commande client sert de fil conducteur à cette démonstration.

IV.4.1. Initialisation du projet

La génération commence par l'initialisation du projet cible. Cette étape fixe l'identité de l'application à produire : nom du projet, espace de travail, package de base, options d'authentification, profils d'exécution et paramètres généraux. Dans le cas d'étude, cette initialisation a produit le workspace E-commerce Application, avec le package de base `com.prooweb.generated`, un backend Spring Boot, un frontend React et une base de données cible PostgreSQL. Le processus `commande-client` est ensuite choisi comme exemple de conception et d'exécution. La figure 31 présente l'écran d'initialisation obtenu dans le cas d'étude.

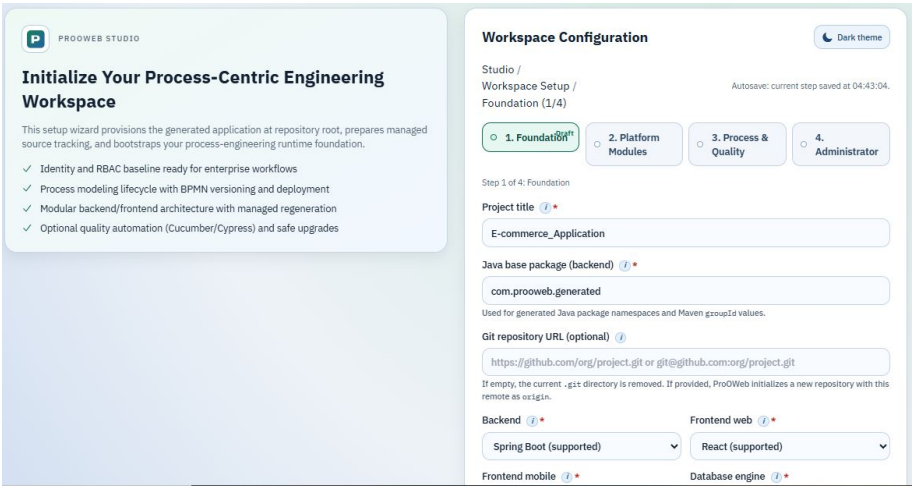


FIGURE 31 – *Initialisation du projet* `E-commerce_Application`

IV.4.2. Modélisation du processus

La commande client est modélisée dans le fichier `Commande_Client.bpmn`. Le processus réunit trois acteurs, représentés par les *lanes* Client, Service commercial et Gestionnaire de stock. Il comprend un événement de début, douze tâches utilisateur, deux passerelles exclusives et trois événements de fin. La première passerelle décide si la commande est recevable ; la seconde vérifie la disponibilité du stock. Les flux de séquence relient ces éléments et conduisent respectivement vers la confirmation, l'annulation ou la mise en attente. BPMN décrit ainsi l'ordre des activités, les responsabilités et les décisions du processus.

La figure 32 montre le modèle importé dans l'environnement de ProOWeb.

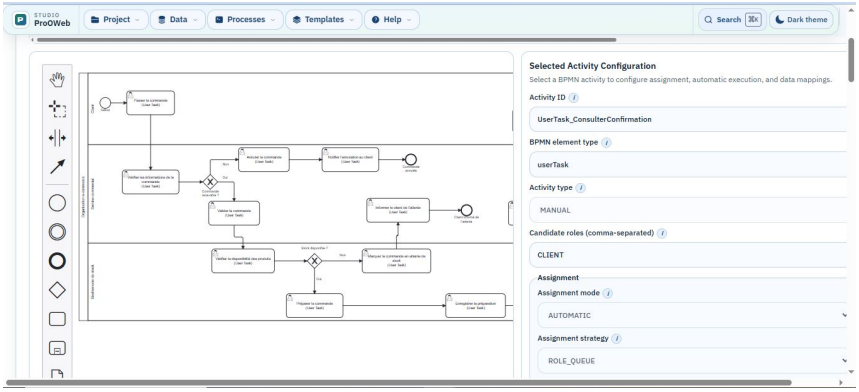


FIGURE 32 – *Modélisation du processus de commande client dans ProOWeb*

IV.4.3. Enrichissement du BPMN avec *spec-v1*

Le BPMN est complété par le fichier `commande-client.spec-v1.json`. Cet enrichissement respecte la grammaire formelle $G_{\text{spec-v1}} = (N, T, P, S)$ définie par l'équation III.1. La figure 33 illustre cette spécification pour l'activité « Passer la commande ».

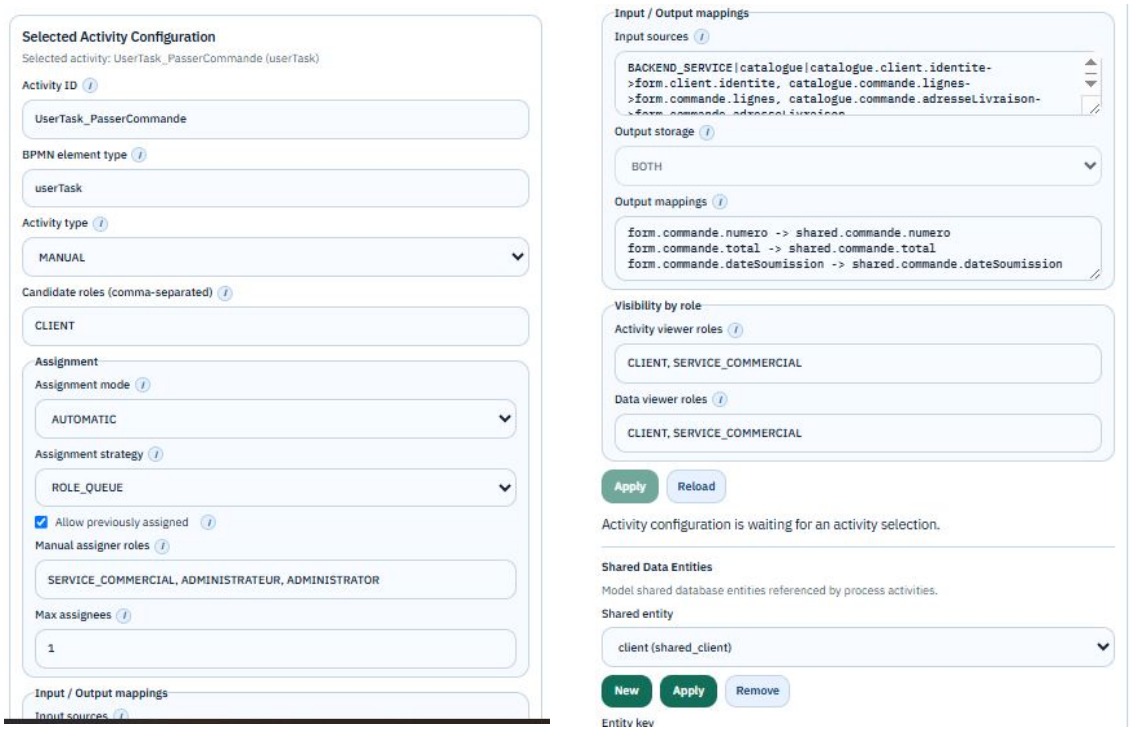


FIGURE 33 – *Spécification ajoutée à l'activité « Passer la commande »*

IV.4.4. Analyse et validation

ProOWeb analyse conjointement le BPMN et la spécification avant de poursuivre. Il vérifie que l'identifiant du processus correspond, que chaque activité

enrichie existe dans le diagramme et que les codes de rôles respectent le format attendu. Il contrôle également les sources d'entrée, les sorties, les destinations des mappings, la présence des règles de visibilité et les références aux entités partagées. Une tâche manuelle doit notamment posséder au moins un rôle candidat. Lorsqu'une erreur bloquante est détectée, la construction des contrats est interrompue ; la génération ne part donc pas d'un couple de modèles incohérent.

Le listing IV.1 reproduit le rapport obtenu en appliquant le validateur de ProOWeb au bloc `specification` de `v1.json`, avec le BPMN déployé `v1.bpmn` et le mode strict. Les valeurs correspondent à la version 1 présente dans le workspace.

```
Action: specification/validate
Model: commande-client
Version: 1
Schema version: 1
Activities: 12
Manual activities: 12
Automatic activities: 0
Validation: valid
Errors: 0
Warnings: 0
```

Listing IV.1 – *Résultat de la validation BPMN–spec-v1 de la commande client*

Ce résultat indique que les douze activités détectées dans le BPMN disposent d'une représentation acceptée par le validateur et qu'aucune erreur ni aucun avertissement n'est produit pour cette version. Le contrôle de l'existence effective des rôles dans un annuaire organisationnel externe ne fait toutefois pas partie de cette vérification.

IV.4.5. Construction du modèle pivot

Après validation, la transformation M2M rassemble les informations des deux modèles dans le pivot ProOWeb. Le *ProcessModel* porte l'identité et la version du processus. Les *ActivityModel* représentent ses tâches ; les *RolePolicy* conservent l'affectation et la visibilité ; les *FormModel* décrivent les données attendues et produites. Le *RuntimeContract* organise les nœuds et les transitions, tandis que le *DataContract* formalise les mappings, le stockage et les données partagées. Les *ArtifactTrace* maintiennent enfin le lien entre les éléments sources et les artefacts qui seront générés. Cette représentation intermédiaire évite aux générateurs de manipuler directement et séparément le BPMN et *spec-v1*.

Dans l'implémentation, le pivot est matérialisé par plusieurs contrats plutôt que par un fichier JSON unique. Le listing IV.2 rassemble un extrait du contrat runtime

et du contrat data de la version 1. Une seule activité est conservée afin de faire apparaître la structure sans reproduire les fichiers complets.

```json
{
  "processModel": {
    "modelKey": "commande-client",
    "versionNumber": 1
  },
  "activityModel": {
    "activityId": "UserTask_AnnulerCommande",
    "candidateRoles": ["SERVICE_COMMERCIAL"],
    "assignment": {
      "mode": "AUTOMATIC",
      "strategy": "ROLE_QUEUE"
    },
    "formModel": {
      "outputs": ["commande.statut", "commande.motifAnnulation"]
    }
  },
  "runtimeContract": {
    "transition": {
      "id": "Flow_05",
      "source": "UserTask_AnnulerCommande",
      "target": "UserTask_NotifierAnnulationClient"
    }
  },
  "dataContract": {
    "storage": "BOTH",
    "mapping": {
      "from": "form.commande.statut",
      "to": "shared.commande.statut"
    }
  },
  "traceSimplifiee": {
    "sourceBpmn": "UserTask_AnnulerCommande",
    "contracts": ["v1.runtime.json", "v1.data.json"]
  }
}
```

Listing IV.2 – *Vue synthétique du modèle pivot de la commande client*

Le bloc `processModel` identifie la version du processus. Le bloc `activity Model` réunit l'activité, son rôle, son affectation et les sorties du formulaire. Le contrat runtime conserve la transition vers la notification du client, tandis que le contrat de données projette le statut saisi vers la donnée partagée `commande`. Le dernier bloc est une trace de lecture simplifiée, reconstruite à partir des identifiants et des chemins versionnés ; il ne constitue pas un champ supplémentaire du contrat

brut. Ces informations donnent aux générateurs une entrée stable pour produire les services, les formulaires et les règles de progression.

Le listing constitue ici la preuve textuelle retenue pour le pivot. Il relie explicitement le modèle de processus aux contrats `runtime` et `data`, sans reproduire une seconde fois leur contenu complet.

IV.4.6. Génération des artefacts applicatifs

La transformation M2T projette le modèle pivot dans le workspace. Pour la commande client, ProOWeb conserve le BPMN dans `v1.bpmn` et la spécification normalisée dans `v1.json`. Il produit parallèlement les contrats runtime et data propres à cette version. Ces contrats alimentent les contrôleurs et services backend, les composants frontend, les formulaires d'activité, les règles d'accès, les fichiers de configuration et les catalogues communs. Le manifeste enregistre les chemins et les empreintes des fichiers gérés afin de soutenir les régénérations ultérieures. Le résultat constitue un socle applicatif organisé, destiné à être vérifié et complété par le développeur.

Le listing IV.3 reprend les artefacts effectivement observés dans le workspace. Il est limité aux fichiers utiles à la commande client ; les chemins intermédiaires ont été raccourcis, sans modifier les noms des fichiers.

```
E-commerce_Application/
|-- .prooweb-managed.json
`-- src/
    |-- backend/springboot/
    |   |-- prooweb-application/src/main/resources/
    |   |   |-- processes/
    |   |   |   |-- runtime-catalog.json
    |   |   |   |-- data-catalog.json
    |   |   |   `-- commande-client/
    |   |   |       |-- v1.bpmn
    |   |   |       |-- v1.json
    |   |   |       |-- v1.runtime.json
    |   |   |       `-- v1.data.json
    |   |   `-- db/changelog/changesets/
    |   |       |-- 900-process-shared-data.generated.yaml
    |   |       `-- 901-process-lane-roles.generated.yaml
    |   |-- system/system-domain/.../
    |   |   `-- CommandeClientProcessV1Spec.java
    |   |-- system/system-application/.../
    |   |   `-- ProcessRuntimeEngineService.java
    |   |-- system/system-infrastructure/.../
    |   |   |-- SharedCommandeEntity.java
    |   |   `-- SharedCommandeJpaRepository.java
    |   `-- gateway/.../ProcessRuntimeController.java
```

```
    `-- frontend/web/react/
        |-- src/modules/processes/
        |   |-- generatedProcessRegistry.js
        |   `-- commande-client/
        |       |-- ProcessCommandeClientV1Descriptor.js
        |       |-- ProcessCommandeClientV1RuntimeContract.js
        |       |-- ProcessCommandeClientV1DataContract.js
        |       `-- GeneratedCommandeClientV1ProcessPage.jsx
        `-- cypress/e2e/process-commande-client-v1.cy.js
```

Listing IV.3 – *Arborescence simplifiée des artefacts générés pour la commande client*

Cette arborescence montre que la transformation M2T relie les ressources versionnées, le runtime backend, la persistance des données partagées, les composants React et le scénario de test propre au processus. Le listing suffit ici à identifier les principaux artefacts produits et leur position dans le workspace.

IV.4.7. Observation de l'application web générée

L'observation ne porte pas seulement sur l'ouverture du portail ou l'affichage d'un formulaire. Elle suit une même instance du processus `commande-client`, depuis sa création par un client jusqu'à sa clôture, en faisant intervenir successivement les rôles prévus par le modèle. Le frontend généré a été lancé localement. Le runtime a été remplacé, pour cette observation, par des réponses contrôlées qui conservent l'identifiant, l'état et la chronologie de l'instance entre les changements d'utilisateur. Ce dispositif permet de vérifier le parcours frontend et la projection des rôles sans assimiler le résultat à un test d'intégration avec le backend et PostgreSQL.

Le scénario nominal observé comprend huit activités manuelles : le client passe la commande ; le service commercial vérifie les informations puis valide la demande ; le gestionnaire de stock contrôle la disponibilité, prépare la commande et enregistre cette préparation ; le service commercial confirme ensuite la commande préparée ; enfin, le client consulte la confirmation. Toutes ces étapes portent sur l'instance `cmd-memoire-2026-001`.

Après la connexion de `yuri.client` avec le rôle `CLIENT`, la page générée autorise le démarrage du processus. Le formulaire rassemble l'identité du client, l'adresse de livraison, le produit, la quantité et le montant calculé. La figure 34 montre la commande renseignée avant sa soumission.

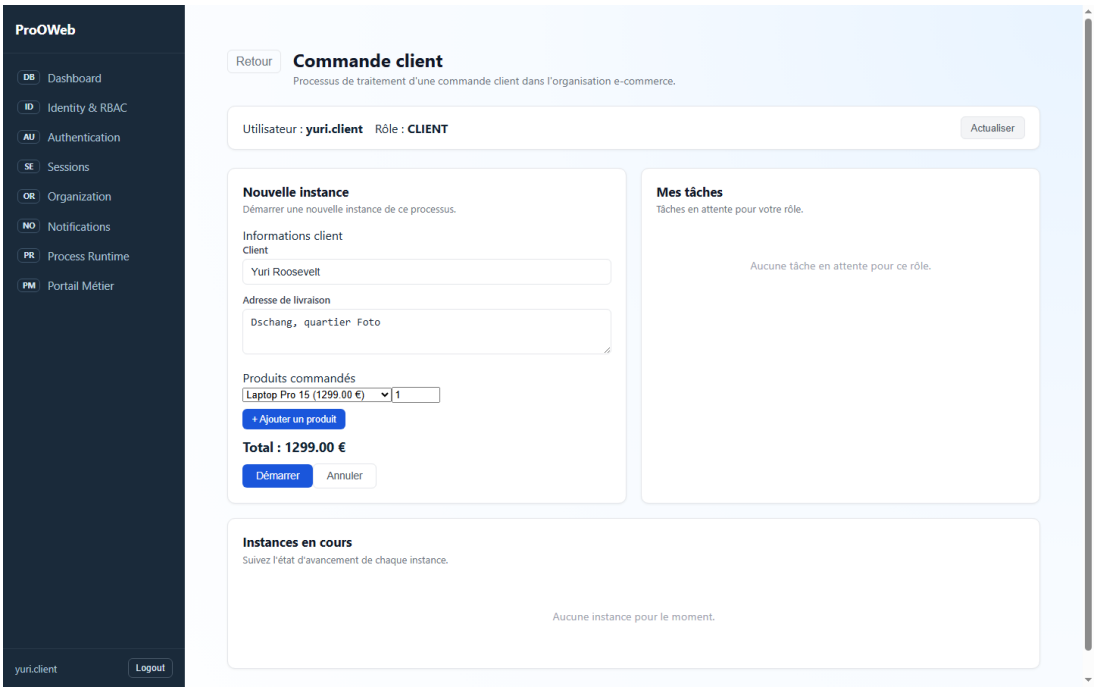


FIGURE 34 – *Saisie d'une commande par l'utilisateur client*

La soumission crée l'instance et la première tâche. La vue de détail de la figure 35 conserve l'utilisateur à l'origine du démarrage et les premiers événements de la chronologie. L'identifiant affiché sert ensuite de repère lors des interventions des autres rôles.

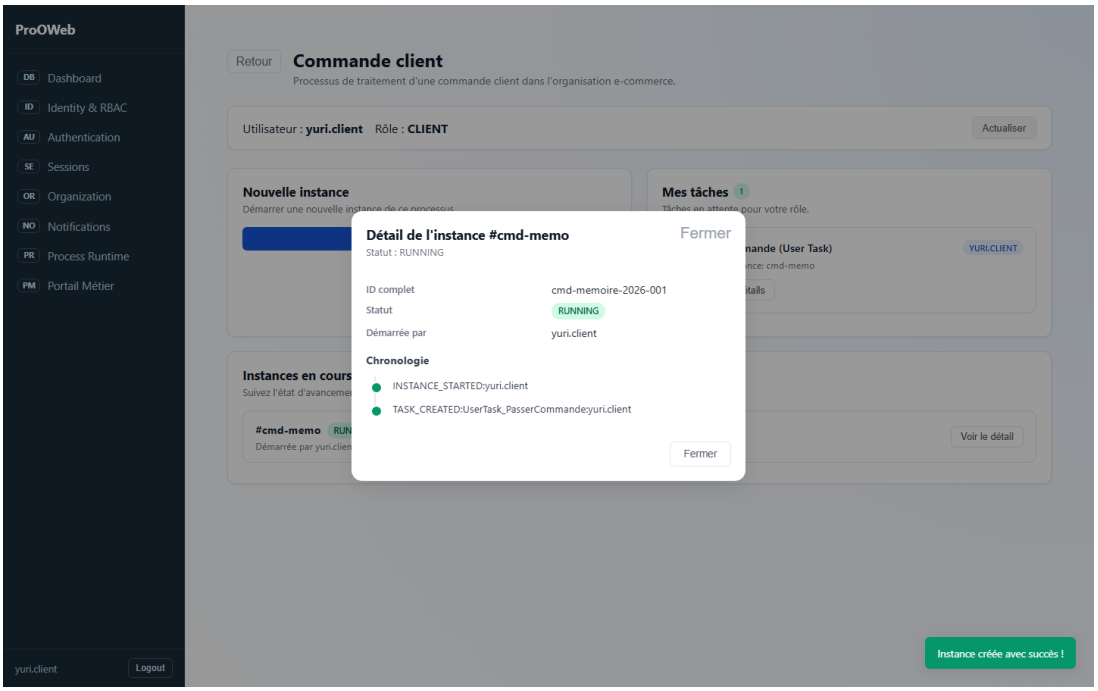


FIGURE 35 – *Instance de commande démarrée par le client*

Une fois l'activité « Passer la commande » terminée, l'utilisateur `yuri.commercial` retrouve la même instance dans son espace. Il ne peut pas en démarrer une nouvelle avec son rôle, mais la tâche « Vérifier les informations de la commande » lui est proposée. Cette séparation, visible à la figure 36, traduit dans l'interface les droits de démarrage et les rôles candidats issus de *spec-v1*.

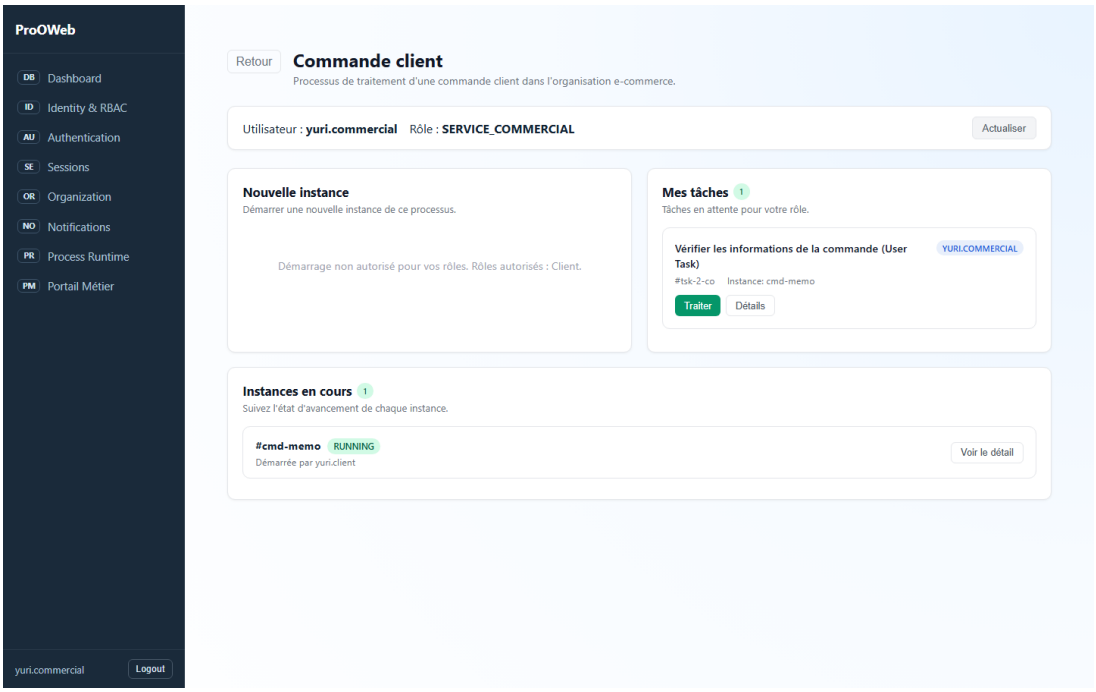


FIGURE 36 – *Reprise de l'instance par le service commercial*

Après la vérification et la validation commerciales, la tâche suivante devient visible pour `yuri.stock`. La figure 37 montre que l'instance reste attribuée à son initiateur, tandis que l'activité « Vérifier la disponibilité des produits » est présentée au gestionnaire de stock. Celui-ci poursuit ensuite le parcours par la préparation de la commande et l'enregistrement de cette préparation.

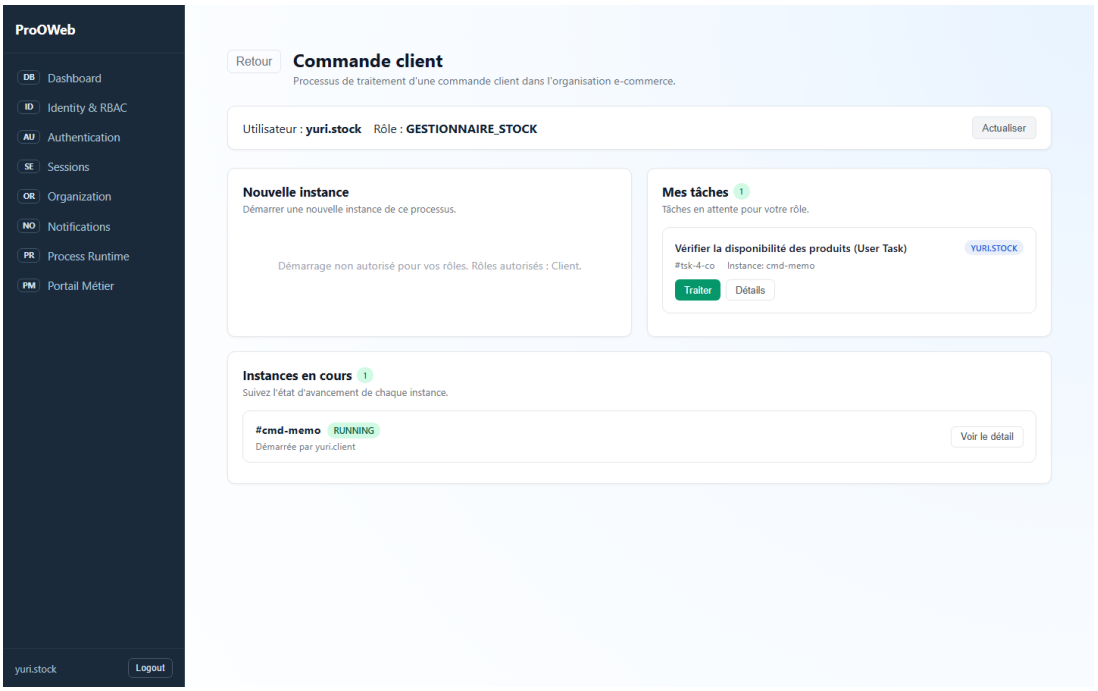


FIGURE 37 – *Reprise de l'instance par le gestionnaire de stock*

Le service commercial confirme alors la commande préparée, puis le client traite la dernière activité de consultation. La figure 38 présente l'état `COMPLETED` et la chronologie complète. Celle-ci relie chaque création et chaque achèvement de tâche à l'utilisateur concerné, jusqu'à l'événement `INSTANCE_COMPLETED`.

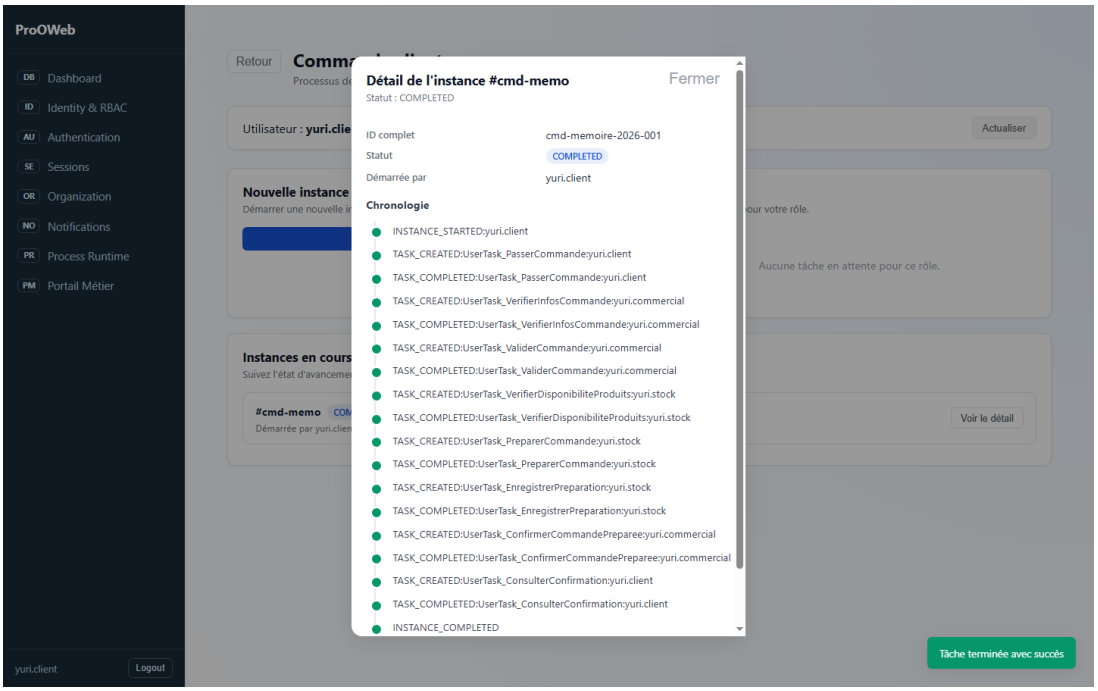


FIGURE 38 – *Chronologie finale de l'instance de commande client*

Ce parcours constitue une preuve d'observation de l'interface générée : les vues, les formulaires, les restrictions de démarrage, les files de tâches et la chronologie restent cohérents lors des changements de rôle. Il ne démontre pas encore l'authentification JWT ni la persistance réelle de l'instance. Dans la version exécutée, l'interface métier transmet encore une authentification Basic, alors que le backend généré protège les points d'entrée runtime par jeton JWT. L'intégration de ces deux mécanismes doit donc être alignée avant de qualifier ce scénario de test de bout en bout sur l'ensemble du socle généré.

IV.5. Discussion et positionnement

Discussion générale

Le cas d'étude e-commerce montre que ProOWeb établit une continuité entre des processus exprimés en BPMN et un socle applicatif web métier structuré. Pour les trois processus étudiés, le BPMN est complété par *spec-v1*, validé, puis projeté dans des contrats *runtime* et *data* avant la production des artefacts applicatifs. L'observation du processus de commande client complète cette inspection : une même instance est reprise successivement par le client, le service commercial et le gestionnaire de stock jusqu'à la fin du parcours nominal. Le résultat principal ne réside donc pas dans la seule présence de fichiers, mais dans le maintien d'un lien explicite entre le processus, son enrichissement, le modèle pivot, les artefacts générés et les interfaces observées.

Forces du framework

La première force du framework ProOWeb tient à la séparation des responsabilités. BPMN conserve le déroulement du processus, tandis que la grammaire attribuée de *spec-v1* structure les rôles, les données, les règles d'affectation, les mappings, la visibilité et les modalités d'exécution. La validation croisée empêche la promotion des incohérences bloquantes. Le modèle pivot stabilise ensuite les informations acceptées dans une représentation commune, exploitée successivement par les transformations M2M et M2T. Les contrats intermédiaires, les identifiants conservés et le manifeste rendent enfin l'origine des artefacts observable et préparent leur régénération. Cette organisation produit un projet cible lisible et extensible sans confondre le framework, le socle généré et les choix techniques retenus pour le prototype.

Limites observées et positionnement

Ces résultats restent circonscrits au périmètre observé. Le cas d'étude concerne une seule organisation e-commerce et ne permet pas de généraliser les résultats à tous les domaines métier. Le parcours multi-rôles vérifie les formulaires, les vues, les files de tâches, les restrictions de démarrage et la chronologie dans un environnement contrôlé. Il ne constitue pas encore un test d'intégration du frontend, du backend et de PostgreSQL : l'interface observée emploie une authentification Basic, alors que les points d'entrée runtime générés attendent un jeton JWT. La persistance réelle, les services externes, les branches alternatives des processus et les interactions entre plusieurs instances restent également à éprouver. Enfin, la préservation du code personnalisé doit être vérifiée sur des régénérations successives plus importantes.

ProOWeb ne se positionne donc ni comme un BPMS classique, ni comme une plateforme low-code généraliste, ni comme un substitut au développeur. Son résultat est un socle applicatif web métier fonctionnel, structuré et traçable, que le développeur doit vérifier, adapter et compléter selon les règles et les contraintes de l'organisation.

IV.6. Conclusion

Ce chapitre a présenté la mise en œuvre de ProOWeb et son application à une organisation de commerce électronique. Il a montré comment les modèles BPMN enrichis sont importés, validés, transformés en contrats, puis projetés dans un socle applicatif web métier.

Le cas pratique porte sur trois processus appartenant au même contexte : commande client, réapprovisionnement du stock et réclamation de produit. Les ressources générées confirment leur coexistence dans un socle commun et traçable, tandis que le scénario de commande restitue un parcours nominal multi-rôles dans un environnement contrôlé. Cette évaluation démontre la faisabilité de la chaîne dans le périmètre retenu, mais pas encore son aptitude à un usage en production. Des essais intégrant réellement l'authentification, le backend, la persistance et la régénération après personnalisation restent nécessaires.

CONCLUSION GÉNÉRALE

SOMMAIRE

Bilan du travail 84
Quelques perspectives 85

Bilan du travail

Ce mémoire est parti d'un décalage concret : BPMN décrit avec précision le déroulement d'un processus, mais ne porte pas, à lui seul, toutes les décisions nécessaires à une application web métier. Les formulaires, les données persistées, les affectations, les règles de visibilité, la sécurité et la cohabitation entre code généré et code personnalisé restent à préciser. La problématique consistait donc à produire un socle applicatif web à partir de modèles BPMN enrichis tout en maintenant la cohérence entre ces dimensions et les évolutions futures du projet.

ProOWeb répond à ce problème par une chaîne d'ingénierie explicite. Le BPMN reste la référence du workflow ; la spécification *spec-v1* décrit les informations applicatives absentes du diagramme. Leur validation croisée alimente un modèle pivot, matérialisé par des contrats runtime et data. Des transformations modèle-à-modèle puis modèle-à-texte projettent ces contrats vers un backend Spring Boot, un frontend React, des ressources de données, des règles d'accès et des composants d'exécution. Enfin, les clés de processus, les versions, les contrats et le manifeste maintiennent un lien observable entre les modèles et les fichiers produits.

Les principales contributions de ce travail sont les suivantes :

— une spécification enrichie, formalisée par une grammaire et des contraintes de validité, qui complète BPMN sans transformer le diagramme en description technique de toute l'application ;

— une architecture séparant les sources, les validations, les contrats intermédiaires, les générateurs et le projet cible ;
— des algorithmes explicites de validation, de construction des contrats, de génération et de régénération couvrant le runtime, le backend, le frontend, les données et la sécurité ;
— un dispositif de traçabilité et de régénération fondé notamment sur un manifeste, des empreintes et des points d'extension ;
— une mise à l'épreuve sur trois processus liés d'une organisation de commerce électronique.

Le cas d'étude fournit des résultats vérifiables, mais circonscrits. Les trois processus disposent chacun d'un BPMN, d'une spécification normalisée, d'un contrat runtime et d'un contrat data dans un même workspace. Des catalogues globaux, des classes Java, des composants React, des scénarios Cucumber et trois tests Cypress complètent ces ressources. Cette observation confirme la production d'un socle multi-processus cohérent et traçable. Elle ne démontre pas encore toutes les propriétés d'une application en production : les scénarios Cypress simulent le backend et ne couvrent que deux ou trois activités par processus ; la persistance réelle, la sécurité, la charge, les interactions inter-processus et la régénération après personnalisation demandent des essais supplémentaires.

Le résultat doit donc être formulé avec mesure. ProOWeb ne remplace ni l'analyse métier ni le développeur. Il réduit la distance entre le processus et le projet logiciel en produisant un socle fonctionnel, traçable et extensible. L'équipe de développement reste responsable des règles non modélisées, des intégrations externes, de l'expérience utilisateur et de la validation dans son environnement.

Quelques perspectives

Les prolongements prioritaires découlent directement de ces limites :
— exécuter des campagnes d'intégration sans API simulée, avec PostgreSQL, authentification, autorisations et services externes ;
— tester systématiquement les branches BPMN, les transitions, les rôles, les formulaires et les règles de visibilité ;
— évaluer la régénération sur des fichiers effectivement personnalisés et mesurer les conflits, sauvegardes et migrations produits ;
— assister la construction conjointe du BPMN et de *spec-v1*, notamment par des diagnostics compréhensibles par le concepteur ;

— reproduire l'étude sur d'autres domaines, des modèles plus complexes et d'autres piles techniques.

En définitive, ce travail montre l'intérêt d'une continuité explicite entre modèle métier, spécification enrichie, contrats intermédiaires, artefacts générés et évolution contrôlée. C'est dans cette continuité, plus que dans la seule quantité de code produite, que se situe la contribution de ProOWeb.

BIBLIOGRAPHIE

[1] Marlon Dumas, Marcello La Rosa, Jan Mendling, and Hajo A. Reijers. *Fundamentals of Business Process Management*. Springer, Berlin, Heidelberg, 2 edition, 2018.

[2] Mathias Weske. *Business Process Management : Concepts, Languages, Architectures*. Springer, Berlin, Heidelberg, 2 edition, 2012.

[3] Object Management Group. Business Process Model and Notation (BPMN), Version 2.0.2. Technical report, Object Management Group, 2013. Consulté le 22 mai 2026.

[4] Object Management Group. Model Driven Architecture Guide, Revision 2.0. Technical report, Object Management Group, 2014. Guide officiel de la Model Driven Architecture.

[5] Jean-Marc Jézéquel, Benoît Combemale, and Didier Vojtisek. *Ingénierie dirigée par les modèles : des concepts à la pratique*. Ellipses, Paris, 2012.

[6] Eduardo Díaz, José Ignacio Panach, Silvia Rueda, and Oscar Pastor. Towards a method to generate GUI prototypes from BPMN. In *2018 12th International Conference on Research Challenges in Information Science (RCIS)*, pages 1–12. IEEE, 2018.

[7] Abir Sajji, Yassine Rhazali, and Youssef Hadi. A methodology for transforming bpmn to ifml into mda. *Bulletin of Electrical Engineering and Informatics*, 11(5) :2773–2782, 2022.

[8] Abir Sajji, Yassine Rhazali, and Youssef Hadi. Graphical user interfaces generation from bpmn via ifml up to psm level. *International Journal of Advanced Computer Science and Applications*, 14(2) :419–431, 2023.

[9] Imke Drave, Judith Michael, Erik Müller, Bernhard Rumpe, and Simon Varga. Model-driven engineering of process-aware information systems. *SN Computer Science*, 3(479), 2022.

[10] Michael Hammer. Reengineering work : Don't automate, obliterate. *Harvard Business Review*, 1990.

[11] Wil M. P. van der Aalst, Arthur H. M. ter Hofstede, and Mathias Weske. *Business Process Management : A Survey*, volume 2678 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 2003.

[12] Marlon Dumas, Marcello La Rosa, Jan Mendling, and Hajo A. Reijers. *Fundamentals of Business Process Management*, volume 1. Springer, Berlin, 2013.

[13] Bonitasoft. L'essentiel du bpmn 2 : Comment réconcilier les besoins de la modélisation métier et informatique pour la gestion des processus métiers. Guide technique, Bonitasoft, s.d.

[14] Peter Pichler, Barbara Weber, Stefan Zugal, Jakob Pinggera, Jan Mendling, and Hajo A. Reijers. Imperative versus declarative process modeling languages : An empirical investigation. In *Business Process Management Workshops*, volume 99 of *Lecture Notes in Business Information Processing*, pages 383–394. Springer, 2011.

[15] Mark von Rosing, Stephen White, Fred Cummins, and Henk de Man. Business process model and notation – bpmn. In Mark von Rosing, Henrik von Scheel, and August-Wilhelm Scheer, editors, *The Complete Business Process Handbook : Body of Knowledge from Process Modeling to BPM, Volume I*, pages 429–453. Morgan Kaufmann/Elsevier, Waltham, MA, 2015.

[16] Samir Meghzili. Ingénierie des logiciels : chapitre ingénierie dirigée par les modèles, 2022. Support de cours.

[17] Object Management Group. Meta object facility (mof). https://www.omg.org/spec/MOF/, s.d. Spécification officielle OMG.

[18] Object Management Group. Mof query/view/transformation (qvt). https://www.omg.org/spec/QVT/, s.d. Spécification officielle OMG.

[19] Ian Jacobs and Norman Walsh. Architecture of the world wide web, volume one. Technical report, World Wide Web Consortium, 2004. Consulté le 22 mai 2026.

[20] Roy T. Fielding, Mark Nottingham, and Julian Reschke. HTTP Semantics. Technical Report RFC 9110, Internet Engineering Task Force, 2022. Consulté le 22 mai 2026.

[21] MDN Web Docs. How the web works. https://developer.mozilla.org/en-US/docs/Learn_web_development/Getting_started/Web_standards/How_the_web_works, 2026. Consulté le 22 mai 2026.

[22] Avraham Leff and James T. Rayfield. Web-application development using the model/view/controller design pattern. In *Proceedings Fifth IEEE International Enterprise Distributed Object Computing Conference*, pages 118–127. IEEE, 2001.

[23] MDN Web Docs. Spa : Single-page application. https://developer.mozilla.org/en-US/docs/Glossary/SPA, 2026. Consulte le 09 juin 2026.

[24] Roy Thomas Fielding. *Architectural styles and the design of network-based software architectures*. University of California, Irvine, 2000.

[25] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. *Computer*, 29(2) :38–47, February 1996.

[26] Camunda. Camunda Modeler and BPMN Process Orchestration Documentation. https://docs.camunda.io/docs/components/modeler/about-modeler/, 2026. Documentation officielle consultée en 2026.

[27] Barbara Kitchenham and Stuart Charters. Guidelines for performing systematic literature reviews in software engineering. Technical report, Keele University and Durham University, 2007.

[28] Matthew J. Page, Joanne E. McKenzie, Patrick M. Bossuyt, Isabelle Boutron, Tammy C. Hoffmann, Cynthia D. Mulrow, Larissa Shamseer, Jennifer M. Tetzlaff, Elie A. Akl, Sue E. Brennan, et al. The prisma 2020 statement : an updated guideline for reporting systematic reviews. *BMJ*, 372 :n71, 2021.

[29] Flowable. Flowable Open Source BPMN Documentation. https://www.flowable.com/open-source/docs/bpmn/ch02-GettingStarted/, 2026. Documentation officielle consultée en 2026.

[30] jBPM. jBPM Documentation : BPMN 2.0 Workflow Engine. https://www.jbpm.org/learn/documentation.html, 2026. Documentation officielle consultée en 2026.

[31] Activiti. Activiti BPMN Workflow Engine Documentation. https://www.activiti.org/userguide/, 2026. Documentation officielle consultée en 2026.

[32] Wil M. P. van der Aalst. Business process management : a comprehensive survey. *ISRN Software Engineering*, 2013 :507984, 2013.

[33] Eike Schäffer, Marvin Schobert, Tobias Reichenstein, Andreas Selmaier, Volker Stiehl, Matthias Herhoffer, Michael Mala, and Jörg Franke. Reference architecture and agile development method for a process-driven web platform based on the BPMN-standard and process engines. *Procedia CIRP*, 103 :146–151, 2021.

[34] Zef Hemel, Ruben Verhaaf, and Eelco Visser. Webworkflow : An object-oriented workflow modeling language for web applications. In *Proceedings of the 11th International Conference on Model Driven Engineering Languages and Systems (MODELS 2008)*, volume 5301 of *Lecture Notes in Computer Science*, 2008.

[35] Ulisses Telemaco Neto, Toacy Oliveira, Raquel Pillat, Paulo Alencar, Don Cowan, and Glaucia Melo. Akip process automation platform : A framework for the development of process-aware web applications. In *Proceedings of the International Conference on Enterprise Information Systems*, 2022.

[36] Abir Sajji, Yassine Rhazali, and Youssef Hadi. From interaction flow modeling language to symfony : an automated model transformation methodology. *Bulletin of Electrical Engineering and Informatics*, 14(5) :3765–3777, 2025.

[37] Naziha Laaz, Nassim Kharmoum, and Samir Mbarki. Combining domain ontologies and bpmn models at the cim level to generate ifml models. In *Procedia Computer Science*, volume 170, pages 851–856, 2020.

[38] Daniel Pietruck and Ulrike Steffens. Generating REST APIs for workflow engines with process and data models. In *Proceedings of the 12th Central European Workshop on Services and their Composition*, volume 2825 of *CEUR Workshop Proceedings*, 2020.

[39] Jean Pierre Alfonso Hoyos and Felipe Restrepo-Calle. Automatic source code generation for web-based process-oriented information systems. In *Proceedings of ENASE*, 2017.

[40] Burak Uyanik and Ahmet Sayar. Developing web-based process management with automatic code generation. *Applied Sciences*, 13(21) :11737, 2023.

[41] Brice Arléon Zemtsop Ndadji. Vers une approche basée sur l'ingénierie dirigée par les modèles, de gestion des workflows dans une application web.

Mémoire de master en informatique, Université de Dschang, Dschang, Cameroun, 2022.

[42] Eugene Syriani, Lechanceux Luhunu, and Houari Sahraoui. Systematic mapping study of template-based code generation. *Computer Languages, Systems & Structures*, 52 :43–62, 2018.

[43] Arianna Fedeli, Fabrizio Fornari, Andrea Polini, Barbara Re, Victoria Torres, and Pedro Valderas. Flobp : a model-driven approach for developing and executing iot-enhanced business processes. *Software and Systems Modeling*, 23 :1217–1246, 2024.

[44] Rubén Salado-Cid, Antonio Vallecillo, Kamran Munir, and José Raúl Romero. Swel : A domain-specific language for modeling data-intensive workflows. *Business & Information Systems Engineering*, 66 :137–160, 2024.

[45] Timo Greifenberg, Katrin Hölldobler, Carsten Kolassa, Markus Look, Pedram Mir Seyed Nazari, Klaus Müller, Antonio Navarro Pérez, Dimitri Plotnikov, David Reiss, Andreas Roth, Bernhard Rumpe, Martin Schindler, and Andreas Wortmann. A comparison of mechanisms for integrating handwritten and generated code for object-oriented programming languages. In *Proceedings of the 3rd International Conference on Model-Driven Engineering and Software Development*, pages 74–85, 2015.

[46] Khandoker Rahad, Omar Badreddin, and Syed Mohammad Reza. The human in model-driven engineering loop : A case study on integrating handwritten code in model-driven engineering repositories. *Software : Practice and Experience*, 51(6) :1308–1321, 2021.

[47] Bojan Dragaš, Nikola Todorović, Teodora Rajačić, and Goran Milosavljević. SeamlessMDD : Framework for seamless integration of generated and handwritten code. In *Web Engineering – 24th International Conference, ICWE 2024*, pages 163–177. Springer, 2024.

[48] Kristaps Rokis and Marite Kirikova. Exploring low-code development : A comprehensive literature review. *Complex Systems Informatics and Modeling Quarterly*, 36 :68–86, 2023.

[49] Małgorzata Pańkowska. Low code development cycle investigation, 2024.

[50] Andrea Delgado, Daniel Calegari, Ana Arrigoni, Alejandro Zunino, and Rodrigo Sosa. Model-driven management of BPMN-based business process families. *Software and Systems Modeling*, 2022.

[51] Paul Klint, Ralf Lämmel, and Chris Verhoef. Toward an engineering discipline for grammarware. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 14(3) :331–380, 2005.

[52] Eric Badouel, Loïc Hélouët, Georges-Edouard Kouamou, Christophe Morvan, and Nsaibirni Robert Fondze Jr. Active workspaces : distributed collaborative systems based on guarded attribute grammars. *ACM SIGAPP Applied Computing Review*, 15(3) :6–34, 2015.